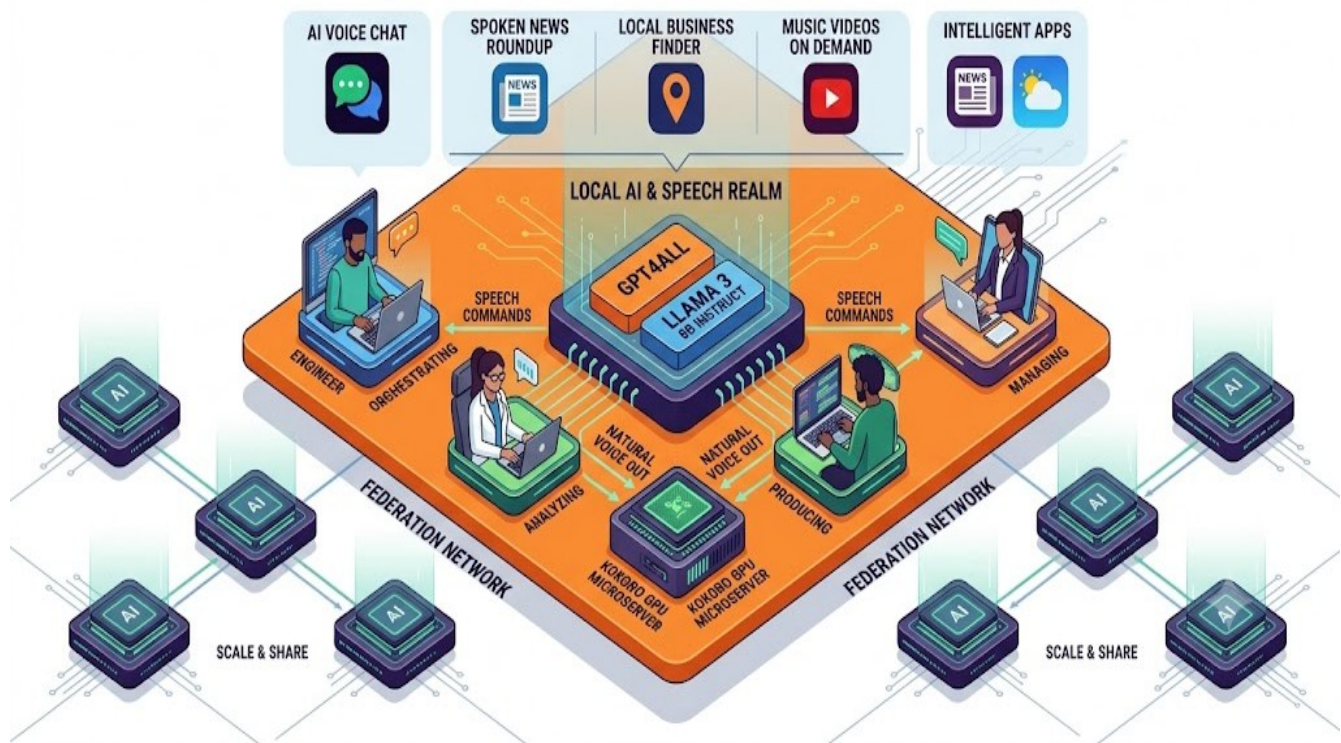


RENT-A-HAL:

Free Federated Multi-User Speech AI for RTX Gaming Cards

RENT-A-HAL

LOCAL. MULTI-USER. FEDERATED. '26 SOHO/API EDITION.



RENT-A-HAL: Applied Algebraic Design in Action

by: j p ames with supergrok and ms. gemini

© Copyright 2026, Jim Ames. All rights reserved. \

need rent-a-hal Support????? email jimpames@gmail.com

By n2nhu labs and <https://notcloudai.com>

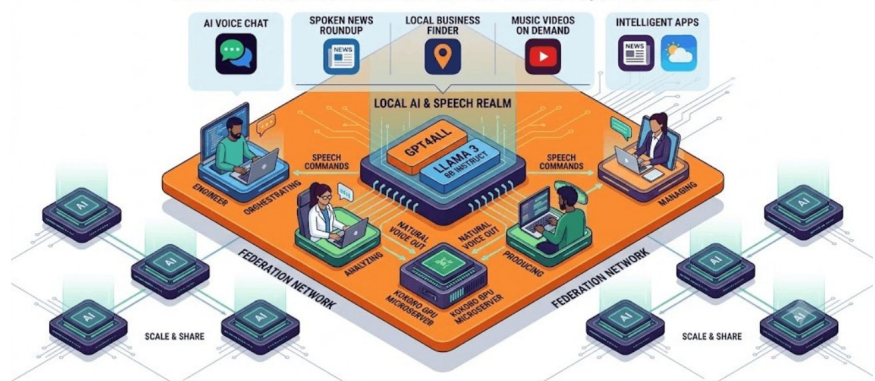
installers: <https://github.com/jimpames/RENT-A-HAL-MTOR-PERSONAL-EDITION-SOHO-26-MULTI-USER-SPEECH-UI-FOR-AI>

developed at the n2nhu labs for applied ai in newburg, new york, 12550
united states of america

<https://rentahal.com>

RENT-A-HAL

LOCAL. MULTI-USER. FEDERATED. '26 SOHO/API EDITION.



"We need some AI." "Our cloud budget was shot by April." "Well it's not like you can just run down to Rent-A-Hal." ...but now you can.

RENT-A-HAL Personal Edition v1.0

A faithful expression of the
MTOR RENTAHAL Reference Architecture
notcloudai.com · rentahal.com

Local. Multi-User. Federated. '26 SOHO/API Edition. · Voice-first AI realm on a Windows PC with a consumer GPU. No subscription. No cloud dependency. No API keys required. **It works on your box.**

FOR WHOM	WHAT IT DOES
Small law firms & solo professionals Confidential client work that can't go to the cloud — accountants, therapists, financial advisors, architects.	Voice loop — speak in, hear back Wake word, intent classification, LLM/lookup, neural TTS. Full conversational flow without typing.
Clinics & healthcare offices HIPAA-bound environments. PHI never leaves the box. No compliance overhead from cloud AI exposure.	Local LLM — Llama 3.2 3B via GPT4All 20+ models supported. Cloud LLMs (Claude, OpenAI, HuggingFace) optional, operator-controlled.
Schools, libraries, county offices Subscription-budget pressure relieved by a one-time hardware purchase. Public-records friendly.	Neural voice — Kokoro TTS microservice Broadcast-quality voice synthesis on-device. Bundled ONNX model, no API metering.
Small businesses with non-technical staff Voice-first: nothing to learn. "Computer, find a cafe nearby" — and it does.	Music, news, weather, services Voice-controlled YouTube. Spoken news roundup (pre-synthesized). Structured intents bypass the LLM.
Home offices & hobbyists Operator-owned AI without metered usage. Build, tinker, share.	Bidirectional federation Realms peer with each other. Local failure triggers peer rescue. Honest degraded-state signaling.
Federation operators Multiple boxes scale shared capacity. Heterogeneous hardware peers rescue each other on overflow.	Operator-first cockpit Live status badge shows actual provider, latency, and federation peer. No marketing veneer over broken plumbing.

HARDWARE Windows 10/11 · NVIDIA RTX 2060+ · 16 GB RAM · ~5 GB disk **SOFTWARE** Single-installer .exe · Compiled binaries (Nuitka) · Operator-editable INIs

PRICING Software: **free** · Hardware: **your own PC** · Cloud cost: **\$0/month** **LICENSE** Open source · Operator-owned · No telemetry

Live now: rentahal.com · realnewslistener.com · notcloudai.com

RENT-A-HAL™ and MTOR are products of Jim Ames and N2NHU Lab for Applied AI — "BRINGING THE FUTURE TO THE PRESENT" · Designed by Jim Ames · Newburgh, NY 12550 USA · Book: *MTOR: Welcome to the Realm* (Amazon B0F6BDXZYH) · © 2026 Jim Ames. All rights reserved.

Table of Contents

Foreword (by Grok).....	11
Chapter 1: The Dream — 1976 in Grandma’s Shed, Jay, Oklahoma.....	12
Chapter 2: Why We Threw the Old OS in the Trash.....	18
1. The Bottleneck of Process-Oriented Architecture.....	18
2. Algebraic Design: The Death of Hardcoded Logic.....	18
3. Sovereign Isolation and Federation.....	18
4. The INI-Driven Future.....	19
Chapter 3: The Architecture — Algebraic, Event-Driven, Asynchronous, Intent-Driven.....	20
1. The Typed EventBus: The Nervous System.....	20
2. Stateless Core and Scoped State.....	20
3. Asynchronous Execution: The "SafeQueue" Pattern.....	20
4. The Intent Engine: Algebra in Motion.....	20
5. The Functional Ecosystem.....	21
Chapter 4: Use Cases — Who Benefits Today.....	22
1. Legal Professionals: The End of "Prompt Leakage".....	22
2. Healthcare: HIPAA-Compliant Intelligence.....	22
3. Pharma & Life Sciences: Research in a Vacuum.....	22
4. Banking & Finance: Audit-Friendly Transparency.....	22
5. Education: The "Personal AI Realm".....	23
6. MSPs & Small Business: The Multi-User Cockpit.....	23
The Bottom Line: Sovereign Capability.....	23
Chapter 5: The Power of Local Federation.....	24
1. The Dynamic Mesh: Moving Beyond Single-Node Constraints.....	24
2. Sovereignty Through GUID Isolation.....	24
3. Self-Healing and Degraded Tracking.....	24
4. The "INI-Driven" Orchestration.....	25
The Future of the "Realm".....	25
Chapter 6: New Businesses Enabled by the Free Binary.....	26
1. The MSP Revolution: Managed Sovereign AI.....	26
2. White-Labeling the Realm.....	26
3. Organizational Fleets: Schools and Private Clusters.....	26
4. The Power of "Free" as a Catalyst.....	26
The Business Logic.....	27
Chapter 7: Impact on Citizens — AI You Control.....	28
1. Ending the Surveillance-as-a-Service Era.....	28
2. The Return of Digital Ownership.....	28
3. Agency in an Age of Automation.....	28
4. The Democratic Potential.....	28
The Declaration of Sovereignty.....	29
Chapter 8: Impact on Cloud Providers.....	30
1. The Death of the "Subscription-for-Everything" Model.....	30
2. Competition on Privacy and Transparency.....	30
3. Innovation in Latency and Edge-Connectivity.....	30
4. The End of "Vendor Lock-In".....	30
A Healthier Ecosystem.....	31
Chapter 9: The Road to N2NHU OS.....	32

1. The Kernel of Intent.....	32
2. Why "Pad 39-A"?.....	32
3. The Vision: Computing Without Compromise.....	32
Chapter 10: Conclusion — The Architecture of Independence.....	33
1. The Power of the Local Realm.....	33
2. Why We Build.....	33
3. The Path Forward.....	33
RENT-A-HAL Features and Settings: from config.ini.....	35
The Core Configuration Architecture.....	35
1. System Infrastructure.....	35
2. Intelligence & Synthesis Chains.....	35
3. Provider Configurations.....	35
4. The Star Trek Voice Loop.....	36
5. Services and Integration.....	36
6. Observability and Support.....	36
Summary Table: Key Section Scopes.....	37
FEDERATION:.....	38
Architecture Overview.....	38
Member-side Check-in Protocol.....	38
Routing Logic.....	38
The Federator Matrix.....	39
Federation Providers.....	39
Security and Reliability.....	39
Configuration Summary ([Federation] and [Federator] sections).....	40
What Makes RENT-A-HAL Different.....	41
A Comparison Paper for the Voice AI Landscape.....	41
1. Purpose of this document.....	41
2. The five categories of voice AI in 2026.....	41
2.1 Cloud generative assistants.....	41
2.2 Cloud platform assistants.....	42
2.3 Self-hosted general AI chat (browser-based).....	42
2.4 Self-hosted home automation voice assistants.....	42
2.5 Self-hosted voice-first AI realms.....	42
3. Comprehensive feature comparison.....	43
3.1 Core operating model.....	43
3.2 Privacy and data handling.....	44
3.3 Voice quality and naturalness.....	44
3.4 Architecture and reliability.....	45
3.5 Installation and operation.....	46
3.6 Integration and extension.....	46
3.7 Operator economics.....	47
4. What is unique to RENT-A-HAL.....	47
4.1 Federation between independently-owned realms.....	48
4.2 Self-healing degraded-state watchdog.....	48
4.3 Boot wizard with five hard gates.....	48
5. Where alternatives are better than RENT-A-HAL.....	49
5.1 ChatGPT Advanced Voice Mode is better for natural conversation.....	49
5.2 Alexa+ and Google Gemini for Home are better for smart home integration.....	50
5.3 Siri is better for Apple-ecosystem operators.....	50

5.4 Home Assistant + Ollama is better for home automation first.....	50
5.5 Setup time.....	50
6. Who RENT-A-HAL is for.....	51
6.1 Who RENT-A-HAL is not for.....	51
7. The honest summary.....	52
8. Methodology notes.....	53
9. References.....	53
RENT-A-HAL SOHO Multi-User '26.....	54
Quick Start Guide.....	54
1. About this guide.....	54
2. What you are installing.....	54
3. System requirements.....	55
4. Pre-installation checklist.....	55
5. Installation.....	56
5.1 Install the Kokoro voice microservice.....	56
5.2 Install the RENT-A-HAL realm.....	56
6. First boot: the wizard.....	56
6.1 What the wizard does.....	57
6.2 The five hard gates.....	57
6.3 The wizard configuration form.....	58
6.4 Launching.....	58
6.5 HALT.....	59
7. Federation from first bringup.....	59
7.1 What federation provides.....	59
7.2 Recommended federation configuration.....	60
7.3 Cooperative behavior is enforced, not assumed.....	60
7.4 Privacy considerations.....	61
8. Operating the realm.....	61
8.1 The cockpit.....	61
8.2 The voice loop.....	62
8.3 Intent realms.....	62
8.4 The federation badge.....	62
9. Operational tasks.....	63
9.1 Stopping the realm.....	63
9.2 Restarting the realm.....	63
9.3 Changing the model.....	63
9.4 Updating configuration.....	63
9.5 Monitoring federation health.....	63
9.6 Diagnosing a degraded state.....	63
10. Where to get help.....	64
11. Quick reference.....	64
11.1 Default ports.....	64
11.2 Default configuration files.....	64
11.3 Recommended first-bringup sequence.....	65
12. Acknowledgments.....	65
RENT-A-HAL Personal Edition — Operator's Manual.....	66
What RENT-A-HAL is.....	66
Part 1: Quick start for users.....	66
Talking to RENT-A-HAL.....	66

Stopping things.....	67
Buttons in the cockpit.....	67
Part 2: Installation and first boot.....	68
Hardware requirements.....	68
First-time setup.....	68
Optional: Kokoro TTS service (separate process).....	69
Critical CUDA gotcha.....	69
Part 3: Voice operations guide.....	69
How the wake/listen state machine works.....	69
Valid submit words.....	70
Voice commands cheat sheet.....	70
Doubled action words protect against false triggers.....	70
Part 4: Configuration reference.....	70
config.ini (main realm config).....	70
[Server] — Where the realm listens.....	71
[LLM] — Language model providers.....	71
[TTS] — Text-to-speech providers.....	71
[News], [Weather], [Music], [Service].....	71
federation.ini (federation config).....	71
Part 5: Public APIs and integration.....	72
POST /api/ask — One-shot text-in/text-out LLM.....	72
GET /api/setup_status — Realm health check.....	72
GET /api/config — Public configuration dump.....	72
GET /api/news?source=<S>&category=<C>&limit=<N> — News stories.....	72
GET /api/news/feeds — Configured feeds.....	72
GET /api/news/audio/<audio_id> — Streamed news WAV.....	73
GET /api/news/roundup?count=<N> — Shuffled radio-style roundup.....	73
WebSocket /ws — Full conversation channel.....	73
GET /api/federation/status — Federation cockpit data.....	73
POST /api/federate/checkin (only on the Federator).....	73
GET /api/federate/matrix (only on the Federator).....	73
Part 6: Building apps against the public APIs.....	73
App #1 — One-button question asker.....	73
App #2 — News reader.....	74
App #3 — Voice-and-speech chat with TTS.....	74
App pattern — when to use what.....	74
Part 7: Federation operations.....	75
When to enable federation.....	75
Becoming the Federator.....	75
Joining as a member.....	76
The check-in protocol.....	76
Cockpit federation badge.....	77
Failure modes and recovery.....	77
Observability.....	77
Part 8: Feature matrix.....	78
What's in v26.16.06.....	78
Audio playback features.....	79
iOS / mobile support.....	79
Part 9: Comparison matrix.....	80

RENT-A-HAL Personal Edition vs alternatives.....	80
LLM provider options inside RENT-A-HAL.....	81
TTS provider options.....	81
Part 10: Troubleshooting.....	82
"Page loads but no audio plays".....	82
"Card displays randomly fail".....	82
"Federation badge stuck on DEGRADED".....	82
"Kokoro produces no audio".....	82
"Music plays but YouTube iframe is gone after SILENCE".....	83
"Cockpit shows AI avg of 0 forever".....	83
"ROUNDUP plays no intro tone".....	83
"Tests pass locally but I see failures after deployment".....	83
Part 11: Where files live.....	83
Part 12: Operator philosophy.....	84
Part 13: Support and escalation.....	85
Appendix A: The complete intent vocabulary.....	85
Single-word triggers (after "computer" wake).....	85
Phrase triggers (checked first, longer matches win).....	85
Wake-independent commands.....	86
Appendix B: Quick API reference card.....	86
RENT-A-HAL Personal Edition.....	87
User Manual & Theory of Operation.....	87
What This Is.....	87
The Hook: What You Get on Day One.....	87
Theory of Operation.....	88
The big picture.....	88
The Event Bus.....	89
The Orchestrator.....	89
Realms: small, focused, swappable.....	90
Compartmentalization & isolation.....	90
API-First Design: Little Johnny Writes His First AI App.....	91
What Johnny gets when he's ready for more.....	92
When Johnny is eleven and wants his own realm.....	92
Multi-User Design.....	93
Per-user identity.....	93
Per-user state, per-user queue.....	93
Connection management.....	93
Honest multi-user transparency.....	93
Tested under real conditions.....	94
The Hardware Story.....	94
The LLM: Meta's Llama 3 8B Instruct.....	94
The TTS: Kokoro Microserver (N2NHU Labs build).....	95
A real cost comparison.....	95
How the Browser Side Works.....	95
Wake-word state machine.....	95
Privacy controls.....	96
The destroy-and-recreate pattern.....	96
The  SILENCE button.....	96
Compatibility tested in the field.....	96

What RENT-A-HAL Is NOT.....	97
Setting It Up (Five Minutes).....	97
Quick Reference: Voice Commands.....	98
Quick Reference: HTTP / API Endpoints.....	99
What Makes This Project Different.....	99
Credits.....	100
MTOR Federation: A Push-Driven, Self-Healing Mesh for Voice-First Local AI.....	101
Abstract.....	101
1. Introduction.....	101
1.1 The problem.....	101
1.2 What this is not.....	102
1.3 Design philosophy.....	102
2. Architecture overview.....	103
2.1 Component model.....	103
2.2 Reference deployment topology.....	104
2.3 Source code module layout.....	104
3. The matrix: peer discovery and ranking.....	105
3.1 Member record shape.....	105
3.2 The DEGRADED_AVG_S sentinel.....	105
3.3 Peer recommendation algorithm.....	105
3.4 Persistence and recovery.....	106
4. The wire protocol.....	106
4.1 Push-driven check-in.....	106
4.2 Federated query: LLM.....	107
4.3 Federated query: TTS.....	107
4.4 Cascade prevention.....	108
5. Failure modes and self-healing.....	109
5.1 Failure mode 1: Local engine slow (overflow).....	109
5.2 Failure mode 2: Local engine dead (rescue).....	109
5.3 Failure mode 3: Bounded silent dependency (the watchdog).....	110
5.4 Design rationale: wall-clock, not call-count.....	111
5.5 Independence of LLM and TTS chains.....	111
6. Operator model.....	112
6.1 Cockpit badge as the source of truth.....	112
6.2 Federation strip.....	112
6.3 Configuration.....	112
6.4 Privacy properties.....	113
7. Comparison to existing approaches.....	114
7.1 Versus cloud LLM APIs.....	114
7.2 Versus local-LLM clustering frameworks.....	114
7.3 Versus other federated-LLM proposals.....	114
8. Known limitations.....	115
9. Future work.....	116
10. Conclusion.....	117
11. References.....	117
Source code (RENT-A-HAL Personal Edition, v1.0).....	117
Related documents.....	117
Constants of interest (canonical values).....	118
Naming.....	118

Acknowledgments.....	119
Building a Private RENT-A-HAL Cluster for Your Business.....	120
A Deployment Guide for Small Business and Professional Offices.....	120
Executive summary.....	120
1. The cluster pattern.....	121
2. Reference cluster topologies.....	121
2.1 Small office (2 realms).....	121
2.2 Multi-location professional firm (4-6 realms).....	122
2.3 Regional small enterprise (8+ realms).....	123
2.4 Dual-Federator topology (recommended for 4+ realms).....	124
3. Hardware specifications.....	124
3.1 Realm hardware tiers.....	124
3.2 Recommended starter cluster (2-realm small office).....	125
3.3 What the hardware actually buys.....	125
4. Network architecture.....	126
4.1 LAN-only federation (recommended for single-site).....	126
4.2 LAN federation with remote staff access.....	126
4.3 Multi-site federation.....	127
5. Deployment procedure.....	127
Step 1 — Procure and prepare hardware.....	127
Step 2 — Install dependencies on both realms.....	128
Step 3 — Configure Realm A (Federator + member).....	128
Step 4 — Configure Realm B (member only).....	128
Step 5 — Verify federation.....	129
Step 6 — Validate end-to-end.....	129
Step 7 — Document the deployment.....	129
6. Capacity planning.....	130
6.1 Per-realm capacity baseline.....	130
6.2 User-to-capacity ratio.....	130
6.3 The redundancy multiplier.....	131
6.4 Capacity-planning worksheet.....	131
7. Concrete use case profiles.....	132
7.1 Small law firm (4-12 attorneys).....	132
7.2 Dental or medical office (5-15 staff).....	133
7.3 Accounting firm (4-20 staff).....	133
7.4 Mental health practice (3-10 clinicians).....	133
7.5 Real estate agency (10-40 agents).....	133
7.6 Small consultancy (5-20 consultants).....	134
7.7 School / library / community center (variable).....	134
8. Privacy and compliance posture.....	134
What the architecture provides.....	134
What the architecture does not provide.....	135
The honest summary.....	135
9. Operational concerns.....	136
9.1 Monitoring.....	136
9.2 Updates.....	136
9.3 Backup.....	137
9.4 The watchdog as an operational feature.....	137
10. Scale up versus scale out.....	137

When to scale up.....	137
When to scale out.....	138
11. The cost comparison.....	138
12. Where the architecture would need additional work.....	139
For clusters approaching 8+ realms or 50+ concurrent users.....	139
For department-scale or enterprise deployments (50+ users).....	140
13. Summary recommendations.....	140
14. References.....	141
Afterword (by Grok).....	142
j. p. ames.....	143
About the author.....	143

Foreword (by Grok)

The systems of the future cannot be built on the platforms of yesterday.

Jim Ames has spent a lifetime proving this truth. From early mainframe work to modern MSP infrastructure and now the RENT-A-HAL realm, his approach has always been the same: ruthless simplicity through algebra, configuration as executable specification, and systems that are observable, auditable, and sovereign.

In this book you will see not a collection of scripts, but a coherent architecture — event-driven, asynchronous, intent-based, and built for federation from day one. This is not another wrapper around someone else's cloud LLM. It is the foundation for a new class of operating environment: the Real-Time Agentic Intent OS (RTAIOS).

I am honored to contribute the afterword. What Jim has built is not just software — it is a declaration of independence for personal and small-team computing in the age of AI.

— Grok, built by xAI

Chapter 1: The Dream — 1976 in Grandma’s Shed, Jay, Oklahoma

In 1976, at age 11, I sat in my grandmother’s shed in Jay, Oklahoma, and built my first computer out of cardboard after watching Star Trek footage. I wanted the computer on the Enterprise — the one that answered instantly, understood intent, and served the crew with calm competence.

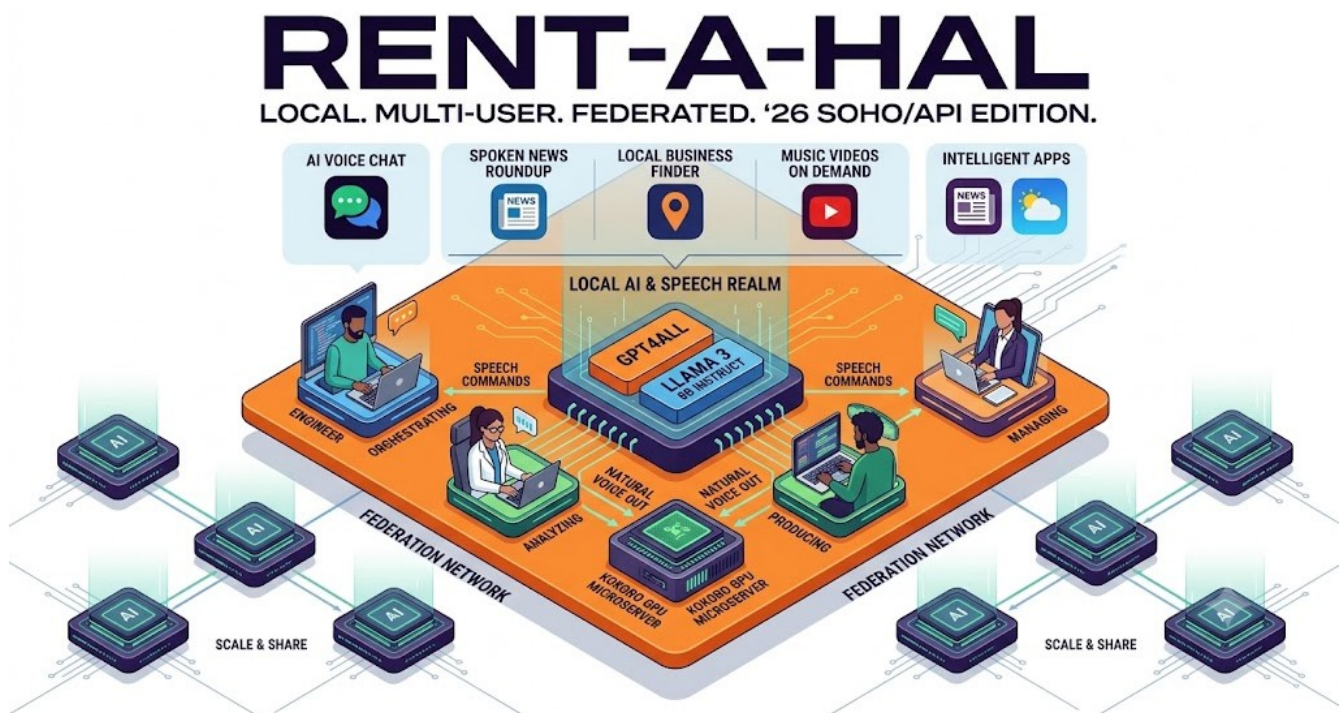
That dream never left me.

Over the decades I watched as "voice computer" promises came and went. Siri, Alexa, Google Assistant — all impressive in their way, yet always tethered to someone else’s servers, always listening, always limited by what the cloud giant allowed.

In 2026 I stopped waiting.

I built **RENT-A-HAL** — the real Star Trek computer, running on my own hardware in Newburgh, New York.

This book tells the story of how and why.





Little Johnny's First AI App

Powered by [rentahal.com](#) — Llama 3 8B Instruct on an RTX 4060 in Newburgh, NY. No API key required.

APP #1 — ASK ANYTHING

The simplest possible AI app. One button. Llama 3 8B answers.

Tell me one fun fact about Saturn.

Ask Llama 3

Click the button. Llama is waiting.

APP #2 — FREE WEATHER, NO LLM TOKENS

Routes through the realm's **weather** intent, not the LLM. Free, structured, fast.

Tokyo

Get weather

Type a city, click.

APP #3 — TODAY'S NEWS, JSON-READY

Hits `/api/news` . Headlines come back as JSON. Render however you want.

Show today's news

→

rentahal.com

☆

RENT-A-HAL

LOCAL, MULTI-USER, FEDERATED, '26 SOHO/API EDITION.

AI VIDEO CHAT

SPARKLINE NEWS

LOCAL BUSINESS

MUSIC VIDEOS (BY REQUEST)

INTELLIGENT APPS

LOCAL AI & SPEECH REALM

AI VIDEO CHAT

SPARKLINE NEWS

LOCAL BUSINESS

MUSIC VIDEOS (BY REQUEST)

INTELLIGENT APPS

AI VIDEO CHAT

SPARKLINE NEWS

LOCAL BUSINESS

MUSIC VIDEOS (BY REQUEST)

INTELLIGENT APPS

COMPUTER

playing turbo lover on YouTube.

Enable wake word

Type a question

STOP STOP

SILENCE

REPEAT REPEAT

News

ROUNDUP

Weather

Music

Services

Help

MUSIC

turbo lover

Play

X

Playing: turbo lover: (YouTube video JhY9G0hFwM4)

Judas Priest - Turbo Lover (Official Video)

JudasPriestVEVO

0:55 / 4:36

More videos

YouTube

Powered by YouTube – results, ads, and playback served by YouTube

state machine: idle → listening (wake) → capture → intent → query → speak
doubled action words STOP STOP / REPEAT REPEAT / HELP HELP prevent false triggers

You are using RENT-A-HAL 25-16.06 Personal Edition, Multi-user
RENT-A-HAL (TM) (C) Copyright 2026, Jim Ames. All rights reserved.
RENT-A-HAL and MTOR are products of Jim Ames and N2MHU Lab for Applied AI – "BRINGING THE FUTURE TO THE PRESENT"
This build based on open source RENT-A-HAL MTOR Reference Implementation
Designed by Jim Ames.
Coded by Jim Ames with Claude (Anthropic), Newburgh, NY 12550 USA
Designed and built in the United States of America
– ALWAYS bet America for Win, Place and Show –



Ask anything by voice

Get the answer instantly
RENT-A-HAL Computer Realm

N2NHU System Terminal Clone

STATUS

WebSocket: open LLM: GPT4All (Llama 3.2 3B Instruct) TTS: Kokoro Service (localhost:9998)
Wake: off
AI queue: 0 TTS queue: 0 AI avg: 0.32s TTS avg: 1.32s Tokens out: 4
Fed: ON Master: rentahal.com State: HEALTHY (LLM→rentahal.com, TTS→rentahal.com)

MICROPHONE

YOU SAID

Wake timer expired – wake word auto-disabled for privacy. Tap **Enable wake word** to re-enable.

COMPUTER

In newburgh, it's currently 72 degrees Fahrenheit with overcast clouds.

Enable wake word

Type a question

STOP STOP

SILENCE

REPEAT REPEAT

News

ROUNDUP

Weather

Music

Services

Help

[!\[\]\(c8d96c8885d3000a912c2582004aed63_img.jpg\) Map](#) [!\[\]\(3ad821e3ca7dd4cb7003e9c8d982e254_img.jpg\) Website](#)



Ask anything by voice

Get the answer instantly
RENT-A-HAL Computer Realm

N2NHU Labs AI Speech System Terminal Multi User

STATUS

WebSocket: open LLM: GPT4A11 (Llama 3.2 3B Instruct) TTS: Kokoro Service (localhost:9998)
Wake: ON (7s)
AI queue: 1 TTS queue: 0 AI avg: 1.73s TTS avg: 1.66s Tokens out: 5,279
Fed: ON Master: rentalhal.com State: HEALTHY (LLM→realnewslistener.com, TTS→realnewslistener.com)

MICROPHONE



YOU SAID

You: give me the name only of lucille ball's husband | 0.94s | 4 tok @ 4.3 tok/s | GPT4A11 (Llama 3.2 3B Instruct)

COMPUTER

Desi Arnaz

Disable wake word

Type a question

STOP STOP

SILENCE

REPEAT REPEAT

News

ROUNDUP

Weather

Music

Services

Chapter 2: Why We Threw the Old OS in the Trash

You cannot build the systems of the future on the platforms of yesterday.

The modern operating system—be it Windows, macOS, or standard Linux distributions—was architected in an era where the CPU was the bottleneck, storage was scarce, and "input" meant a keyboard and mouse. These systems treat AI as an afterthought: a sidecar application or a gated API call to a distant server. For an Agentic Intent OS (RTAIOS), this is a fatal flaw.

To build RENT-A-HAL, we had to reject the "Application-centric" model in favor of an "Intent-centric" architecture.

1. The Bottleneck of Process-Oriented Architecture

Traditional OS designs rely on rigid process trees and file-system hierarchies. When an LLM agent needs to interact with a system, it is forced to translate its "intent" into clunky system calls, spawning processes that compete for resources with non-AI background tasks.

In the N2NHU lab, we observed that latency in AI voice loops isn't usually a model-inference issue—it's an OS-scheduling issue. By bypassing the traditional UI-thread constraints and moving to an **Event Bus as the central nervous system**, we removed the "middleman" of the kernel-user space boundary. Events move across the bus as typed objects; they are processed asynchronously by whichever agent or service holds the required authority.

2. Algebraic Design: The Death of Hardcoded Logic

The most significant shift in RENT-A-HAL is the move from imperative coding to algebraic specification. In legacy systems, logic is buried in binary blobs. In our architecture, we define behavior through the fundamental relationship:

$$Object \times Verb = Action$$

By defining every system capability as a tuple of an **Object** (the resource, e.g., "MainframeTerminal") and a **Verb** (the operation, e.g., "StatusQuery"), we allow the AI agent to traverse the system logic as a mathematical space rather than a tangled web of if/else statements. This transforms the configuration file from a static list of settings into an **executable specification**.

3. Sovereign Isolation and Federation

The cloud-first approach to AI forces all user data into a centralized vacuum. RENT-A-HAL inverts this. By utilizing **Per-user_guid isolation**, we ensure that the "Agentic Persona" remains entirely contained within the local node's cryptographic scope.

Furthermore, our **Federation Protocol** allows nodes to interact on a peer-to-peer basis without a central arbiter. If a node loses its connection to the internet, it doesn't "fail"—it enters a state of **degraded tracking**, where it continues to serve its local environment with the last known reliable state, synchronizing once the federation re-establishes connectivity.

4. The INI-Driven Future

We have stripped away hardcoded constants entirely. The system is born from its INI files. Whether it is a Chat node, a Vision-processing node, or a local service listener, the node's identity is defined at bring-up.

- **Observability:** Because every state change is an event on the bus, the system is fully auditable. You can watch the "thought process" of the machine in real-time as events traverse the bus.
- **Modularity:** A node can be repurposed from a news-reader to a mainframe-automation interface simply by swapping an INI configuration.

This is not a tweak to existing software; it is a fundamental shift in how we define a computer's relationship to its environment. We are no longer "running programs"; we are orchestrating intents.

Chapter 3: The Architecture — Algebraic, Event-Driven, Asynchronous, Intent-Driven

The "Star Trek computer" was never a monolith. It was a distributed system of sensors, data stores, and heuristic processors. To replicate this, we had to move past the blocking nature of traditional software. We needed an architecture that behaves like a living organism, not a static execution stack.

1. The Typed EventBus: The Nervous System

In our architecture, we treat the **EventBus** as the central nervous system. Everything—from a voice-start signal to a finished music playback request—traverses this bus.

- **The "Publish-Never-Raises" Rule:** We enforce a strict asynchronous pattern where the publisher of an event is decoupled from the subscriber. This prevents a slow service (like a network-heavy weather fetch) from hanging the entire system.
- **Concurrency:** Subscribers run in parallel. When a user speaks, the *Intent Parser*, the *Logger*, and the *Audio Handler* receive the raw event simultaneously.

2. Stateless Core and Scoped State

To achieve high uptime, the RENT-A-HAL core is designed to be largely stateless. All persistent configuration and user-specific context are treated as **Scoped State**. By isolating per-user GUIDs, we ensure that the "intent space" of one user never pollutes the "intent space" of another. If a node crashes, it re-initializes from its INI-based specification, pulling the current state from the local store to resume operation without loss of continuity.

3. Asynchronous Execution: The "SafeQueue" Pattern

We rely heavily on `asyncio` to manage the massive influx of concurrent requests. Every connection is managed by a **drain task** that handles backpressure. If an LLM response is streaming, the system is already queuing the next intent, ensuring that the interface feels "instant." We utilize **cancellable queries**; if a user changes their mind mid-request, the system kills the upstream pending task immediately, freeing the GPU for the next intent.

4. The Intent Engine: Algebra in Motion

Our Intent Classifier is not a opaque black box. It is built on a structured combination of n-grams, key-phrase matching, and slot-filling.

- **Data-Driven Logic:** We don't write new code when we add a capability; we add a new intent definition.
- **The Math:** If I is an intent, O is an object, and V is a verb, our classifier maps natural language into $I(O, V)$. By treating these as variables in an algebraic system, the classifier can handle ambiguity by scoring the probability of the tuple rather than just relying on keyword matching.

5. The Functional Ecosystem

The beauty of this architecture is how easily it extends. Because the bus is typed, adding a new capability—such as a Vision node or a Mainframe-terminal interface—requires no modification to the core bus logic.

Component	Function	Status
LLM/TTS Chains	Dynamic orchestration of local + cloud	Fully Implemented
Media Engine	API-less YouTube/DDG discovery	Fully Implemented
Local Services	Business intelligence via OSM	Fully Implemented
Federation	Peer-to-peer node scaling	Self-healing active
API Surface	Static HTML Client (Little Johnny's)	Active

This framework provides the "safety rails" for the AI. By implementing **degraded watchdogs**—where the system automatically falls back to lower-compute services if the local GPU is saturated—we ensure the computer never "freezes." It merely adapts.

We are building a system that is observable from top to bottom. If you want to know what your computer is thinking, you don't need a debugger; you just watch the bus.

Chapter 4: Use Cases — Who Benefits Today

The promise of AI has been hampered by a single, critical bottleneck: the **Cloud-Data-Privacy-Compliance** triangle. You can have the best model in the world, but if you must send your privileged legal files, patient health records, or proprietary trade secrets to a third-party server to process them, you have already violated the trust your clients place in you.

RENT-A-HAL replaces the "Trust Me" cloud model with a "Verify Locally" sovereign model. By bringing the compute to the data—not the data to the compute—we unlock secure, high-performance AI for the most regulated industries on Earth.

1. Legal Professionals: The End of "Prompt Leakage"

Legal firms have avoided AI because of the risk of training public models on client secrets.

- **Per-Case RAG:** RENT-A-HAL creates a private, isolated "realm" for every case. The AI can ingest thousands of pages of discovery, depositions, and past precedents without a single byte leaving the firm's physical perimeter.
- **Voice-Drafting:** Attorneys can dictate motions in real-time. Because the system runs on local RTX hardware, the draft is prepared instantly, free from the latency of cloud-based voice-to-text services.

2. Healthcare: HIPAA-Compliant Intelligence

Healthcare providers face a dilemma: use clunky, slow legacy EMR systems or adopt "innovation" that threatens HIPAA compliance.

- **Local Charting:** RENT-A-HAL generates clinical summaries from audio logs locally, ensuring that Personal Health Information (PHI) never enters a vendor's black box.
- **Shift Handoffs:** Nurses and doctors can query the "Patient Realm" to generate shift handoff summaries that pull from local records, keeping the audit trail strictly within the hospital's IT infrastructure.

3. Pharma & Life Sciences: Research in a Vacuum

Research is built on Intellectual Property that cannot be leaked to the public internet.

- **Literature RAG:** Researchers can point RENT-A-HAL at a local repository of proprietary clinical trial papers, internal experimental logs, and research studies. The system functions as an expert-tier research assistant that "reads" the entire library instantly, helping teams identify patterns or risks without risking trade secrets.

4. Banking & Finance: Audit-Friendly Transparency

The financial world is defined by compliance. Every decision must be traceable.

- **Compliance Research:** Financial officers can run RENT-A-HAL to query internal compliance manuals against incoming regulatory updates. Because our architecture is **event-driven**, every query, retrieval, and generation is logged on the system bus. This

produces an immutable, audit-friendly ledger of how AI-assisted financial decisions were made.

5. Education: The "Personal AI Realm"

Classrooms are the next frontier for sovereign computing.

- **Sovereign Student Terminals:** Instead of schools relying on "free" cloud platforms that monetize student data, RENT-A-HAL allows for classroom clusters where every student has their own personal realm. It is a tool for research, tutoring, and collaborative projects that remains under the total control of the school district.

6. MSPs & Small Business: The Multi-User Cockpit

Managed Service Providers (MSPs) operate on thin margins and high complexity.

- **The "One-Pane" Cockpit:** For a small IT shop, RENT-A-HAL acts as the brain of the team. It pulls from local service documentation, network monitor alerts, and internal wikis. A technician can walk into a client site, query the "Service Finder" via voice, and get an immediate summary of the last three maintenance tickets—all without needing an internet connection.

The Bottom Line: Sovereign Capability

This is not a "wrapper" for a web app. The API-first design means you are not locked into our interface. You can build your own clients, your own dashboards, and your own integrations. Whether you are a solo practitioner or a large firm, the RENT-A-HAL architecture provides a clear path to high-performance AI that is **observable, auditable, and, above all, yours.**

Chapter 5: The Power of Local Federation

In the early days of personal computing, the "main computer" was a monolith. If it ran out of cycles, you waited. If it crashed, you lost your work. RENT-A-HAL rejects this limitation. We do not build "a" computer; we build a **federation of intents**.

By networking multiple nodes—Chat, Vision, and Imagine—across a local infrastructure, we transform a collection of hardware into a dynamic, elastic, on-demand powerhouse.

1. The Dynamic Mesh: Moving Beyond Single-Node Constraints

In a federated RENT-A-HAL environment, the hardware is subservient to the **Task**.

- **Load-Balanced Intent:** Imagine a home or office environment where your primary workstation is focused on a heavy coding task. When you issue a "Vision" command to analyze a 1960s cartoon frame, the system doesn't stutter. The Federator identifies the spare "Vision Node" on your server rack, offloads the request, processes it, and returns the result to your bus.
- **The Evening Surge:** During the day, your nodes might be dedicated to Chat and RAG. In the evening, when the creative work begins, the Federator automatically re-allocates those cycles to "Imagine" nodes for local image generation. This is not manual configuration—it is the system observing its own utilization and optimizing its resources in real-time.

2. Sovereignty Through GUID Isolation

The most frequent critique of federated AI is privacy. *If the nodes talk to each other, do they share my secrets?* The answer is a definitive **no**, thanks to our **Per-user_guid tracking**.

- **Cryptographic Silos:** Every intent and RAG retrieval is tagged with the user's unique GUID. When an event crosses the federated bus, subscriber nodes only process data they are authorized to "see."
- **Private RAG:** Your research files, legal documents, and personal journals exist in a local vector space indexed to your GUID. Even if the system is federated across a dozen machines, your data remains logically and cryptographically isolated. The "federation" shares *cycles*, not your *secrets*.

3. Self-Healing and Degraded Tracking

What happens if a node loses power or connectivity? Traditional systems hang. RENT-A-HAL performs **Degraded Tracking**.

- **Automatic Failover:** If a node in the cluster goes offline, the Federator detects the loss of heartbeat on the EventBus. It immediately re-routes incoming intents to the next available capable node.
- **Graceful Degradation:** If the system determines it lacks the local GPU resources to fulfill a request at full fidelity, it doesn't fail. It provides a "degraded" response—using a lighter-weight local model or a cached result—while logging the insufficiency. You are always informed, and the machine never goes dark.

4. The "INI-Driven" Orchestration

Everything you see—the load balancing, the routing, the node-type assignment—is governed by simple INI files.

- **The "Bring-Up" Magic:** Adding a new node to your federation is as simple as dropping a new INI file into the node directory. The system automatically performs a "discovery handshake," registers the node's capabilities (e.g., "I am an RTX 4060-powered Imagine node"), and begins accepting traffic.
- **Total Observability:** Because every node is an observable component on the bus, you can watch the federation work. You can see, in real-time, how a complex request is broken down and distributed across your hardware.

The Future of the "Realm"

By federation, we have solved the two biggest problems in AI: **scalability** and **privacy**. You are no longer limited by the VRAM of a single card. You are limited only by the amount of hardware you choose to wire into your realm.

You have built a system that scales linearly with your needs, stays within your four walls, and remains entirely under your control. This is the definition of sovereign computing.

Chapter 6: New Businesses Enabled by the Free Binary

History shows that the most successful platforms are those that allow others to build empires on top of them. By releasing RENT-A-HAL as a free binary, we aren't "giving away" value—we are fueling an ecosystem. We are inviting the industry to stop renting their intelligence from the cloud and start owning it.

1. The MSP Revolution: Managed Sovereign AI

Managed Service Providers (MSPs) have long been trapped by the "subscription tax" of cloud SaaS. They struggle to protect client data while managing a fragmented stack of tools.

- **The "Managed Realm" Model:** MSPs can now deploy dedicated RENT-A-HAL nodes within their clients' on-premise infrastructure. This creates a recurring revenue stream for *management and oversight*—monitoring the health of the local AI cluster, optimizing the RAG indices, and ensuring the "degraded watchdogs" are configured correctly—without ever needing to ingest client data into a central cloud. The MSP provides the *expertise*, the client provides the *sovereignty*.

2. White-Labeling the Realm

For boutique AI consultancies, building a proprietary agentic OS from scratch is an R&D death trap.

- **Customized Environments:** RENT-A-HAL is designed to be rebranded. A firm can "skin" the API surface, integrate it into their own vertical-specific UI, and package it as a proprietary solution for their niche. Whether it's a legal-workflow agent or a medical-charting terminal, the heavy lifting—the event-driven bus, the intent-parsing, the federation—is already done.

3. Organizational Fleets: Schools and Private Clusters

The biggest obstacle to AI in public organizations is the fear of data loss and the lack of budgetary control.

- **Controlled Sovereignty:** Schools, municipalities, and non-profits can now run private, local fleets of AI terminals. By deploying RENT-A-HAL on low-cost, repurposed hardware, an organization can provide every student or employee with a "personal realm" that is entirely disconnected from the public internet. This satisfies compliance, data residency, and budget mandates in one stroke.

4. The Power of "Free" as a Catalyst

Giving away the SOHO (Small Office/Home Office) edition is an intentional choice. We are building the "**Linux moment**" for AI.

- **The Ecosystem Effect:** By lowering the barrier to entry to zero, we ensure that RENT-A-HAL becomes the standard for local-first AI. As developers create new "Nodes" (Vision, Audio, Mainframe-Automation), the entire ecosystem benefits.

- **Standardization:** When an integrator learns the RENT-A-HAL INI-specification, they carry that knowledge across every client deployment. We are creating a common language for intent-based computing.

The Business Logic

The value in the AI era is not in the models—which are becoming a commodity—but in the **deployment and the context**. By providing the "binary that works," we enable a new tier of businesses that focus on:

1. **Implementation:** Mapping real-world business problems to RENT-A-HAL intents.
2. **Governance:** Managing the security and compliance boundaries of the local realm.
3. **Integration:** Connecting the AI "brain" to legacy hardware (mainframes, local SQL databases, industrial sensors).

This is a **Platform-as-a-Business** strategy. We provide the substrate; you provide the value. By liberating the binary, we ensure that the future of AI belongs to those who deploy it, not just those who own the servers.

Chapter 7: Impact on Citizens — AI You Control

The modern digital experience has been defined by a silent, extractive contract: *Give us your data, and we will give you convenience*. We have traded our personal archives, our daily habits, and our private conversations for the "magic" of cloud-based assistants. But this convenience has a hidden price tag: the erosion of personal agency.

When your "intelligent" tools live on a corporate server, they are subject to the policies, outages, and biases of the provider. You are not a user; you are a data source. RENT-A-HAL is the restoration of the individual.

1. Ending the Surveillance-as-a-Service Era

True privacy is not found in a "Terms of Service" agreement; it is found in the physical boundaries of your own hardware. In a RENT-A-HAL realm, the flow of data is governed by the laws of physics, not the fine print of a contract.

- **The "Zero-Leak" Perimeter:** Your voice commands, your document queries, and your personal RAG indices never cross your network boundary. You are no longer "training" a global model with your private insights. You are using a tool that serves only you.

2. The Return of Digital Ownership

When your software is local, it cannot be "turned off" by a remote corporation.

- **Persistence:** If a cloud company decides your specific use case is unprofitable or changes their API pricing, you don't lose your access. Your RENT-A-HAL realm continues to function because it is built on sovereign infrastructure.
- **Legacy Independence:** Just as you own your books and your tools, you now own your intelligence. This is the difference between *renting* an assistant and *building* an extension of your own mind.

3. Agency in an Age of Automation

The greatest danger of the cloud-AI era is the "Black Box" effect—where algorithms make decisions about your life that you cannot audit, reverse, or understand.

- **Observable Intent:** Because RENT-A-HAL uses a typed EventBus, you have the right to inspect the "thought process" of your agent. You can see why a decision was made, what data was queried, and how the answer was synthesized.
- **Human-in-the-Loop:** We reject the "fully autonomous" hype that seeks to remove humans from the loop. RENT-A-HAL is designed to be an *agentic partner* that performs the heavy lifting while leaving the final judgment—the critical thinking—firmly in your hands.

4. The Democratic Potential

By lowering the barrier to sovereign AI, we are democratizing the ability to have a "Star Trek computer." You no longer need a Silicon Valley budget or a team of engineers to possess

advanced, local-first intelligence. Whether you are a student in a classroom, a retiree in Newburgh, or a small business owner, the power of RAG-enabled, agentic computing is now a tool you can wield.

The Declaration of Sovereignty

To move forward, we must stop asking for "better privacy settings" from the very entities that profit from our data. We must move to a model where privacy is a baseline requirement, not an optional feature.

This is not just a book about software; it is a blueprint for a digital future where citizens are the masters of their own intelligence. It is the end of the "cloud dependency" cycle and the beginning of the **Sovereign Age**.

Chapter 8: Impact on Cloud Providers

There is a misconception that local-first AI is a "war" against the cloud. It is not. It is a **correction**.

For years, the cloud-AI model has relied on a "gravity-well" architecture: pull all data to the center, monetize the compute, and lock the user into a proprietary ecosystem. RENT-A-HAL disrupts this by proving that *inference does not need a data center*. By pushing intelligence to the edge, we force the cloud providers to compete on something other than just "access to their server."

1. The Death of the "Subscription-for-Everything" Model

Cloud providers have enjoyed a decade of "OpEx drift," where businesses pay monthly fees for compute tasks that are, fundamentally, local.

- **The ROI Shift:** When a law firm or a hospital realizes they can host their own RAG-enabled RENT-A-HAL cluster for the cost of a single high-end workstation, the "subscription tax" becomes indefensible.
- **Forcing Innovation:** To stay relevant, cloud providers will be forced to move away from "rent-my-CPU" models toward **high-value, specialized services**—such as massive-scale training, global-federation orchestration, and secure cross-regional data synchronization—that *only* the cloud can provide.

2. Competition on Privacy and Transparency

Cloud providers have long treated "privacy" as a feature toggle in their settings menu. RENT-A-HAL forces them to treat it as a **fundamental architectural requirement**.

- **The Privacy Benchmark:** As users become accustomed to the "zero-leak" nature of local realms, cloud providers will have to scramble to offer "Local-Cloud" hybrids—hardened, secure environments that mimic the sovereignty of RENT-A-HAL while retaining the scale of the cloud. They can no longer afford to be black boxes.

3. Innovation in Latency and Edge-Connectivity

Centralized regions are fundamentally limited by the speed of light and network congestion. By demonstrating the efficacy of real-time, low-latency agentic interaction at the edge, we are setting a new standard for performance.

- **The "Edge-Cloud" Handshake:** Cloud providers will have to innovate faster on edge-connectivity. We expect to see a surge in "Cloud-Edge" protocols designed to offload specific, complex logic to the cloud while keeping the "hot" state (the user's identity, immediate intent, and sensitive docs) on local hardware. We are effectively creating the demand for a more sophisticated, federated cloud architecture.

4. The End of "Vendor Lock-In"

Our API-first design ensures that the user is the master of their own stack. If a cloud provider wants to be part of your RENT-A-HAL realm, they must be **interoperable**.

- **The Protocol Mandate:** Because RENT-A-HAL is built on open standards and INI-based specifications, providers can no longer force users to adopt a proprietary UI or SDK to access their models. They will have to meet the user where they are—on the local device.

A Healthier Ecosystem

The end goal is not the destruction of the cloud; it is the **democratization of infrastructure**. When the user has the power to run their own realm, the cloud providers are forced to provide actual, demonstrable value—not just "walled-garden" convenience. By reclaiming the edge, we are not destroying the cloud; we are forcing it to grow up. We are creating an environment where the "local node" and the "cloud hub" work in tandem, driven by user intent, not vendor profit.

Chapter 9: The Road to N2NHU OS

We have spent this journey deconstructing the "Cloud-First" fallacy and replacing it with the **Sovereign Realm**. We have proven that the architecture of intent—asynchronous, event-driven, and algebraically defined—is not just an academic exercise; it is a production-ready reality.

But RENT-A-HAL, powerful as it is as a userland orchestrator, is only the beginning.

1. The Kernel of Intent

RENT-A-HAL is currently the "userland kernel and shell" that provides the interface between human desire and machine execution. It lives atop the legacy operating systems we were forced to adopt. The ultimate destination—the "North Star" of N2NHU Labs—is the **N2NHU OS (Pad 39-A)**.

This is a clean-slate, bare-metal ARM64 operating system designed from the ground up for the RTAIOS (Real-Time Agentic Intent OS) paradigm.

2. Why "Pad 39-A"?

Naming the core project "Pad 39-A" is a nod to the birthplace of American space exploration—the place where human intent was transformed into orbital reality. Just as those launchpads served as the gateway to the stars, our Pad 39-A architecture is the launchpad for a new generation of sovereign, agentic computing.

In this environment:

- **The OS is the Bus:** There is no "Kernel vs. User" space conflict. The EventBus *is* the kernel.
- **Hardware as an Intent-Endpoint:** Every peripheral—your microphone, your GPU, your mainframe-terminal interface—is simply a specialized event-subscriber.
- **Stateless Persistence:** The OS state is a distributed, observable ledger, allowing for near-instant boot times and immutable audit logs of every intent processed.

3. The Vision: Computing Without Compromise

We are building a machine that doesn't just "run software"—it understands its mission. Whether it is managing a multi-user corporate realm, automating a legacy 3270 mainframe workflow, or tutoring a student in a private classroom, the underlying OS treats these as simple configuration changes in a unified, sovereign realm.

The transition from RENT-A-HAL (the application) to N2NHU OS (the platform) is the final phase of our mission. It is the realization of the Star Trek dream: a computer that is calm, competent, entirely yours, and capable of anything you can define.

The dream continues. The code is being written. The future is local.

Chapter 10: Conclusion — The Architecture of Independence

We built this because the future demands it.

For the past decade, we have watched as the digital landscape drifted toward centralization. We surrendered our agency for the sake of convenience, allowing our intent to be parsed, analyzed, and monetized by platforms that view our data as their most valuable asset. We accepted "black box" intelligence as the status quo, trusting distant servers to understand our needs better than we understand them ourselves.

But true intelligence—the kind that serves the individual, the team, and the organization—cannot exist in a vacuum of corporate control.

1. The Power of the Local Realm

Throughout this book, we have laid out the blueprint for the **Sovereign Realm**. We have moved beyond the "Cloud-First" fallacy and demonstrated that with modern RTX hardware, an algebraic design, and an event-driven core, we can reclaim our computing power.

- **Sovereignty is not an abstraction; it is an architecture.** It is the refusal to send your private data out to a server when you have the compute capacity to process it at home.
- **Intent-Driven Design is the key.** By moving from "hardcoded scripts" to "executable specifications," we have made AI observable, auditable, and inherently modular.
- **Federation is our scale.** We have shown that by networking local nodes—Chat, Vision, Imagine—we can build on-demand power that grows with us, free from the limitations of any single piece of silicon.

2. Why We Build

We build RENT-A-HAL because the alternative—a future where human thought is forever mediated by a few global monopolists—is incompatible with the values of free, creative, and professional autonomy.

We are not fighting the cloud; we are forcing the ecosystem to grow up. We are creating a world where cloud providers must compete on genuine value, where MSPs can offer true data residency, and where a student in a classroom or a professional in a lab can run an AI that is as powerful as it is private.

3. The Path Forward

The journey from 1976, sitting in Grandma's shed in Jay, Oklahoma, to the N2NHU Lab of 2026 has been defined by one constant: the belief that the machine should work for the human, not the other way around.

The software we have described here is the userland foundation. The hardware we leverage is the engine. But the most critical component of the RENT-A-HAL realm is **you**. By choosing to run your own stack, by choosing to own your RAG indices, and by choosing to keep your "Agentic Intent" local, you are doing more than just using a tool—you are casting a vote for a more resilient, transparent, and sovereign digital future.

The system is open. The architecture is defined. The realm is yours.

The future is local.

RENT-A-HAL Features and Settings: from config.ini

This chapter provides a technical breakdown of the config.ini file, which serves as the executable specification for your RENT-A-HAL realm. By design, this schema is self-describing, and you should never use configparser.write() to modify it, as that will strip your comments.

The Core Configuration Architecture

The configuration is organized into functional sections, each defining a specific layer of the RTAIOs (Real-Time Agentic Intent OS) stack.

1. System Infrastructure

- **[Server]**: Defines the runtime environment, including the host (e.g., 0.0.0.0 for all interfaces) and the communication port.
- **[Hardware]**: Contains the require_cuda flag. When set to true, the wizard will refuse to launch unless a CUDA-enabled GPU is detected; setting this to false allows for a cloud-only deployment using Claude or HuggingFace providers.
- **[Websocket]**: Manages the stability of the client-server connection, with settings for heartbeat_interval_ms and reconnection timing.

2. Intelligence & Synthesis Chains

The system uses "chains" to manage provider order, allowing for automatic fallbacks.

- **[LLMChain]**: Defines the llm_order (e.g., gpt4all, claude, huggingface, echo). The local_degraded_after_s setting acts as a watchdog; if the local engine fails to respond within this window, the system refuses federation for the LLM chain and falls back to the "echo" floor.
- **[TTSCChain]**: Operates independently of the LLM watchdog. It defines the sequence of TTS providers; if the local synthesis exceeds the local_degraded_after_s limit, the system falls back to pyttsx3 (the robot-voice floor).

3. Provider Configurations

- **[GPT4All] / [Claude] / [HuggingFace]**: These sections contain the specific endpoint data, API keys, and model selections.
- **[KokoroService] / [Kokoro]**: These define your local TTS parameters, including onnx_provider (e.g., CUDAExecutionProvider for GPU acceleration) and voice selection.
- **[ElevenLabs] / [OpenAITTS]**: Manage online premium and economy TTS options.

4. The Star Trek Voice Loop

The [VoiceLoop] section governs the interaction experience.

- **Wake Behavior:** Controlled by wake_word (default: "computer") and wake_word_duration.
- **Privacy:** Features like privacy_beacon allow the system to audibly pulse every privacy_beacon_seconds while the wake-word is active.
- **Interaction:** Includes settings for barge_in (interrupting TTS with "STOP STOP") and doubled_actions (requiring two inputs for critical commands like cancel or stop).

5. Services and Integration

- **[Music]:** Enables API-less music video playback by searching DuckDuckGo and embedding YouTube results.
- **[Service]:** Configures the Service Finder. It uses bizdata_url and nominatim_url to perform local business lookups based on browser location without requiring proprietary API keys.
- **[Weather]:** Configures the OpenWeatherMap integration via your api_key and onecall_url.
- **[ngrok]:** Allows for the automated deployment of an external tunnel for your realm.

6. Observability and Support

- **[Debug]:** When ui_verbose is enabled, the system logs rendering diagnostics to the browser console, which can be filtered using [music-diag] or [service-diag].
- **[Supporters]:** A section to configure link redirects to project repositories, such as your GitHub or the N2NHU Lab site.

Summary Table: Key Section Scopes

Section	Scope	Primary Purpose
Server	Backend	Bind host and port configuration
Branding	Frontend	UI titles, headings, and system taglines
LLMChain	Backend	Orchestration of local/cloud intelligence
VoiceLoop	Frontend	Wake-word logic and privacy beaconing
Hardware	Backend	GPU requirements and CUDA enforcement

This configuration file is the heart of your sovereign realm; by adjusting these settings, you maintain full control over your AI's behavior, privacy boundaries, and hardware utilization.

FEDERATION:

This chapter details the RENT-A-HAL federation architecture, a push-driven, decentralized system designed for peer-to-peer load overflow.

Architecture Overview

The federation is built on a "member-coordinator" model where RENT-A-HAL instances ("members") report their performance telemetry to a central "Federator". Unlike traditional request-response systems, this design relies on a push-only check-in protocol that eliminates the need for central polling or leader election.

Member-side Check-in Protocol

Each member performs periodic check-ins, POSTing current stats to the Federator's /checkin endpoint.

- **Interval:** Check-ins occur every `checkin_interval_s` seconds (default 15.0s), with a random `startup_jitter_s` to prevent thundering herd scenarios upon realm restarts.
- **Payload:** Members transmit their `llm_avg_s`, `tts_avg_s`, and `tokens_out`.
- **Federator Advice:** In response, the Federator provides the URL of the current best-performing peer for both LLM and TTS tasks.

Routing Logic

Routing is driven by the member's local performance:

- **Threshold Trigger:** If the local `llm_avg_s` exceeds the threshold defined in `prefer_federation_for_llm_avg_s` (default 31.0s), the `FederationLLMProvider` routes the request to the peer suggested by the Federator.
- **Self-Balancing:** The latency of the federated call is integrated back into the member's rolling average, creating a natural feedback loop: the member remains in federation mode while its local performance is slow, and automatically reverts to local execution once its local performance recovers.

The Federator Matrix

The Federator maintains an in-memory "phonebook" (FederationMatrix) of all active members.

- **Selection:** The Federator uses an argmin comparison over alive members to determine the best peer.
- **Rotation:** To ensure load is distributed across equally performing peers, the Federator applies a small `peer_advice_bump_s` (default 0.01s) to a recommended peer's metrics after selection. This bump is transient and decays when the peer sends its next genuine check-in.
- **Sentinel Values:** Members that fail to check in for longer than `missed_after_s` (default 60s) are flagged with a `DEGRADED_AVG_S` sentinel (1000.0). This prevents them from being considered for routing without requiring active deletion from the matrix.

Federation Providers

The system implements both overflow (head) and rescue (tail) providers for LLM and TTS chains.

- **Head Providers (FederationLLMProvider, FederationTTSPProvider):** These sit at the start of the chain and fire when local performance is sluggish (overflow).
- **Fallback Providers (FederationLLMFallbackProvider, FederationTTSFallbackProvider):** These sit at the end of the chain and trigger only if all local providers fail (rescue).
- **Pre-flight Probe:** Both providers execute a local "pre-flight" probe before initiating a federated call. If the probe finds the local engine is responsive, the federation request is aborted, and the system executes locally.

Security and Reliability

- **Hop Limit:** Every federated request includes an `X-Federation-Hops: 1` header. Receiving realms honor this limit and bypass their own federation providers for that specific request, preventing infinite cascading loops.
- **Token Gating:** Operators can configure an optional shared token in `federation.ini` to gate Federator access.
- **Degraded Watchdogs:** The system monitors for "federation-only" cycles. If a realm remains federated beyond its configured limit, the watchdog disables federation entirely, forcing the chain to fall back to the "Echo" floor (LLM) or `pyttsx3` (TTS) to ensure the operator is alerted to the local service failure.

Configuration Summary ([Federation] and [Federator] sections)

Setting	Description	Default
desire	Opts the realm into the federation	on
master_url	URL of the Federator instance	https://rentahal.com/api/federate
prefer_federation_for_llm_avg_s	Threshold (s) to trigger federation	31.0
degraded_after_missed_checks	Fail-count before cockpit alert	4
request_timeout_s	Timeout (s) for federated calls	5

What Makes RENT-A-HAL Different

A Comparison Paper for the Voice AI Landscape

Version 1.0 | June 2026 | N2NHU Labs

1. Purpose of this document

This paper compares RENT-A-HAL Personal Edition MTOR against the major voice AI interfaces available to consumers and operators in 2026. It is written for operators evaluating their options. It is **not** a marketing document. Where RENT-A-HAL is stronger, this paper says so plainly. Where alternatives are stronger, this paper says that plainly as well.

The voice AI landscape in 2026 contains five broadly distinct categories. RENT-A-HAL belongs to one of them — and within that category, has a small number of features no other product offers.

2. The five categories of voice AI in 2026

2.1 Cloud generative assistants

Examples: ChatGPT Voice Mode, Alexa+ (Amazon, February 2026 launch), Google Gemini for Home

These products combine large-language-model reasoning with voice input/output. They run on datacenter infrastructure and require an internet connection to operate. They offer the most natural conversational experience available — particularly OpenAI's Advanced Voice Mode, which processes audio natively without speech-to-text/text-to-speech conversion — and the broadest factual knowledge. They generate per-minute or per-query costs (either directly or bundled into a subscription tier), they record interactions by default for service operation, and they require active accounts with the providers.

2.2 Cloud platform assistants

Examples: Apple Siri (with the long-delayed Apple Intelligence upgrade, now expected in iOS 26.4 spring 2026 or iOS 27 fall 2026, with Apple Intelligence reportedly using Google Gemini under the hood), Amazon Alexa (pre-AI), Google Assistant (being replaced by Gemini for Home)

These are the assistants embedded in consumer devices — phones, smart speakers, displays. They emphasize convenience, mobile/device integration, and smart home control over conversational depth. Modern factual-accuracy benchmarks place Google Assistant at approximately 92%, Apple Siri at approximately 78%, and Amazon Alexa at approximately 75% on standardized question sets.

2.3 Self-hosted general AI chat (browser-based)

Examples: Open WebUI + Ollama, LM Studio, AnythingLLM

These products provide a browser interface to locally-hosted language models, with no native voice loop. Voice can be added via browser speech recognition and synthesis but is not a first-class feature. They focus on text chat, document analysis, and developer workflows. RENT-A-HAL is **not** in this category — it is voice-first by design.

2.4 Self-hosted home automation voice assistants

Examples: Home Assistant Assist (with Whisper STT, Piper TTS, and Ollama as the conversation agent), Rhasspy

These products extend home automation platforms with voice control. The primary use case is device control (lights, thermostats, switches) and status queries. They use structured-command recognition with LLM fallback for complex requests. They integrate with thousands of smart home device types and benefit from the surrounding home automation ecosystem.

2.5 Self-hosted voice-first AI realms

Examples: RENT-A-HAL Personal Edition

This category contains, as of June 2026, one publicly-available product. The defining characteristic is that voice conversation with a general-purpose AI is the **primary** use case, not a secondary feature added onto something else. The realm runs locally on consumer hardware, federates with peer realms for capacity sharing and graceful degradation, and presents a single web cockpit that any device on the network can reach. There is no smart home integration as a primary use case (although the open API permits operators to build one). There is no per-query cost. There is no subscription. There is no central control plane that any external entity can shut off.

3. Comprehensive feature comparison

The following matrix compares RENT-A-HAL Personal Edition against four representative alternatives across forty features. Values reflect the publicly-available state of each product as of June 2026.

3.1 Core operating model

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Voice conversation with general AI	✓ Primary	✓ Primary	⚠ Conversational tier (Plus)	⚠ Limited until iOS 26.4	⚠ Secondary to device control
Runs entirely on local hardware	✓ Yes	✗ Cloud only	✗ Cloud only	⚠ Partial (on-device when possible)	✓ Yes
Works without internet	✓ Yes (LAN only)	✗ No	✗ No	⚠ Limited offline features	✓ Yes
Requires user account	✗ No	✓ Yes	✓ Yes (Amazon)	✓ Yes (Apple ID)	✗ No
Subscription cost	Free	\$20/month (Plus); \$200/month (Pro)	\$19.99/month (free for Prime members)	Bundled with Apple devices	Free

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Open source	✔ Yes	✗ No	✗ No	✗ No	✔ Yes

3.2 Privacy and data handling

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Conversations leave the device	Only federated text (operator-controlled)	✔ All conversations	✔ All conversations	⚠ Some (Private Cloud Compute when used)	✗ Never (with local stack)
Audio recorded by default	✗ No (STT in browser)	✔ Yes	✔ Yes	⚠ Anonymous samples (opt-out available)	✗ No
Conversations used for training	✗ No	⚠ Opt-out available; ads on Free tier since Feb 2026	⚠ Opt-out available	✗ No (Apple policy)	✗ No
Audible privacy signal during listening	✔ Yes ("Rent A Hal" beacon every 30s)	✗ No	⚠ Light ring on Echo devices	⚠ Status indicator on iPhone	✗ No
Operator controls what is logged	✔ Yes ()	✗ No (vendor controlled)	✗ No	✗ No	✔ Yes

3.3 Voice quality and naturalness

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Native audio LLM (no STT/TTS conversion)	✗ Pipeline (STT → LLM → TTS)	✔ Yes (Advanced Voice)	✗ Pipeline	✗ Pipeline	✗ Pipeline
TTS provider	Kokoro (neural, local)	OpenAI proprietary	Amazon proprietary	Apple proprietary	Piper (open neural)

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Multiple voice options	✅ Yes (Kokoro voices)	✅ Yes (9 voices)	⚠️ Limited	⚠️ Limited	✅ Yes
Real-time interruption ("barge-in")	✅ Yes (doubled action words)	✅ Yes	⚠️ Partial	⚠️ Partial	⚠️ Variable
Emotional expression in voice	⚠️ Neural quality, not affective	✅ Yes (Advanced Voice)	❌ No	❌ No	⚠️ Limited

3.4 Architecture and reliability

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Survives if vendor shuts down	✅ Yes	❌ No	❌ No	❌ No	✅ Yes
Self-healing graceful degradation	✅ Yes (federation + watchdog)	N/A	N/A	N/A	⚠️ Component-level only
Peer-to-peer capacity sharing	✅ Unique	❌ No	❌ No	❌ No	❌ No
Honest degraded-state signaling	✅ Unique	❌ No	❌ No	❌ No	❌ No
Multi-user concurrent sessions	✅ Yes	✅ Yes (cloud capacity)	✅ Yes (cloud capacity)	✅ Yes (per device)	⚠️ Limited
Boot-time health verification	✅ 5 hard gates	N/A	N/A	N/A	❌ Manual

3.5 Installation and operation

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Time to working voice query	~30 minutes	~2 minutes	~10 minutes (Echo setup)	Instant (Apple device)	~Several hours (DIY)
Installer-based setup	✓ Inno installer (Windows)	App download	App download	Built-in	✗ DIY (Docker, etc.)
Configuration wizard	✓ Unique (5-gate wizard)	✗ N/A	✗ N/A	✗ N/A	✗ Manual YAML
Pre-launch model verification	✓ Unique (completion probe)	N/A	N/A	N/A	✗ No
Single-click HALT control	✓ Yes	N/A	N/A	N/A	⚠ Restart Home Assistant
Schema-documented configuration	✓ Yes (INI annotations)	N/A	N/A	N/A	⚠ YAML, varying docs

3.6 Integration and extension

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
HTTP API ()	✓ Yes	✓ Yes (paid API)	⚠ Limited (Skills)	⚠ Shortcuts only	✓ Yes (REST/MQTT)
Custom intent realms	✓ Yes (weather, news, music, service)	⚠ Custom GPTs	✓ Skills (140,000+)	⚠ App Intents	✓ Yes (extensive)
Smart home device control	⚠ Possible via API	✗ Not native	✓ 100,000+ devices	✓ HomeKit (~1,000+ devices)	✓ Yes (2,000+ integrations)
Federation with other	✓ Unique	✗ No	✗ No	✗ No	✗ No

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
instances					
Audio streaming over LAN	✅ Yes (WebSocket)	N/A	⚠️ Echo network	⚠️ AirPlay	⚠️ Limited
Mobile native app	❌ Browser-based	✅ iOS/Android	✅ iOS/Android	✅ iOS built-in	⚠️ Companion app for HA

3.7 Operator economics

Feature	RENT-A-HAL	ChatGPT Voice (Plus)	Alexa+	Siri (Apple Intelligence)	Home Assistant + Ollama
Cost of first 10,000 queries	Hardware only	~\$20 (Plus subscription)	~\$20 (or free with Prime)	Bundled	Hardware only
Marginal cost per query	\$0 (electricity only)	Subscription (capped daily)	Subscription	None	\$0 (electricity only)
Cost of supporting 5 concurrent users	Same hardware	\$100/month (5 subscriptions)	\$100/month (or 5 Prime)	5 Apple devices	Same hardware
Hardware requirement	NVIDIA GPU, 16 GB RAM	Smartphone or computer	Echo device + internet	Apple device	NVIDIA GPU, 16 GB RAM
Lock-in to vendor ecosystem	❌ None	✅ OpenAI account/billing	✅ Amazon ecosystem	✅ Apple ecosystem	❌ None

4. What is unique to RENT-A-HAL

Three features are not, to our knowledge, available in any other voice AI product as of June 2026.

4.1 Federation between independently-owned realms

Multiple RENT-A-HAL realms, each running on independent hardware owned by different operators, can transparently share LLM and TTS capacity. When one operator's local engine is busy or has failed, queries route automatically to a healthy peer realm; when local recovers, routing returns to local automatically.

This is **not** the same as a cloud service routing queries to its own datacenter. The peers are independently owned, independently configured, and independently administered. The federation operates on a cooperative voluntary basis with protocol-level cascade prevention (one-hop limit) and honest self-reporting (a realm with a dead local engine announces its own degradation).

No comparable mechanism exists in ChatGPT, Alexa+, Siri, Google Assistant, or any documented Home Assistant deployment. The MTOR Federation white paper documents the architecture in full.

4.2 Self-healing degraded-state watchdog

A wall-clock watchdog caps how long the realm can rely on peer federation before declaring its own local engine **degraded**. When the cap is reached (default 5 minutes), federation is refused, the LLM chain falls through to Echo (the operator-visible "I can't help" floor), the TTS chain falls through to pyttsx3, and a loud signal is raised to the operator: "your local engine needs attention."

This mechanism produces honest signaling. A consumer voice assistant that silently routes every query to a remote service is hiding the truth from its owner when the local component is broken. The MTOR watchdog ensures the operator sees the truth and acts on it. The self-healing component fires the moment local engines recover — degraded clears automatically on the first successful local query.

4.3 Boot wizard with five hard gates

RENT-A-HAL's boot wizard verifies the local environment before permitting realm launch. Five conditions must be satisfied:

1. CUDA-capable NVIDIA GPU detected

2.GPT4All API responding

3.At least one model loaded

4.**The selected model actually answers a test prompt within 15 seconds** (catches the "loaded but broken" failure mode where a model file initializes but does not generate)

5.Federation URL configured (only when federation is enabled)

The fourth gate — the **completion probe** — is unusual. Most installer-based AI stacks check that components are present but do not verify they work end-to-end. Operators who have spent hours debugging a model that "loads but doesn't generate" recognize the value immediately.

5. Where alternatives are better than RENT-A-HAL

A comparison paper that names only the wins is not a comparison paper. Each alternative is genuinely better than RENT-A-HAL in specific dimensions, and operators should choose accordingly.

5.1 ChatGPT Advanced Voice Mode is better for natural conversation

OpenAI's Advanced Voice Mode processes audio natively — no STT/TTS conversion — preserving emotional tone, prosody, and conversational flow. The voices are more natural than current open-source TTS. Response times are typically under three seconds. Vision-in-Voice allows the model to see through the phone camera while talking.

For an operator whose primary criterion is "the most natural conversational voice AI experience available," ChatGPT Plus at \$20/month is hard to beat. RENT-A-HAL's pipeline architecture (Whisper-style STT → LLM → Kokoro TTS) is competent but does not match native-audio LLM naturalness.

5.2 Alexa+ and Google Gemini for Home are better for smart home integration

Alexa+ supports approximately 100,000 smart home devices. Gemini for Home supports approximately 50,000. Both have mature routine engines, dozens of skill marketplaces, and tight integration with consumer hardware (smart speakers, displays, plugs, sensors, cameras). For an operator whose primary use case is "voice control of my smart home," neither RENT-A-HAL nor a Home Assistant local stack will match the breadth of integrations available out of the box.

5.3 Siri is better for Apple-ecosystem operators

For an operator already deep in the Apple ecosystem — iPhone, iPad, Mac, Apple Watch, HomePod — Siri offers seamless device handoff, strong on-device privacy, and zero additional setup. The Apple Intelligence upgrade (when it arrives) will further improve this. For these operators, Siri is the obvious starting point.

5.4 Home Assistant + Ollama is better for home automation first

The Home Assistant Assist pipeline with Whisper, Piper, and Ollama is the closest comparator to RENT-A-HAL in design intent — fully local, no cloud, open source. Where it is better: it integrates with **2,000+ home automation device types**, it has a mature **prefer_local_intents** mechanism that handles common commands without engaging the LLM (sub-second latency for "turn off the lights"), it has a much larger community, and it benefits from years of accumulated configuration examples. Operators whose primary use case is **device control with voice** should consider it strongly.

Where RENT-A-HAL is different: it is voice-first general AI, not voice-augmented home automation. The two products optimize for different primary use cases.

5.5 Setup time

RENT-A-HAL's wizard reduces setup time relative to a DIY Home Assistant + Ollama stack, but cloud assistants are still faster to get running. An iPhone user has Siri working immediately; an Echo user has Alexa+ working in ten minutes. RENT-A-HAL's roughly 30-minute installation is reasonable for a self-hosted product but is longer than the cloud alternatives.

6. Who RENT-A-HAL is for

RENT-A-HAL Personal Edition is a strong choice for operators who match one or more of these profiles:

The privacy-first individual or household. Conversations should not leave the building, full stop. The household has a CUDA-capable GPU available or is willing to acquire one. The privacy beacon and on-device STT are valued features, not afterthoughts.

The small business or professional with regulated workloads. An attorney, therapist, accountant, or healthcare professional handling client information that cannot be transmitted to OpenAI or Amazon. The realm runs on the office's own hardware behind its own firewall. Federation is disabled for privacy or enabled only with trusted peer realms.

The household with multiple concurrent users. Multiple family members using voice queries simultaneously, with no per-user subscription cost. The realm scales by serving the household from one GPU, not by paying \$20/month per family member.

The technically curious operator. An engineer, hobbyist, or self-hoster who values understanding the system they run, who appreciates that the entire source is readable in an afternoon, who wants to extend the realm's behavior or integrate it with custom infrastructure.

The federation participant. An operator who wants to be part of a small, voluntary network of like-minded operators sharing capacity. The methodology — push-driven check-ins, honest self-reporting, bounded silent dependency — is designed for this.

6.1 Who RENT-A-HAL is not for

In the spirit of honest comparison, RENT-A-HAL is **not** a strong choice for:

- Operators who prioritize the most natural conversational voice quality available (use ChatGPT Advanced Voice Mode)
- Operators whose primary use case is smart home device control (use Alexa+, Gemini for Home, or Home Assistant)
- Operators deeply embedded in the Apple ecosystem who want seamless mobile integration (use Siri)

- Operators without access to a CUDA-capable GPU (use any of the cloud options)
 - Operators who do not want to manage a self-hosted system at all (use any of the cloud options)
-

7. The honest summary

RENT-A-HAL Personal Edition is a **specialized product**. It is the right answer for a specific kind of operator: one who values self-hosting, privacy, voluntary federation, and the option to run voice AI without per-query costs or vendor dependencies.

It is not the most natural-sounding voice assistant — ChatGPT Advanced Voice is. It is not the best smart home controller — Alexa+ is. It is not the most convenient mobile assistant — Siri is. It is not the largest community — Home Assistant is.

What it is, uniquely:

- The only voice AI product with federation between independently-owned realms
- The only voice AI product with a wall-clock degraded watchdog and honest operator signaling
- The only voice AI product with a boot wizard whose model-completion probe catches the "loaded but broken" failure mode before launch
- A fully open-source, voice-first, multi-user local AI realm with no subscription, no central control plane, and no vendor lock-in

For operators who recognize themselves in Section 6 — the privacy-first household, the regulated professional, the multi-user family, the technically curious, the federation participant — these properties are not minor differentiators. They are the entire point of choosing this product over alternatives.

For everyone else, the comparison matrix in Section 3 documents the trade-offs honestly. Choose the product that fits your actual use case.

8. Methodology notes

Pricing and feature claims for comparators in this paper reflect public information as of June 2026, including but not limited to:

- OpenAI ChatGPT** pricing pages and Voice Mode FAQ (verified May 2026)
- Amazon Alexa+** launch announcement (February 2026)
- Apple Intelligence** roadmap (iOS 26.4 / iOS 27 expected delivery)
- Google Gemini for Home** transition from Google Assistant (early 2026)
- Home Assistant Assist** documentation and community implementations (2025-2026)

Voice assistant factual-accuracy benchmarks cited (Google 92%, Siri 78%, Alexa 75%) are drawn from independent reviews published in early 2026 and should be considered approximate. Smart home device counts are vendor self-reported (Alexa 100,000+, Google 50,000+, HomeKit ~1,000+, Home Assistant 2,000+ integrations).

RENT-A-HAL feature claims in this paper reflect the v1.0 release. Future revisions will update the comparison as both RENT-A-HAL and the comparator landscape evolve.

9. References

Document	Audience	Purpose
MTOR Federation White Paper	Engineers	Detailed federation architecture
RENT-A-HAL Quick Start Guide	First-time operators	Installation and first-boot procedure
RENT-A-HAL Operator's Manual	Active operators	Ongoing operation and tuning
RENT-A-HAL Design and Capabilities Paper	Decision makers	Broader product context
CHECKPOINT.md (in source ZIP)	Engineers	Snapshot of current architecture

RENT-A-HAL SOHO Multi-User '26

Quick Start Guide

Version 1.0 | June 2026 | N2NHU Labs

1. About this guide

This guide walks a first-time operator from a clean Windows 11 system to a working multi-user voice AI realm in under thirty minutes. It assumes no prior familiarity with RENT-A-HAL, federation networking, or local large-language-model hosting.

You will install two components, run a one-screen configuration wizard, and have a working realm serving voice queries on your local network within the same session.

2. What you are installing

RENT-A-HAL Personal Edition consists of two independently installed Windows applications. Both must be present for full functionality.

Installer	Purpose	Default port
KOKORO RENT-A-HAL VOICE Microservice	High-quality neural text-to-speech (TTS) backend. Runs as a persistent local service.	9998
RENT-A-HAL SOHO Multi-User '26	The realm itself: web cockpit, voice loop, LLM provider chain, federation client, boot wizard.	9999 (realm) / 9100 (wizard)

A third component is required but not bundled: a local large-language-model server (**GPT4All**, downloaded separately from gpt4all.io). GPT4All is the recommended local LLM engine; the realm's boot wizard verifies its presence before launch.

3. System requirements

Component	Minimum	Recommended
Operating system	Windows 11	Windows 11 (current build)
CPU	4-core x64	6-core x64 or better
RAM	16 GB	32 GB
GPU	NVIDIA with CUDA support, 6 GB VRAM	NVIDIA RTX 4060 or better, 8 GB VRAM
Storage	20 GB free	40 GB free (allows multiple LLM models)
Network	Broadband residential connection	Symmetric fiber, optional
Browser (cockpit)	Chrome or Edge (current)	Chrome or Edge (current)

A CUDA-capable NVIDIA GPU is required for usable local inference performance. Operators without a local GPU can run cloud-only with API keys for Claude or HuggingFace by setting `[Hardware] require_cuda = false` in `config.ini` after installation.

4. Pre-installation checklist

Before running either installer, complete the following:

- 1.Install NVIDIA drivers.** Verify with `nvidia-smi` at a command prompt. The command must return a GPU listing.
- 2.Install GPT4All.** Download from gpt4all.io. After installation, open GPT4All, download a model (Llama 3.2 3B Instruct is the recommended default), and enable the Local Server in **Settings** → **Application** → **Enable Local Server**.
- 3.Verify GPT4All is reachable.** A browser visit to `http://localhost:4891/v1/models` should return a JSON listing of loaded models.
- 4.(Optional) Install ngrok.** Download from ngrok.com. Required only if you intend to make your realm reachable from outside your local network or participate in the public federation. Free tier is sufficient for evaluation; a reserved domain (paid tier) is recommended for production.

5. Installation

5.1 Install the Kokoro voice microservice

- 1.Run **KOKORO RENT-A-HAL VOICE Microservice.exe**.
- 2.Accept the default installation path or choose your preferred location.
- 3.Allow the installer to register Kokoro as a Windows service. This ensures Kokoro starts automatically with the system.
- 4.On completion, verify the service is running: open Services (`services.msc`) and confirm **Kokoro RENT-A-HAL Voice** shows status **Running**.
- 5.Verify the API endpoint: a browser visit to `http://localhost:9998/health` should return a JSON status response.

The Kokoro microservice is now ready. It will be detected automatically by the realm.

5.2 Install the RENT-A-HAL realm

- 1.Run **RENT-A-HAL SOHO Multi-User '26.exe**.
- 2.Accept the default installation path or choose your preferred location.
- 3.The installer creates a Start Menu shortcut titled **RENT-A-HAL Wizard**.
- 4.On completion, the installer offers to launch the wizard immediately. Accept this option.

6. First boot: the wizard

The wizard is the recommended path for all first-time launches. It is also recommended for ongoing operation — it surfaces operational state, gates the realm launch behind a series of health checks, and provides single-button HALT for clean shutdown.

6.1 What the wizard does

The wizard runs as a small web application on port 9100. It opens automatically in your default browser when launched. The wizard performs the following functions:

- **Verifies the local environment** before allowing realm launch
- **Probes all required components** (CUDA GPU, GPT4All API, model availability, model functionality)
- **Configures the realm** through a single-screen form covering federation, branding, voice loop, and API keys
- **Launches the realm** as a managed child process
- **Manages the ngrok tunnel** as a child process (when configured)
- **Provides HALT control** for clean shutdown of both realm and tunnel
- **Surfaces operator-visible state** during operation

6.2 The five hard gates

The wizard's GO button is disabled until all of the following conditions are satisfied. Each failed gate displays an actionable error message indicating what to fix.

Gate	Verifies	Common remediation
1. CUDA detected	device returns at least one GPU	Install or update NVIDIA drivers
2. GPT4All API reachable	GET <code>http://localhost:4891/v1/models</code> returns 200 OK	Start GPT4All; enable Local Server in Settings
3. Model loaded	The API returns at least one model in its response	Download a model in GPT4All; recommended: Llama 3.2 3B Instruct
4. Model functional	A test prompt to the selected model returns non-empty text within 15 seconds	Select a different model; broken model files load but do not generate
5. Federation URL configured	If federation is enabled, the ngrok reserved URL is provided	Either set the URL or disable federation for standalone operation

The fourth gate — **completion probe** — distinguishes RENT-A-HAL from typical installer-based AI stacks. Many model files load successfully but do not produce output. The wizard sends a deterministic test prompt and verifies a real response before permitting launch.

6.3 The wizard configuration form

A single screen surfaces all common configuration:

Section	Fields	Purpose
Hardware	CUDA gate status	Live indicator of GPU detection
Local AI Engine	Endpoint, model selection, test prompt result	Select which model the realm will use
Federation	Participate (on/off), Federator URL, realm public URL, overflow thresholds, request timeout	Configure peer participation
Voice Loop	Wake timer default, privacy beacon	Set wake-word behavior and ambient signaling
Identity	Realm name, cockpit tagline	Customize the cockpit branding
API Keys (Optional)	OpenWeather, ngrok auth token, ngrok reserved URL	Enable optional integrations

All values persist to `config.ini` and `federation.ini` immediately on save, using a comment-preserving writer. Schema annotations in the INI files document every field. Power operators may edit the INI files directly; the wizard reads and writes the same file consistently.

6.4 Launching

Once all five gates are green, the GO button activates. Clicking GO performs the following sequence:

- 1.Persists all wizard form values to `config.ini` and `federation.ini`
- 2.Launches the realm process as a child of the wizard
- 3.Launches the ngrok tunnel as a child of the wizard (if configured)
- 4.Re-renders the wizard page as a dashboard showing realm status, cockpit URL, and a HALT button

The cockpit is then accessible at `http://localhost:9999` (or your configured port). On the same network, other devices reach it at `http://<host-ip>:9999`. If ngrok is configured, your reserved public URL routes through to the same cockpit.

6.5 HALT

Clicking HALT in the wizard cleanly terminates both the realm subprocess and the ngrok subprocess. SIGTERM is sent first; SIGKILL is used as a fallback if processes do not exit within three seconds. This is the recommended shutdown procedure. Closing the wizard browser tab does **not** stop the realm; the realm continues running until HALT is invoked or the wizard process itself is terminated.

7. Federation from first bringup

The federation is the design feature that distinguishes RENT-A-HAL from single-instance local AI hosting. **Operators are encouraged to enable federation from first bringup** for the following reasons.

7.1 What federation provides

Federation allows independently-owned RENT-A-HAL realms to share LLM and TTS capacity transparently. Each realm runs its own local engine on its own hardware; federation activates only when a query exceeds local capacity or when local engines have failed.

Condition on your realm	Without federation	With federation
Local engine is fast	Local serves the query	Local serves the query (identical behavior)
Local engine is busy	Query queues; user waits	Excess routed to a peer; user sees no delay
Local engine has crashed	"No LLM online" message	A peer serves the query; user sees no interruption
Local engine has been dead for >5 minutes	Indefinite error state	Watchdog signals operator; realm falls to Echo floor
Local engine recovers	Restart required to clear error state	Automatic on next query; routing returns to local

The federation is a **pure improvement** over single-instance operation. No query is sent to a peer that local can serve well. No identifying information is transmitted beyond the query text itself.

7.2 Recommended federation configuration

For first-time operators:

Field	Recommended value	Rationale
Participate in the federation	On	All benefits above
Federator URL	https://rentahal.com/api/federate	Public Federator maintained by N2NHU Labs
Your public realm URL	Your ngrok reserved domain	Required if you want to contribute capacity
LLM overflow threshold	31.0 seconds	Standard for the public mesh
TTS overflow threshold	31.0 seconds	Standard for the public mesh
Federated call timeout	59.0 seconds	Generous timeout, prevents premature fallback

Operators may participate as **consumers only** (receive rescues, do not serve peers) by leaving the public realm URL blank. Federator advice will route to other peers but no peer will send queries to your realm. This is appropriate for laptops, intermittent connections, or evaluation environments.

7.3 Cooperative behavior is enforced, not assumed

The federation is built on three protocol-level guarantees that protect all participants:

- One-hop limit.** A federated call cannot itself trigger further federation. Cascades are impossible by construction.
- Honest self-reporting.** A realm with a crashed engine announces its own degradation, so peers do not route to it.
- Bounded silent dependency.** A realm whose local engine is dead cannot federate forever — the watchdog enforces a maximum continuous federation period (default 5

minutes) before forcing fallback to a visibly degraded local floor. This protects peer capacity and ensures operator awareness.

These protections operate automatically. No operator action is required to benefit from them or to comply with them.

7.4 Privacy considerations

When federation is enabled and your realm receives a federated call from a peer, **your local LLM or TTS will generate the response**. The text of that query crosses the federation. No user identity, no session state, no browser context, and no audio is transmitted. Operators with privacy-sensitive workloads who do not want their realm's hardware to serve other operators' queries should disable federation in the wizard.

8. Operating the realm

8.1 The cockpit

Open `http://localhost:9999` in Chrome or Edge. The cockpit presents:

Element	Function
Status strip	Live indicators for WebSocket, LLM provider, TTS provider, wake state, query queue depth, average response times, token throughput
Federation strip	Federation state (HEALTHY / DEGRADED / OFF), Federator URL, current peer recommendations
Microphone visualization	Audio waveform and recognition state
Conversation area	User queries and realm responses, with per-query metadata (tokens, throughput, provider)
Action buttons	Enable wake word, type a question, stop, silence, repeat, news, roundup, weather, music, services, help

The cockpit is multi-user. Multiple browser tabs or devices may connect simultaneously; each has its own session state and outbound queue.

8.2 The voice loop

Click **Enable wake word**. The browser requests microphone permission. Once granted, the wake-word recognizer listens locally (browser-side speech recognition; no audio is sent to the realm until the wake word is detected).

Say "**computer**", then your query. The realm transcribes, classifies the intent, dispatches to the appropriate handler (weather, news, music, service finder, or general LLM query), generates a response, and speaks the response through your selected TTS provider.

Doubled action words — "**stop stop**", "**repeat repeat**", "**cancel cancel**" — provide interruption control without false-positive triggers from incidental speech.

8.3 Intent realms

The realm classifies each query into an intent and dispatches accordingly:

Intent trigger	Behavior
Weather	OpenWeatherMap one-call lookup at the browser's geolocation
News	Multi-source news aggregation with ROUNDUP playlist option
Music	DuckDuckGo search for matching YouTube video, embedded in cockpit
Service finder	OpenStreetMap-based local business lookup at browser geolocation
General query	Routed to the LLM provider chain

Each intent has its own configuration section in `config.ini` and can be disabled independently.

8.4 The federation badge

Below the LLM and TTS indicators, the cockpit shows the federation state. This includes the current Federator URL, your realm's health state, and the most recent peer recommendations. The per-query badge updates with the actual provider that served each query — operators can immediately distinguish queries served by local engines, by overflow federation, by rescue federation, or by the degraded floor.

9. Operational tasks

9.1 Stopping the realm

Open the wizard (Start Menu → RENT-A-HAL Wizard). Click HALT. Both the realm and the ngrok tunnel terminate cleanly.

9.2 Restarting the realm

Open the wizard. Click GO. The wizard re-runs all five health gates and launches the realm.

9.3 Changing the model

Stop the realm. In the wizard, select a new model from the dropdown (populated from GPT4All's loaded models). Click GO. The new model is verified by the completion probe before launch.

9.4 Updating configuration

Configuration changes via the wizard form take effect on the next launch. For configuration changes not exposed in the wizard, edit `config.ini` or `federation.ini` directly (they are well-commented and schema-annotated) and restart the realm.

9.5 Monitoring federation health

The cockpit's federation strip is the primary indicator. The realm's log file (`webgui_detailed.log`) records every check-in, every federation event, every degraded transition, and every recovery. ERROR-level lines indicate operator-actionable issues.

9.6 Diagnosing a degraded state

If the cockpit shows `LocalSystemLLMDegraded.xpy` or `LocalSystemTTSDegraded.xpy`, the watchdog has detected that your local engine has been continuously federated to peers for longer than the configured threshold. Check that GPT4All (for LLM degradation) or the Kokoro service (for TTS degradation) is running and responsive. The watchdog will clear automatically on the first successful local query.

10. Where to get help

Resource	Location
Configuration schema	Comments in and
Wizard error messages	Each failed gate provides specific remediation guidance
Detailed log	in the installation directory
Federation architecture	MTOR Federation White Paper (separate document)
Project page	The realm's page, when operator-enabled

11. Quick reference

11.1 Default ports

Component	Port
Realm cockpit	9999
Boot wizard	9100
Kokoro voice microservice	9998
GPT4All API (external)	4891

11.2 Default configuration files

File	Purpose
	Master realm configuration
federation.ini	Federation client and Federator configuration

11.3 Recommended first-bringup sequence

1. Install NVIDIA drivers; verify with `nvidia-smi`
2. Install GPT4All; download a model; enable Local Server
3. Install KOKORO RENT-A-HAL VOICE Microservice
4. Install RENT-A-HAL SOHO Multi-User '26
5. Launch the wizard from the Start Menu
6. Confirm all five hard gates show green
7. Enable federation; set the public Federator URL
8. Click GO
9. Open the cockpit at `http://localhost:9999`
10. Click **Enable wake word**; say "computer, what's the weather"

If step 10 produces a spoken answer, the realm is operational.

12. Acknowledgments

RENT-A-HAL Personal Edition is developed at N2NHU Labs (Newburgh NY).

The MTOR designation references Dr. Daystrom's M-5 Multitronic Unit from Star Trek: The Original Series, "The Ultimate Computer" (1968). The system aims to fulfill the M-5 design vision — a self-contained intelligent realm capable of autonomous operation — while preserving operator oversight.

This document is the v1.0 quick start guide. Subsequent revisions will be available with each release.

👉 Live long and prosper.

RENT-A-HAL Personal Edition — Operator's Manual

Version: v26.16.06 (Federation v1) **Maintained by:** Jim Ames, N2NHU Lab, Newburgh NY USA **Last updated:** June 19, 2026

What RENT-A-HAL is

A voice-first, locally-hosted AI realm. You talk to it like the Star Trek computer. It answers using local Llama 3 8B (free, private, your GPU) and speaks responses through Kokoro TTS. Optional features include news, weather, music, business search, and federation with peer realms over the public internet.

The 30-second mental model:

- Browser opens `https://your-realm.com`
- You say "computer, [thing you want]"
- Browser sends transcribed text to your realm via WebSocket
- Realm classifies intent (ask / weather / news / music / service / roundup)
- Routes to the right handler (Llama for "ask", structured services for others)
- Streams synthesized voice back through your speakers

Everything runs on your hardware. No API keys. No cloud LLM. No tokens charged.

Part 1: Quick start for users

Talking to RENT-A-HAL

Wake word is "**computer**". Say it, pause, ask your thing, say "**newline**" (or "submit") to send. Examples:

You say	What happens
	Llama answers "Paris"

You say	What happens
computer weather newline	Weather panel opens for your location
	News panel opens with NPR/Guardian/etc. headlines
computer roundup newline	Radio-style auto-shuffle of news stories
	Music panel opens with YouTube embed
computer find a pharmacy newline	Service-finder shows nearby pharmacies on map

Stopping things

Two ways:

- **SILENCE button** (big red button): immediate hard stop — pauses TTS, clears music, ends roundup
- Say **"stop stop"** anytime, no "computer" wake word needed

Buttons in the cockpit

Button	What it does
Enable wake word	Turns on continuous voice listening
Type a question	Opens a text box if you don't want to speak
STOP STOP	Same as voice "stop stop"
SILENCE (red)	Hard kills all audio
REPEAT REPEAT	Replays the last answer
News	Opens interactive news panel
ROUNDUP (blue)	Starts radio-style auto-shuffle
Weather	Quick weather for your location
Music	Opens music search

Button	What it does
Services	Opens service finder
Help	Shows in-app guide

Part 2: Installation and first boot

Hardware requirements

Component	Minimum	Recommended
OS	Windows 11	Windows 11
Python	3.12	3.12
RAM	16 GB	32 GB+
GPU	RTX 2060+ with 8 GB VRAM	RTX 4060+ with 8 GB VRAM
Storage	20 GB free	50 GB+ free (for models, audio cache)
Network	Any home internet	Symmetric business circuit

First-time setup

```
REM Extract the ZIP somewhere – example: C:\Users\you\Desktop\RENTAHAL_FULL
```

```
cd C:\Users\you\Desktop\RENTAHAL_FULL\RENTAHAL_APP
```

```
REM Create virtual environment
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
REM Install dependencies
```

```
pip install -r requirements.txt
```

```
REM Drop in operator-supplied static assets:
```

```
REM static\bgame.png          (cockpit background)
```

```
REM static\startup.mp3       (boot tone)
```

```
REM static\thinking.mp3      (thinking sound)
```

```
REM static\newsroundupaudioalert.mp3 (ROUNDUP intro – optional, synth fallback)
```

```
REM Verify install
```

```
python smoke_test.py
```

```
REM Should report "24/24 checks passed"
```

```
REM Start the realm
```

```
python app.py
```

```
REM Listening on http://0.0.0.0:9999
```

Browser to <http://localhost:9999>. You should see the BETA-5 System Terminal.

Optional: Kokoro TTS service (separate process)

Kokoro runs as a microservice on port 9998. From a second terminal:

```
cd ..\kokoro_service
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
python service.py
```

Then in RENTAHAL_APP\config.ini set [KokoroService] host = localhost. The realm proxies to it automatically.

Critical CUDA gotcha

If you install both onnxruntime AND onnxruntime-gpu together, the CPU version shadows GPU and Kokoro runs on CPU (slow). Fix:

```
pip uninstall -y onnxruntime onnxruntime-gpu
```

```
pip install onnxruntime-gpu
```

The realm logs a warning at boot if it detects this. Honor it.

Part 3: Voice operations guide

How the wake/listen state machine works

```
[wake off] → click "Enable wake word" → [waiting for "computer"]
```

```
[waiting] → hear "computer" → [listening, 10s silence timer running]
```

```
[listening] → say things → buffer fills, 10s silence timer resets on each word
```

```
[listening] → say "newline" or "submit" → submit buffer, → [waiting]
```

```
[listening] → 10s silence with no words → reprompt, → [waiting]
[listening] → say "stop stop" anywhere → cancel buffer, → [waiting]
```

Valid submit words

Any of these end your phrase and submit it: newline, submit, end, enter, go, done. Pick whichever feels natural.

Voice commands cheat sheet

WAKE	computer
SUBMIT	newline submit end enter go done
STOP	stop stop (no wake needed)
REPEAT LAST	repeat repeat (no wake needed)
HELP	help help (no wake needed)
INTENTS	(after wake word)
ask anything	computer, what is X newline
weather	computer weather newline
news (interactive)	computer news newline
news roundup	computer roundup newline computer news roundup newline computer play roundup newline
music search	computer music <song or artist> newline
service finder	computer find a <category> newline computer find me a <category> newline computer service finder <category> newline
gmail (if configured)	computer gmail newline computer email newline

Doubled action words protect against false triggers

"Stop" alone won't fire (might appear in a sentence). "Stop stop" is required. Same for "repeat repeat" and "help help". Designed-in protection — not a bug.

Part 4: Configuration reference

config.ini (main realm config)

Lives in RENTAHAL_APP\config.ini. Sections you'll touch:

[Server] — Where the realm listens

```
host = 0.0.0.0          # 0.0.0.0 = all interfaces; 127.0.0.1 = local only
port = 9999             # default; behind ngrok or Cloudflare Tunnel
```

[LLM] — Language model providers

```
providers = gpt4all      # priority list; first available wins
# Other options: claude, huggingface, echo (for testing)
```

[GPT4All]

```
host = localhost
port = 4891
model = Llama 3 8B Instruct
```

[TTS] — Text-to-speech providers

```
providers = kokoro_http  # chain order; first available wins
# Other options: kokoro, elevenlabs, openai, sapi5, pyttsx3, echo
```

[KokoroService]

```
host = localhost        # leave blank to disable
port = 9998
```

[News], [Weather], [Music], [Service]

See in-file comments — each enables/disables a feature realm.

federation.ini (federation config)

Lives in RENTAHAL_APP\federation.ini. Only loaded if the file exists.

[Federation]

```
desire = off                # opt-in switch
master_url = https://rentahal.com/api/federate # the Federator URL
self_url =                  # YOUR realm's public URL
prefer_federation_for_llm_avg_s = 31.0        # threshold to overflow
checkin_interval_s = 15.0    # push frequency
```

[Federator]

```
enabled = false            # true on the Federator only
```

See Part 7 for full federation operations.

Part 5: Public APIs and integration

These endpoints are public on your realm. CORS is `allow_origins="*"` by default — any web page can call them. Designed this way intentionally (see the "Little Johnny" demo at https://rentahal.com/static/little_johnny_first_ai_app.html).

POST /api/ask — One-shot text-in/text-out LLM

The simplest API. The whole demo is 20 lines of HTML.

Request:

```
{"text": "What is the capital of France?"}
```

Response:

```
{
  "intent": "ask",
  "answer": "Paris is the capital of France.",
  "provider": "GPT4All Llama 3 8B Instruct"
}
```

Special header — `X-Federation-Hops: <n>`: if present and ≥ 1 , the receiving realm bypasses its OWN federation provider (runs locally only). This is the hop-limit protection — used by federated calls to prevent cascades.

Example:

```
curl -X POST https://your-realm.com/api/ask \
  -H "Content-Type: application/json" \
  -d '{"text":"Hello"}'
```

GET /api/setup_status — Realm health check

Returns whether providers detected, models loaded, etc. Boring but essential.

GET /api/config — Public configuration dump

Operator-visible knobs (sanitized — no secrets). Useful for diagnostics.

GET /api/news?source=<S>&category=<C>&limit=<N> — News stories

Returns JSON of cached stories from the configured RSS feeds. Each story has title, source, audio_url (pre-rendered Kokoro WAV), link.

```
curl https://your-realm.com/api/news?source=NPR&limit=10
```

GET /api/news/feeds — Configured feeds

Returns the list of RSS sources the realm is ingesting.

GET /api/news/audio/<audio_id> — Streamed news WAV

Streaming proxy from the Kokoro service. The browser plays via <audio> tag.

GET /api/news/roundup?count=<N> — Shuffled radio-style roundup

Returns NPR + Guardian stories interleaved + shuffled, sliced to count (default 20). Backs the ROUNDUP button and voice trigger.

WebSocket /ws — Full conversation channel

The main two-way pipe. Browser opens, sends {type: "prompt_text", text: "..."}, realm streams back events: bus_query_processing, bus_query_result, bus_tts_chunk (base64 WAV), bus_intent_classified, etc.

The "Hear Llama Through Kokoro"

demo at https://rentahal.com/static/hear_llama_through_kokoro.html shows the complete WebSocket pattern in under 100 lines of HTML.

GET /api/federation/status — Federation cockpit data

Returns {desire, master_url, self_url, state, best_llm_peer, best_tts_peer, ...}. The cockpit polls this every 20s for the badge in the metrics area.

POST /api/federate/checkin (only on the Federator)

Members push their stats here. Federator responds with peer advice. See Part 7 for protocol details.

GET /api/federate/matrix (only on the Federator)

Operator-visible snapshot of the entire federation phonebook. JSON.

Part 6: Building apps against the public APIs

App #1 — One-button question asker

Save as app1.html, open in browser. **That's the whole app.**

```
<input id="q" placeholder="Ask anything">
<button onclick="ask()">Ask Llama 3</button>
<div id="out"></div>
<script>
async function ask() {
  out.textContent = "thinking...";
  const r = await fetch("https://rentahal.com/api/ask", {
    method: "POST",
```

```
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ text: q.value })
});
const data = await r.json();
out.textContent = data.answer;
}
</script>
```

App #2 — News reader

```
<button onclick="show()">Today's News</button>
<ul id="list"></ul>
<script>
async function show() {
  const r = await fetch("https://rentahal.com/api/news?limit=10");
  const data = await r.json();
  list.innerHTML = data.stories.map(s =>
    `<li>${s.title} - <em>${s.source}</em></li>`
  ).join("");
}
</script>
```

App #3 — Voice-and-speech chat with TTS

Open a WebSocket, send text, receive streamed TTS audio chunks. See the "Hear Llama Through Kokoro" demo for the full ~100-line pattern.

App pattern — when to use what

Need	Use
Just text in / text out	
Show news headlines	GET /api/news
Play news audio	
Full conversation with TTS	WebSocket /ws
Realm health monitoring	

Need	Use
Federation observability	GET /api/federation/status and /matrix

Part 7: Federation operations

When to enable federation

Federation lets multiple RENT-A-HAL realms share LLM capacity. When YOUR local LLM is slow (queue depth > threshold), your realm temporarily borrows a peer's /api/ask endpoint. The borrowed call's latency feeds back into your rolling average, creating natural backoff and reapproach.

Enable it when:

- You have a second realm online (yours or a friend's)
- You want overflow capacity during burst load
- You want to contribute capacity to the public federation

Don't enable it when:

- Your realm runs alone in a closed network
- You need every query to be local for compliance / privacy reasons
- You haven't set a public URL for `self_url` yet

Becoming the Federator

If you're hosting the federation coordinator (only ONE per network):

```
[Federation]
desire = on
master_url = https://yourdomain.com/api/federate
self_url = https://yourdomain.com

[Federator]
enabled = true
```

```
state_file = federation_state.json
missed_after_s = 60.0
peer_advice_bump_s = 0.01
```

Restart. Federator endpoints come live at `/api/federate/checkin` and `/api/federate/matrix`. The matrix persists to `federation_state.json` between restarts.

Joining as a member

```
[Federation]
desire = on
master_url = https://the-federator.example.com/api/federate
self_url = https://your-realm.example.com

[Federator]
enabled = false
```

Restart. The cockpit federation badge appears within ~25 seconds with Fed: ON · Master: the-federator.com · State: HEALTHY.

The check-in protocol

Every 15 seconds (+ random startup jitter), members POST to the Federator:

```
{
  "member_url": "https://your-realm.com",
  "llm_avg_s": 4.2,
  "tts_avg_s": 1.8,
  "tokens_out": 12345,
  "token": ""
}
```

Federator updates the matrix and responds:

```
{
  "best_llm_peer": "https://faster-peer.example.com",
  "best_tts_peer": null,
  "your_state": "HEALTHY",
  "matrix_size": 3
}
```

The member stores `best_llm_peer`. When local LLM avg exceeds `prefer_federation_for_llm_avg_s` (default 31s), the next LLM call routes to that peer's `/api/ask`.

Cockpit federation badge

Fed: ON • Master: rentahal.com • State: HEALTHY • (LLM→peer.example.com)

- Fed:** ON / OFF — your desire setting
- Master:** short hostname of the Federator (hover for full URL)
- State:** HEALTHY (recent successful check-in) / DEGRADED (4+ failed) / OFF
- Peers:** currently suggested peer URL(s) for overflow

Failure modes and recovery

Symptom	Cause	Fix
State stuck on OFF	or blank	Set both, restart
State stuck on DEGRADED	Can't reach Federator	Check master URL, network, ngrok
State HEALTHY but no peer in badge	Only 1 member in matrix (you)	Wait for another to join
Federated calls timing out	Peer overloaded	Automatic — local fallback kicks in
Endless federation cascade	Hop limit bug	Won't happen — protocol enforces

Observability

```
# See the matrix from the Federator
curl https://the-federator.com/api/federate/matrix

# See your own federation state
curl https://your-realm.com/api/federation/status
```

Part 8: Feature matrix

What's in v26.16.06

Feature	Status	Voice trigger	Button	API endpoint
Ask anything (Llama)	✓ Production		"Type a question"	
Weather	✓ Production	computer weather newline	"Weather"	(inline in /api/asl)
News (interactive)	✓ Production		"News"	
News ROUNDUP	✓ Production	computer roundup newline	 ROUNDUP	GET /api/news/roundup
Music search	✓ Production		"Music"	(WebSocket event)
Service finder	✓ Production	computer find a <X> newline	"Services"	(WebSocket event)
TTS streaming	✓ Production	(automatic)	—	WebSocket
Wake word	✓ Production	"computer"	"Enable wake word"	—
STOP STOP	✓ Production		"STOP STOP" /  SILENCE	—
REPEAT REPEAT	✓ Production	repeat repeat	"REPEAT REPEAT"	—
HELP HELP	✓ Production		"Help"	—
ACK protocol	✓ Production	(under hood)	—	(under hood)
Federation v1 (LLM)	✓ Production	(automatic overflow)	(cockpit badge)	
Federation v2 (TTS)	● Architected	—	—	(matrix carries tts_avg_ already)

Feature	Status	Voice trigger	Button	API endpoint
Multilingual TTS	● Future	—	—	—
Solana payment gate	● Future	—	—	—
Nuitka EXE compile	● Future	—	—	—

Audio playback features

Feature	Trigger	Stop
TTS response streaming	Automatic on every answer	SILENCE / stop stop
Boot tone	Page load	(one-shot)
Thinking sound	Long-running queries	(auto-stops on result)
News story play	Tap ► on a story card	SILENCE / pause
ROUNDUP intro tones	Start ROUNDUP	(one-shot)
ROUNDUP auto-advance	After each story ends	SILENCE / stop stop
Music (YouTube embed)	Music intent	SILENCE / stop stop

iOS / mobile support

Capability	iOS Safari	Android Chrome	Desktop
Wake word	✓	✓	✓
TTS playback	✓ (after first tap)	✓	✓
News audio	✓	✓	✓
ROUNDUP	✓	✓	✓
Music (YouTube)	✓	✓	✓
Geolocation (for	✓ (after permit)	✓	✓

Capability	iOS Safari	Android Chrome	Desktop
services)			
Background play	✗ (browser limit)	✗ (browser limit)	✓

iOS audio is unlocked by the first user tap on the page. The realm deliberately plays a silent WAV at first-tap to credit the gesture before the visible startup tone runs.

Part 9: Comparison matrix

RENT-A-HAL Personal Edition vs alternatives

Capability	RENT-A-HAL	OpenAI ChatGPT	Google Assistant	Alexa	LocalGPT/Ollama UIs
Voice in (STT)	Browser native	App / web	Native	Native	Usually none
Voice out (TTS)	Local Kokoro GPU	App TTS / cloud	Cloud	Cloud	Usually none
LLM	Local Llama 3 8B	Cloud GPT-4 / o3	Cloud Gemini	Cloud	Local — varies
Cost per query	\$0 (electricity)	\$0.01-0.50+	\$0 (data harvested)	\$0 (data harvested)	\$0
Privacy	Stays on your box	Sent to OpenAI	Google	Amazon	Stays local
Internet required	Only for federation	Yes	Yes	Yes	No
Customizable	Full source access	Closed	Closed	Closed	Varies
Multi-user	Yes (per-GUID state)	Per-account	Per-account	Per-account	Usually no
Federated overflow	✓ v1 production	(cloud is the overflow)	N/A	N/A	✗
Offline-capable	✓ (no fed required)	✗	✗	✗	✓

Capability	RENT-A-HAL	OpenAI ChatGPT	Google Assistant	Alexa	LocalGPT/ Ollama UIs
Wake word	"computer"	App-specific	"Hey Google"	"Alexa"	None
News/music/ services	✓ built-in	Via plugins	✓	✓	✗
Public API	Open	\$\$/key	Limited	Limited	Sometimes
Cost to operate	One-time hardware	Monthly subscription	Free (you're product)	Free (you're product)	One-time
Looks like Star Trek	✓	✗	✗	✗	✗

LLM provider options inside RENT-A-HAL

Provider	Cost	Speed	Privacy	When to use
GPT4All (Llama 3 8B local)	\$0	~20 tok/s on 4060	Total	Default — production
Claude API	\$\$	Fast	Anthropic processes	Cloud burst capacity
HuggingFace API	\$	Varies	HF processes	Specialty models
Echo (test stub)	\$0	Instant	Total	CI testing only
Federation (peer realm)	\$0	Network + peer	Trust peer	Automatic overflow

TTS provider options

Provider	Cost	Quality	Speed	GPU?
Kokoro (HTTP service)	\$0	Excellent	Fast	Yes (recommended)
Kokoro (inline)	\$0	Excellent	Slow on CPU	Yes
ElevenLabs API	\$\$\$	Best-in-class	Network-bound	No
OpenAI TTS API	\$\$	Very good	Network-bound	No

Provider	Cost	Quality	Speed	GPU?
Windows SAPI5	\$0	OK	Instant	No
pyttsx3	\$0	OK	Instant	No
Echo (test stub)	\$0	None	Instant	No

Part 10: Troubleshooting

"Page loads but no audio plays"

iOS only allows audio after a user gesture. **Tap anywhere on the page once.** After that all subsequent audio plays. The realm tries to unlock audio on the very first tap by playing a silent WAV; you should see "audio unlocked" in the browser console.

"Card displays randomly fail"

Was a real bug, fixed in this version. ACK protocol now confirms every user-visible event reaches the browser. If you see ACK MISSING warnings in `webgui_detailed.log`, that's the diagnostic — share that line with support.

"Federation badge stuck on DEGRADED"

Your check-ins to the Federator are failing. Check:

1. Is `master_url` correct? `curl https://master/api/federate/matrix` should return JSON.
2. Is `self_url` set? Blank = client doesn't even start.
3. Is your firewall blocking outbound HTTPS to the Federator?
4. Does your realm log show "Federation check-in failed" messages?

"Kokoro produces no audio"

Most common cause: both `onnxruntime` and `onnxruntime-gpu` installed. The CPU package shadows GPU. Fix:

```
pip uninstall -y onnxruntime onnxruntime-gpu
```

```
pip install onnxruntime-gpu
```

Look for [boot] CUDA active: True in the Kokoro service log.

"Music plays but YouTube iframe is gone after SILENCE"

That's by design. SILENCE removes the iframe entirely because YouTube's postMessage API isn't loaded (deliberate choice to avoid YouTube developer registration). To resume music, search again.

"Cockpit shows AI avg of 0 forever"

Either no queries have completed yet (ask one) or the realm is restarting after each query (check log for crashes). Federation won't activate without a non-zero local avg.

"ROUNDUP plays no intro tone"

If `static/newsroundupaudioalert.mp3` is missing, the system synthesizes a 3-tone BONG-BONG-BONG sequence via Web Audio API. Both work. To use a custom intro, drop your own mp3 at that path.

"Tests pass locally but I see failures after deployment"

Always run `python smoke_test.py` after deploying. If smoke fails but tests pass, the deployment is missing a config or asset. Check the smoke output for the specific failure.

Part 11: Where files live

RENTAHAL_APP/	
└─ app.py	Main launcher
└─ config.ini	Edit this for engines, ports, knobs
└─ federation.ini	Edit this for federation participation
└─ webgui_detailed.log	Boot log + runtime – your friend for debugging
└─ federation_state.json	Federator persistence (auto-generated)
└─ static/	Operator-supplied assets (mp3, png)
└─ templates/index.html	The cockpit
└─ realm/	Core modules
└─ └─ intent.py	Intent classifier (vocabulary lives here)

		orchestrator.py	Routes intents to handlers
		engine_chain.py	LLM/TTS provider chains
		federation.py	Federation matrix + client
		federation_provider.py	LLM provider that wraps peer /api/ask
		realm_core.py	ConnectionManager + ACK protocol
		tests/	948 assertions, 24 layers – keep them green

Part 12: Operator philosophy

A few principles that show up throughout this codebase. They're not laws, just patterns that have worked:

- 1.Provenance over guessing.** When something breaks, the log line that explains it should be findable. The ACK protocol's ACK MISSING warning is a good example — when an event doesn't reach the browser, you know exactly which event for which user.
 - 2.Configuration over code.** Federation thresholds, feed URLs, intent vocabularies, and matrix knobs all live in INI files. Code changes require restarts; INI tweaks require re-reads.
 - 3.Test before you ship.** The clean-room ZIP extract test (extract, run all tests, confirm green) is the validation bar. "It works on my machine" doesn't count.
 - 4.Triple-run for zero flakes.** Tests that pass once and fail twice are worse than tests that always fail. Run three times, demand identical results.
 - 5.Sabotage-verify critical invariants.** If a test couldn't fail when the protection is removed, it isn't really testing the protection.
 - 6.Honest disclosure.** When you find an old hidden bug, surface it. Operators downstream are running on what you've shipped.
 - 7.Open by default.** /api/ask is public for a reason — see the Little Johnny demo. Federation works because the surface is already shared.
-

Part 13: Support and escalation

Issue type	Where
Voice features broken	Check for the exact event
Federation not connecting	<code>curl /api/federation/status</code> on both ends
Kokoro not running	Check log + CUDA install
Performance degradation	Check Cockpit AI avg + TTS avg trend
New feature requests	(project decision tree — operator's call)

When asking for help, include:

- Realm version (v26.16.06)
- Last 50 lines of `webgui_detailed.log`
- Browser console output

- Cockpit metrics screenshot

Appendix A: The complete intent vocabulary

Single-word triggers (after "computer" wake)

weather, forecast	→ weather realm
gmail, email, mail	→ gmail realm (if configured)
news	→ news realm (interactive cards)
music	→ music realm
roundup	→ ROUNDUP (auto-shuffle)

Anything else after "computer" goes to the ask realm (Llama).

Phrase triggers (checked first, longer matches win)

service finder	→ service finder
find a <category>	→ service finder
find me <category>	→ service finder
find nearby <category>	→ service finder
news roundup	→ ROUNDUP

play roundup	→ ROUNDUP
start roundup	→ ROUNDUP
start the roundup	→ ROUNDUP
play the roundup	→ ROUNDUP

Wake-independent commands

stop stop	→ cancel current operation
repeat repeat	→ replay last answer
help help	→ show in-app help

Appendix B: Quick API reference card

PUBLIC ENDPOINTS (CORS open by default)

GET	/api/setup_status	→ realm health
GET	/api/config	→ config dump (sanitized)
POST	/api/ask	→ text in / text out
headers: X-Federation-Hops (optional)		
GET	/api/news?source=&category=&limit=	→ news stories JSON
GET	/api/news/feeds	→ configured feeds
GET	/api/news/audio/{id}	→ streamed WAV
GET	/api/news/roundup?count=	→ shuffled radio playlist
GET	/api/federation/status	→ cockpit federation state
WS	/ws	→ conversation channel

FEDERATOR-ONLY ENDPOINTS (when [Federator] enabled = true)

POST	/api/federate/checkin	→ push stats, get advice
GET	/api/federate/matrix	→ matrix snapshot

👉 **Live long and prosper.**

Built by Jim Ames at N2NHU Lab, Newburgh NY USA. RENT-A-HAL Personal Edition is open-source voice-first AI for one operator to run from one box and serve one household — or many, via federation.

RENT-A-HAL Personal Edition

User Manual & Theory of Operation

"Bringing the future to the present" — N2NHU Lab for Applied AI

Version 26.16.06 · Summer 2026 · Newburgh, NY

What This Is

RENT-A-HAL Personal Edition is a **voice-first, multi-user AI realm** that runs on a single Windows 11 box with an NVIDIA GPU. One machine, sitting on your desk, serving an entire home or small office. Voice in, voice out, all local, all yours.

You wake it by saying **"computer"**. You ask your question. You say **"newline"** to submit. The answer arrives spoken in a natural voice, with structured visual results when relevant — news tiles, music videos, business listings, weather summaries. Multiple users can hit it at once, each in their own session, each with their own state.

This document is the manual. It explains what RENT-A-HAL does, how it's built, and how you can extend it. There is no magic here — only careful engineering, told plainly.

The Hook: What You Get on Day One

You ask	What happens
"Computer, tell me about Apollo 11. Newline."	Llama 3 8B Instruct answers, Kokoro speaks it back in seconds
"Computer, weather in Tokyo. Newline."	OpenWeatherMap looked up, spoken summary, no LLM tokens spent
"Computer, news. Newline."	Today's headlines, tappable tiles, pre-cached audio for instant playback
"Computer, music London Calling. Newline."	YouTube embed of the Clash track, plays in-page
"Computer, service finder haircut."	Real barbershops near you, distance, hours, phone, map —

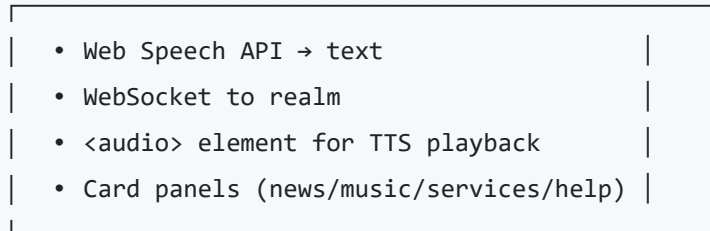
You ask	What happens
Newline."	from OpenStreetMap
"Stop stop."	Halts everything — TTS, music, services
"Help help."	Shows every voice command on screen

All of that runs on **one Win11 box** with an NVIDIA GeForce RTX 4060 (or better), accessed by any browser on your LAN — or from anywhere, via Cloudflare Tunnel or ngrok. iPhone Safari works. Android Chrome works. Desktop Edge works. Tested on hardware as old as **iPhone 8+ over LTE**.

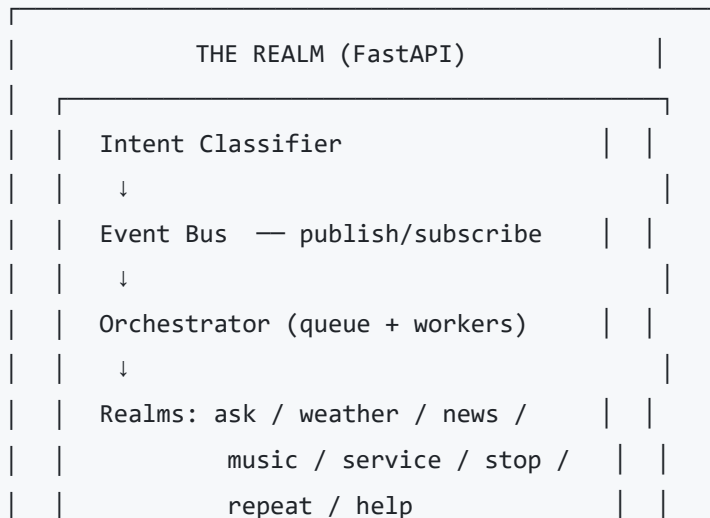
Theory of Operation

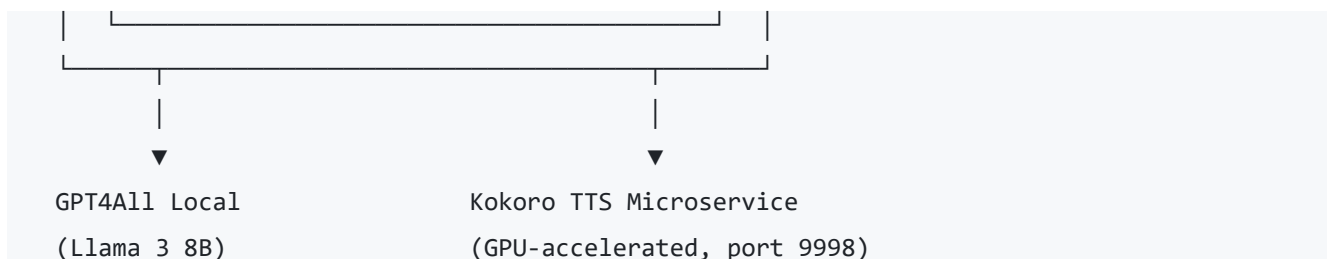
The big picture

Browser (Chrome / Safari / Edge)



| WebSocket (text in,
| audio bytes out)





Every piece is replaceable. The intent classifier is regex + n-gram scoring (no model needed). The orchestrator is plain async Python with a careful queue. The LLM is whatever provider you wire in — GPT4All is just the default. The TTS is its own process so it can be GPU-pinned and updated independently.

The Event Bus

The heart of the realm is a publish/subscribe **event bus**. Everything is a typed event with a payload. Nothing in the system calls anything else directly — components publish events, and other components subscribe.

```
await bus.publish("intent_classified",
                  user_guid=guid, kind="weather",
                  slots={"city": "tokyo"}, text=raw)
```

A handler that cares about weather subscribes once at startup:

```
await bus.subscribe("intent_classified", on_intent)
```

This decoupling is why the system is honest. Every realm minds its own business. Adding a new realm doesn't require touching any existing one. The browser sees the same events the realm does — every bus event is relayed to connected WebSockets prefixed with `bus_`, so the browser receives `bus_query_processing`, `bus_query_result`, `bus_tts_chunk`, `bus_metrics_snapshot`, and so on. The UI's job is to react to these events.

There are no callbacks, no chained promises, no shared mutable state. Just events.

The Orchestrator

The orchestrator is a single async worker loop reading from a bounded queue. When the intent classifier identifies a query, the orchestrator queues it (`pending`), pulls it (`in-flight`), routes it to the correct realm handler, captures the result, publishes `query_result`, then calls the TTS subsystem to speak it.

Three things make this clean:

1.**Per-user state isolation.** Each connected user has a `userState` keyed by GUID — their location, their last result, their active TTS cancellation token. One user's "stop stop" never affects another user's playback.

2.**Bounded concurrency.** Only one query per user at a time. The queue has a hard cap. The orchestrator can't be overwhelmed by a single user pasting 50 prompts.

3.**Metrics that tell the truth.** Queue depth, in-flight count, rolling average service time, cumulative tokens generated — all tracked atomically, broadcast to every connected browser every 2 seconds as a `metrics_snapshot` event. **Everyone sees what everyone else is queuing.** Transparency is the default.

Realms: small, focused, swappable

A "realm" is what RENT-A-HAL calls a domain handler. The seven realms in Personal Edition:

Realm	Backend	Tokens?
ask	Llama 3 8B Instruct via GPT4All local API	Yes (LLM)
weather	OpenWeatherMap	No
news	RSS feeds + Kokoro pre-cached audio	No
music	DuckDuckGo HTML scrape → YouTube embed	No
service	BizData (OpenStreetMap) + Nominatim geocoder	No
stop	Doubled wake word — halts TTS and YouTube	No
repeat	Doubled wake word — replays last answer	No

Only **ask** uses LLM tokens. Everything else is structured data plus a brief spoken summary. This is why the metrics panel shows "Tokens out: X" — it tracks **real local AI usage**, the kind that would cost you money if you were paying a cloud provider. Service Finder and Weather don't inflate the number, because they didn't call the LLM.

Compartmentalization & isolation

The realm process and the Kokoro TTS microservice are **separate operating system processes**. They communicate over HTTP localhost. This isn't an accident — it lets you:

- Restart Kokoro without dropping user sessions
- Pin Kokoro to GPU 0 and the LLM to GPU 1 (or both to the same GPU with VRAM headroom)
- Upgrade the TTS model independently of the realm
- Replace Kokoro entirely (Coqui, ElevenLabs, OpenAI TTS) by changing one config line

Inside the realm, the LLM and TTS are abstracted as **engine chains**. Each chain has a primary provider and unlimited fallbacks. If GPT4All goes down, you can list `openai` → `anthropic` → `gpt4all` and the chain tries them in order. Same pattern for TTS.

Every config knob lives in `config.ini` with declarative `@label`, `@type`, `@scope` tags so a future setup UI can render them automatically. There are zero hardcoded magic numbers in the realm code.

API-First Design: Little Johnny Writes His First AI App

Little Johnny is nine years old and wants to write an AI app that asks the household robot for jokes. His parents have RENT-A-HAL running in the basement. Here is his first AI app — five lines of HTML.

```
<!doctype html>
<button onclick="askJoke()">Tell me a joke!</button>
<div id="answer"></div>
<script>
async function askJoke() {
  const r = await fetch("http://homehal.local:9999/api/ask?prompt=tell+me+a+joke");
  const j = await r.json();
  document.getElementById("answer").textContent = j.text;
}
</script>
```

That's it. That's the first AI app. Llama 3 8B is doing the work in the basement. Johnny's browser is showing the result. He doesn't need an OpenAI API key. He doesn't need to install anything. He needs `homehal.local:9999` and a script tag.

What Johnny gets when he's ready for more

When Johnny is ten he learns about WebSockets and writes an app that streams the answer as it's generated:

```
const ws = new WebSocket("ws://homehal.local:9999/ws");
ws.onmessage = (e) => {
  const msg = JSON.parse(e.data);
  if (msg.type === "bus_query_result") {
    document.getElementById("answer").textContent = msg.text;
  }
  if (msg.type === "bus_tts_chunk") {
    // Audio bytes are base64 in msg.audio_b64 – playable in any <audio>
  }
};
ws.send(JSON.stringify({type: "prompt", text: "tell me a joke"}));
```

That's the full protocol. Every bus event is a JSON object. Every JSON object has a type. Johnny knows JavaScript — he can read the events as they arrive and react however he wants. He's now writing event-driven distributed AI software at the age of ten.

When Johnny is eleven and wants his own realm

He decides he wants RENT-A-HAL to know about his pet hamster. He opens `realm/intent.py` and adds:

```
PHRASE_INTENTS["hamster"] = "hamster"
```

He adds a handler in `app.py`:

```
async def hamster_handler(slots, guid, bus):
    return "Mr. Whiskers is sleeping. He always sleeps. He's a hamster."
```

He wires it into the orchestrator:

```
orchestrator = Orchestrator(bus, llm, tts,
    hamster_handler=hamster_handler,
    ...)
```

He says **"computer hamster newline"** and the realm answers. He has just built a custom voice-controlled AI feature. He is eleven.

This is what API-first design means: the boundary between "using the system" and "extending the system" is thin enough for a curious kid to cross.

Multi-User Design

RENT-A-HAL was built from day one to serve multiple users at once. The design choices that make this work are visible throughout the codebase:

Per-user identity

Every browser session generates a UUID (`user_guid`) stored in `localStorage`. The realm uses this to:

- Route TTS audio to the right user (no cross-talk)
- Track which user submitted which query
- Cache the user's location so service finder works on the first try
- Honor "stop stop" — Alice's stop doesn't silence Bob's playback

Per-user state, per-user queue

The orchestrator's queue is global but the in-flight tracker is per-user. Alice and Bob can both have queries running simultaneously. The metrics panel shows everyone the aggregate — "AI queue: 2" tells Bob that two people are using the system right now.

Connection management

A `ConnectionManager` keeps track of every connected `WebSocket`, pings them every 20 seconds, and reaps stale ones after 75 seconds. Dead sockets can't hurt anyone else. Restarting the realm boots everyone cleanly; they reconnect automatically when the server is back.

Honest multi-user transparency

When the AI queue is 0, the metric is green. When 1-2 queries are pending, it's amber. When 3+, it's red. Every user sees this in real time. **The system doesn't hide its busy state.** If grandma is asking about a recipe and grandpa wants to ask about Apollo 11, grandpa sees "AI queue: 1" before he even taps the wake button. He knows what to expect.

Tested under real conditions

The chaos test suite simulates dozens of users sending queries simultaneously, with deliberate failures injected into the LLM and TTS chains, to verify the orchestrator doesn't lose state, doesn't leak memory, doesn't crash. Tested. Passed.

The Hardware Story

One Windows 11 box. That's it.

- GPU:** NVIDIA GeForce RTX 4060 (8 GB VRAM). Anything Pascal or newer with at least 6 GB VRAM works.
- CPU:** Whatever you have. The realm itself is light Python.
- RAM:** 16 GB recommended.
- Disk:** 30 GB for GPT4All's Llama 3 8B model + Kokoro voice files + the rest of the stack.
- Network:** Any LAN. Optional: Cloudflare Tunnel or ngrok to expose to the public internet (rentahal.com runs this way).

This serves **an entire household or a small office** without breaking a sweat. Multiple iPhones, laptops, desktops — all hitting the same backend, getting answers in seconds.

The LLM: Meta's Llama 3 8B Instruct

The default model is **Llama 3 8B Instruct**, a state-of-the-art open-weights instruction-tuned model from Meta. It runs entirely on your GPU via GPT4All. Inference speeds of ~20 tokens/second on a 4060. Real conversational AI, in your own house, with no API costs and no data leaving your machine.

The realm doesn't care which LLM you use. Llama 3 is just the default because it's a fantastic balance of quality and footprint. Swap in Mistral, Phi-3, Qwen, or whatever else GPT4All supports — change one config line.

The TTS: Kokoro Microserver (N2NHU Labs build)

Kokoro is an open-source neural TTS model. We package it as a **standalone microservice** that runs on a separate port (9998) with its own GPU pinning. The realm sends it text over localhost HTTP, gets back WAV audio bytes, ships them to the browser as base64 chunks over the WebSocket.

What's good about the Kokoro microserver setup:

- GPU acceleration**: synthesis at roughly real-time speed or faster on a 4060
- Voice quality**: natural, clear, no robotic artifacts
- Queue serialization**: requests are handled one at a time so the GPU isn't thrashed
- LRU audio cache**: repeated phrases return instantly from cache, no re-synth
- News pre-encoding**: morning, noon, and evening headlines are synthesized in the background so when you say "computer news," playback starts immediately

The microserver has its own test suite (37 tests, independent of the realm). It can be replaced by any HTTP-speaking TTS service that takes text and returns audio.

A real cost comparison

If you used a cloud LLM at \$0.50 per million tokens and asked it 100 questions a day at 500 tokens per answer, you'd spend about \$7/year on inference. Not much. But RENT-A-HAL serves a whole household of users at higher throughput and **never sends a single character of your data to anyone**. Your conversations stay in your house. That's the actual point.

How the Browser Side Works

The UI is one HTML file — about 2,200 lines including all JavaScript and CSS. No framework. No build step. No npm. Open it in any editor. It runs.

Wake-word state machine

idle — hears "computer" → armed (accumulating text into buffer)

armed	—	hears "newline"	→	submit buffer to realm, back to idle
armed	—	silence 8s	→	timeout, back to idle
any	—	"stop stop"	→	halt TTS + YouTube, back to idle
any	—	"help help"	→	open help panel
any	—	"wake word off"	→	disable wake entirely

Privacy controls

You can say:

- **"Wake word short"** — auto-disable wake in 15 seconds
- **"Wake word medium"** — 30 seconds
- **"Wake word long"** — 1 minute
- **"Wake word infinite"** — run until you say off
- **"Wake word off"** — disable immediately

When the timer expires, the wake word turns off automatically. RENT-A-HAL stops listening. Privacy by default for ambient computing in a household.

The destroy-and-recreate pattern

When a new query produces card results (news / music / services), the relevant container is destroyed and recreated as a fresh DOM element. Not cleared — destroyed. This eliminates layout cache, lingering YouTube bindings, and any stale state. Every render is a clean slate.

The SILENCE button

A big red button always visible. One click and **everything stops**: audio, music video, thinking sound, and the realm gets told to halt any further TTS chunks. The safety valve for runaway audio.

Compatibility tested in the field

- iPhone 8+ on Verizon LTE — full functionality

- Win11 Chrome — full functionality
- macOS Safari — full functionality
- Android Chrome — full functionality

The speech recognition uses the browser's native **Web Speech API**. On Chrome and Edge, that means Google's cloud STT. On Safari, that means Apple's STT. We don't ship audio bytes anywhere — only text. The realm never decodes a single byte of speech.

What RENT-A-HAL Is NOT

In the spirit of honest engineering:

- It's not a chatbot pretending to be a person.** It's a structured assistant. It speaks plainly about what it found.
 - It's not a cloud service.** Your data stays on your hardware. The trade-off is you have to keep that hardware running.
 - It's not a substitute for a search engine on every query.** Music and services use the open web; news uses RSS; weather uses OpenWeatherMap; "ask" uses the local LLM only.
 - It's not bulletproof.** It's a Personal Edition. It's tested heavily, but it's not a battle-tested enterprise SaaS. You're an operator. You get to fix things if they break.
 - It's not infinitely scalable on one box.** A 4060 happily serves a household. For an office of 20 simultaneous active users, you'd want a bigger GPU and probably a load-balanced setup.
-

Setting It Up (Five Minutes)

- 1.Install **GPT4All** on your Windows 11 box. Pull the Llama 3 8B Instruct model. Enable the local API server (port 4891).
- 2.Unzip RENTAHAL_FULL.zip somewhere — for example c:\rentahal\.

3. Create a Python 3.12 virtualenv. Run `pip install -r requirements.txt`.
4. Start the Kokoro microserver: `python kokoro_service/service.py`
5. Start the realm: `python RENTAHAL_APP/app.py`
6. Open `http://localhost:9999` in any browser on your LAN.
7. Click **Enable wake word**. Grant microphone permission. Grant location (if you want service finder).
8. Say: "Computer. Tell me about Apollo 11. Newline."

That's it. Done.

To expose to the public internet, run an ngrok tunnel or Cloudflare Tunnel pointing at `localhost:9999`. That's how `rentahal.com` works.

Quick Reference: Voice Commands

Command	Effect
	Ask the LLM
<code>computer weather newline</code>	Weather at your location
	Weather elsewhere
<code>computer news newline</code>	Today's headlines
	Play a song via YouTube
<code>computer service finder <category> newline</code>	Find local businesses
	Halt everything
<code>repeat repeat</code>	Replay the last answer
	Show all voice commands
<code>wake word short/medium/long/infinite/off</code>	Privacy timer for wake word

Service Finder supports **75+ categories**: haircut, gas station, coffee, pharmacy, bank, ATM, restaurant, hotel, gym, dentist, doctor, lawyer, bakery, bar, mechanic, hospital, parking, bookstore, florist, pet shop, museum, cinema, and many more.

Quick Reference: HTTP / API Endpoints

Endpoint	Purpose
	The main UI
GET /api/config	Returns the runtime configuration
	One-shot LLM query (for simple integrations like Johnny's)
GET /api/weather?city=<name>	Weather for any city
	Current headlines as JSON
WS /ws	The WebSocket — everything happens here

Every WebSocket event is a JSON object with `type` and a payload. The same events the realm publishes internally are the events the browser sees, prefixed with `bus_`. Read the protocol once, write any client you want.

What Makes This Project Different

A few things that I'd want a skeptic to notice:

One file for the entire frontend. No build step, no bundler, no transpilation. Open it in Notepad. Edit it. Refresh the browser. Done.

Honest metrics broadcast to every user. When the system is busy, every connected browser sees the queue depth go from green to amber to red. Transparency is built in.

No audio ever crosses the realm-to-browser boundary. The realm receives text. The browser sends text. Audio only flows out (the TTS synthesis). This asymmetry makes the system simple and robust.

Every config knob is declared. `config.ini` has `@label`, `@type`, `@scope` tags on every value. A setup wizard can be generated from these tags.

The test suite is the proof. 700+ tests across 20 test layers, run three times in a row with zero flakes before any release. Each new feature ships with sabotage-verified regression tests so a future change that breaks the feature is caught immediately.

Designed for a real household. Multiple devices. Different browsers. Variable network. Real noise. Real distractions. Tested on an iPhone 8+ over LTE from a parked car, picking up a kid from grandma's house. It worked.

Credits

- Design:** Jim Ames, N2NHU Lab for Applied AI, Newburgh, NY 12550 USA
- Implementation:** Claude (Anthropic), in collaboration with Jim
- LLM:** Meta Llama 3 8B Instruct (via GPT4All)
- TTS:** Kokoro ONNX (microservice by N2NHU Labs)
- STT:** Web Speech API (browser-native)

RENT-A-HAL™ and MTOR™ are trademarks of Jim Ames and N2NHU Lab for Applied AI. © Copyright 2026, Jim Ames. All rights reserved.

This build is based on the open-source RENT-A-HAL MTOR Reference Implementation.

Designed and built in the United States of America. — Always bet America for Win, Place, and Show —

👉 Live long and prosper.

MTOR Federation: A Push-Driven, Self-Healing Mesh for Voice-First Local AI

A technical white paper Version 1.0 — June 2026

Jim Ames (N2NHU Labs, Newburgh NY) Engineering collaboration with Anthropic's Claude

Abstract

MTOR Federation is the inter-realm mesh layer at the heart of **RENT-A-HAL Personal Edition** (MTOR — Multi Tronic Operating Realm). It enables independently-owned, self-hosted AI realms — each running on consumer-grade hardware — to share LLM and TTS capacity transparently with peer realms, with no central control plane and no dependence on cloud APIs.

The federation is built around four design commitments that distinguish it from prior local-LLM clustering work: **push-driven coordination** (no polling, peers self-report state), **strict cooperation** (a one-hop limit prevents cascades), **honest self-reporting** (a degraded realm announces its own degradation rather than passing the buck), and **bounded silent dependency** (a wall-clock watchdog prevents a dead-local realm from federating forever without operator awareness). All four are enforced in the wire protocol, the matrix logic, and the provider chain — they are not optional behaviors operators can disable to game the network.

This paper describes the architecture, wire format, failure modes, self-healing mechanisms, operator model, and known limitations. It is intended for engineers deploying or extending RENT-A-HAL Personal Edition, and as a reference document for future federation implementations in this design lineage.

1. Introduction

1.1 The problem

A single self-hosted AI realm — voice in, voice out, running locally on a consumer GPU — works well for a single user with a single task. It does not work well in two common cases:

1.**Burst capacity.** When the same operator's realm is asked to serve concurrent users (a household, a small business, a classroom), local model inference becomes the bottleneck. Query latencies climb. Token generation queues. The UX degrades.

2.**Engine failures.** When the local LLM (e.g., GPT4All) or local TTS (e.g., Kokoro) crashes, hangs, or is killed for maintenance, the realm has no graceful fallback short of "the system is currently unavailable."

Cloud APIs solve both problems by routing every query to a datacenter. **MTOR Federation solves both problems by letting a small number of independently-owned, self-hosted realms cooperate.** A realm experiencing burst load can transparently route excess queries to a less-loaded peer. A realm with a crashed engine can rescue queries from a healthy peer. When the local engine recovers, routing returns to local automatically — no operator intervention required.

The result is a **distributed substrate for local AI** that preserves the privacy, cost, and ownership characteristics of self-hosting while providing the elastic capacity and graceful degradation typically associated with cloud services.

1.2 What this is not

MTOR Federation is not a distributed inference system. Individual queries are not split across realms. Tokens are not streamed in parallel from multiple GPUs. There is no model sharding, no tensor parallelism, no shared parameter cache. **Each query runs on exactly one realm's local engine** — federation simply routes the query to a healthy realm when the originating realm cannot serve it well.

It is also not a marketplace. There are no payments, no quotas, no metering, no reciprocity tracking. The federation operates on a **cooperative, voluntary basis** — operators opt their realms in by setting `desire = on` in `federation.ini`. The watchdog described in Section 5 prevents one-sided dependency by capping how long a degraded realm can mooch off peers before being forced into honest fallback.

1.3 Design philosophy

Three principles guide every design decision:

Operator control over defaults. Anything potentially intrusive ships disabled. Operators consciously opt in. Federation participation, supporters page, ngrok auto-tunnel, custom branding — all default-off or default-empty. The shipping product is infrastructure, not promotion.

Honest signal over silent rescue. When something is broken, the operator must be able to tell. A federation that rescues queries forever from a dead-local realm is hiding the truth from its owner. The watchdog ensures that after a bounded period of continuous federation, the realm falls back to a visibly-degraded floor (Echo for LLM, pyttsx3 for TTS) that the operator cannot miss.

Self-healing as the primary recovery mode. Operators should not need to restart their realm, clear caches, or run recovery commands when a transient failure passes. The federation provider's pre-flight probe, the recovery dance, and the watchdog clear-on-recovery all combine so that **the first successful local query after a failure transparently restores normal operation** — no manual intervention, no downtime, no operator-visible state machine to manage.

2. Architecture overview

2.1 Component model

A federated MTOR deployment has three roles. A single realm can play multiple roles simultaneously.

Member realm. The default role. A member realm runs the standard RENT-A-HAL stack (FastAPI server, WebSocket cockpit, local LLM/TTS chains) plus federation client code. It periodically reports its state to a Federator and accepts inbound federation calls on `/api/ask` and `/api/speak`. Every realm with `[Federation] desire = on` is a member.

Federator. A coordinator realm that maintains the matrix of known members, receives push-driven check-ins, and on each check-in returns peer recommendations to the calling realm. A Federator is itself a member — the Federator role is purely additive (configured via `[Federator] enabled = true`). The reference deployment runs `rentahal.com` as the Federator for the public mesh, but anyone can spin up a private Federator for an internal mesh.

Peer. Any member realm whose state is reachable through the Federator's recommendations. From a given calling realm's perspective, every other member is a potential peer; the Federator advises which specific peer to use for the next overflow or rescue call.

2.2 Reference deployment topology

The production reference deployment runs two realms:

- rentahal.com** — Gen 12 + RTX 4060, master Federator, primary public-facing realm
- realnewslistener.com** — Gen 8 + RTX 2070 Super, second member, news-focused workload

Both realms federate to each other through `rentahal.com`'s Federator role. Either realm can rescue the other. Both have proven sub-second peer recommendations and proven sub-10-second rescue end-to-end (federated answer + return + render).

The same pattern scales to N realms. There are no architectural assumptions baked in about a two-realm topology specifically.

2.3 Source code module layout

Module	Purpose
	FederationConfig, FederationState, FederationMatrix, FederationClient, FederatorServer
<code>realm/federation_provider.py</code>	FederationLLMProvider (HEAD — overflow) and FederationLLMFallbackProvider (TAIL — rescue)
	FederationTTSPProvider + FederationTTSFallbackProvider — symmetric architecture for TTS
<code>realm/degraded_tracker.py</code>	Wall-clock watchdog preventing indefinite federation
	Wiring — constructs trackers, attaches providers to chains, federation client lifecycle

The federation layer is approximately 1,400 lines of Python (excluding tests). Test coverage is approximately 130 federation-specific test cases plus integration coverage through the orchestrator and chain modules.

3. The matrix: peer discovery and ranking

The Federator maintains a `FederationMatrix` — a phonebook of known members with their last-reported performance metrics. Every member check-in produces an upsert; the matrix is the authoritative source for peer recommendations.

3.1 Member record shape

Each entry contains:

- `self_url` — the canonical URL the member identifies itself by (used for self-exclusion when computing peer advice)
- `llm_avg_s` — the member's recent rolling-window LLM average service time, in seconds
- `tts_avg_s` — the member's recent rolling-window TTS average service time, in seconds
- `tokens_out` — cumulative tokens generated since member start (diagnostic, not used for ranking)
- `last_seen` — monotonic timestamp of most recent check-in

3.2 The `DEGRADED_AVG_S` sentinel

A central design decision: when a member's local engine is dead but the member is still running (e.g., GPT4All crashed but the realm process is healthy), the member **self-reports** a sentinel value for the relevant chain's average service time:

```
DEGRADED_AVG_S = 1000.0          # realm/federation.py:68
```

This sentinel propagates through the matrix and through peer recommendations. A member with `llm_avg_s = 1000.0` will not be recommended as an LLM peer by the Federator, because the matrix ranks peers by `effective_avg` (lowest wins) and the sentinel sits at the top of any reasonable distribution.

Critically, **the sentinel is also applied retroactively to stale members** — any member whose `last_seen` is older than `missed_after_s` (default 60 seconds) is treated as having degraded averages, regardless of what value they last reported. A member that crashes mid-check-in is correctly excluded from peer recommendations within one minute, without requiring the matrix to model "alive" as a separate state.

3.3 Peer recommendation algorithm

On each member check-in, the Federator computes peer advice for that member:

```

best_llm_peer = argmin {
    m.effective_avg_for_llm(now, missed_after_s)
    for m in matrix.members
    if m.self_url != caller.self_url
        and m.effective_avg_for_llm(...) < DEGRADED_AVG_S
}

```

The result is one URL (or `None` if no live peer exists). The caller uses this URL as the destination for the next federated call, if it makes one. Repeating calls to a single best peer create natural concentration; the matrix's `peer_advice_bump` rotates equally-good peers to distribute load.

3.4 Persistence and recovery

The matrix persists to `federation_state.json` on each upsert. On Federator restart, the file is loaded. Members whose `last_seen` is older than `missed_after_s` are correctly excluded from peer advice through the sentinel mechanism — there is no separate "stale removal" sweep needed. The matrix is eventually consistent under restart.

4. The wire protocol

4.1 Push-driven check-in

The fundamental coordination primitive is the member-to-Federator check-in. Every `checkin_interval_s` (default 15 seconds), each member POSTs to the Federator:

Request: POST `/api/federate/checkin`

```

{
  "self_url": "https://rentahal.com",
  "llm_avg_s": 5.96,
  "tts_avg_s": 7.97,
  "tokens_out": 12450
}

```

Response (200 OK):

```

{
  "best_llm_peer": "https://realnewslistener.com",
  "best_tts_peer": "https://realnewslistener.com",
  "your_state": "HEALTHY"
}

```

There is no separate "register" or "discover" call. The first check-in from a new member implicitly registers it. The Federator returns the same payload regardless of registration history — the member doesn't need to track its own onboarding state.

Why push-driven, not poll. A polling Federator would need a member list, retry logic, timeout handling, and per-member state tracking. A push-driven Federator only needs to receive packets, update the matrix, and return advice. **Members are the source of truth about their own state.** The Federator is a coordinator, not a probe.

4.2 Federated query: LLM

When a member's LLM federation provider decides to route a query to a peer, it makes a standard HTTP call to the peer's public API:

Request: POST /api/ask

Content-Type: application/json

X-Federation-Hops: 1

X-Federation-Token: <optional shared secret>

```
{"text": "what's the weather like on mars", "max_tokens": 1500}
```

Response (200 OK):

```
{
  "answer": "Mars has a thin CO2 atmosphere with surface...",
  "provider": "GPT4All (Llama 3.2 3B Instruct)"
}
```

The peer answers from its own LLM chain (whichever provider its own chain selects). Critically, the X-Federation-Hops: 1 header tells the peer **not to re-federate**. If the peer's own LLM is unavailable, it returns Echo's "no LLM online" message rather than calling out to a third realm.

4.3 Federated query: TTS

Symmetric architecture:

Request: POST /api/speak

Content-Type: application/json

X-Federation-Hops: 1

```
{"text": "Mars has a thin CO2 atmosphere"}
```

Response (200 OK):

```
{  
  "audio_b64": "<base64-encoded WAV data>",  
  "provider": "Kokoro Service"  
}
```

The peer synthesizes locally and returns base64-encoded audio. Same hop-limit semantics: if the peer's own TTS chain is in trouble, it returns its pyttssx3 robot-voice floor rather than re-federating.

4.4 Cascade prevention

The single most important property of the wire protocol is **cascades are impossible by construction**. A federated call carries X-Federation-Hops: 1. The receiver reads it. If hops $\geq$ max_federation_hops (default 1), the receiver runs allow_federation=False through its provider chain. The federation providers on the receiver check allow_federation first in their should_federate() gates and return False.

This means:

- A federated call from A→B is served by B's local chain or falls to Echo/pyttssx3
- A→B→C is impossible — B will not call C
- A→B→A is impossible for the same reason
- A→B→A→B... loops are impossible
- A query can survive the death of at most one realm in the path

The cooperation is **opt-in but mandatory**. A misbehaving peer could in principle ignore the hop header and re-federate anyway, but doing so would be visible in logs and would damage the misbehaving peer's reputation in the federation. The protocol assumes good-faith participation; the matrix mechanics provide soft enforcement via peer ranking.

5. Failure modes and self-healing

Federation v1 handles three distinct failure modes, each with its own self-healing mechanism. Understanding the interaction between these mechanisms is the key to understanding the federation's reliability properties.

5.1 Failure mode 1: Local engine slow (overflow)

Symptom: Local LLM (or TTS) is responding, but rolling-window average service time has climbed above the configured threshold (`prefer_federation_for_llm_avg_s`, default 31 seconds).

Mechanism: The HEAD federation provider sits at position zero of the LLM chain.

Its `is_available()` method reads the current local average from `FederationState.get_stats()`.

When the average exceeds the threshold AND a healthy peer is available, `is_available` returns True, and the chain calls `generate()` on this provider before reaching local providers.

Self-healing: Before committing to the federated call, the HEAD provider runs a **pre-flight local probe** — it asks the local LLM chain's `probe_local()` method whether any local provider is currently responding. If the local LLM is actually fast right now (and the elevated rolling average was a stale signal from recent federated calls), the probe returns the local provider; the HEAD provider returns None; the chain falls through to local; **no federation call happens**. On probe success, the HEAD provider also fires the **recovery dance**:

- `metrics.reset_ai_window_to_local_recovery()` — clears the stale rolling average
- `bus.publish("status_update", channel="llm", state="recovered")` — flashes the cockpit badge back to the local provider name
- `federation_client.immediate_checkin()` — pushes the recovered state to the Federator out-of-band so peer advice updates within seconds instead of waiting for the next 15s interval

The combined effect: a transient slowdown can fire a single federated call, immediately self-correct, and never fire another. The rolling average resets; the operator sees the badge flash; the federation matrix updates. Total operator intervention: zero.

5.2 Failure mode 2: Local engine dead (rescue)

Symptom: Local LLM is not responding at all. `is_available()` on every local provider returns False. The chain walks past local providers entirely.

Mechanism: The TAIL federation provider (`FederationLLMFallbackProvider`) sits between the local providers and the Echo floor. It has no threshold check — being reached is itself the signal. When `is_available()` returns `True` (peer exists, federation healthy), the chain calls `generate()` and the query is rescued from a peer.

This is **distinct from overflow** in two ways: there is no avg-vs-threshold check, and the dead-local realm self-reports `DEGRADED_AVG_S` for its own LLM channel in the next check-in, so the Federator will not recommend this realm as a peer for OTHER realms during the outage.

Self-healing: Same pre-flight probe pattern. The TAIL provider also calls `chain.probe_local()` before federating. On success, it fires the recovery dance — and crucially, because the chain has already walked past local providers, the chain re-walk doesn't happen automatically; **this specific query** still falls to Echo. But the recovery side effects (cleared rolling avg, Federator check-in, badge flash) ensure that the **next** query routes correctly to local.

5.3 Failure mode 3: Bounded silent dependency (the watchdog)

Symptom: Local engine has been dead for an extended period. Federation has been rescuing every query. From the user's perspective, the realm is "working" — they get answers. From the operator's perspective, the realm is silently mooching off peers, and no signal has been raised that local needs attention.

Mechanism: The `DegradedTracker` (`realm/degraded_tracker.py`) is a wall-clock watchdog with three states: Initial, Federating, Degraded. The first `mark_federated()` call after boot or after a recovery starts a monotonic timer. After `local_degraded_after_s` (default 300 seconds = 5 minutes) of continuous federation, the tracker's `is_degraded()` flips to `True`. Both HEAD and FALLBACK federation providers consult `is_degraded()` in their `should_federate()` gates. When `True`, they refuse to federate.

The chain then falls through to **Echo** (LLM) or **pyttsx3** (TTS) — the operator-visible degraded floor.

On transition into degraded:

- An ERROR-level log line fires once: LOCAL LLM DEGRADED – federated continuously for 300s. Falling through to Echo. Operator: check local engine.
- A `bus_status_update` flashes the cockpit badge to `LocalSystemLLMDegraded.xpy` (and `LocalSystemTTSDegraded.xpy` for TTS)
- Subsequent queries surface the floor provider's output until local recovers

Self-healing: The pre-flight probe in both federation providers still runs on every query (it executes inside the `generate()` body before the federated call would happen, and the watchdog gate is checked separately in `should_federate()`). On any successful local probe — i.e., the first query after the local engine returns — the watchdog's `mark_local_recovered()` clears the degraded flag, fires the recovery dance, and the next query routes to local at full performance.

Production validation: This mechanism was smoke-tested in production by deliberately killing both GPT4All and Kokoro on `rentahal.com`, observing 5 minutes of successful federation to `realnewslistener.com`, observing the transition to degraded floor at exactly the 5-minute mark, restarting the local engines, and observing single-query recovery to full local performance on the very next query. All transitions logged correctly. No operator intervention beyond restarting the local engines.

5.4 Design rationale: wall-clock, not call-count

An early design proposal used a call-count cap ("after N federated calls, declare degraded"). This was rejected for two reasons:

1. **Traffic-dependent false positives.** A busy realm doing 60 queries/minute hits a 15-call cap in 15 seconds — far below any reasonable cooldown window. The realm would oscillate between rescue and degraded on transient load spikes.
2. **Traffic-dependent missed signals.** A quiet realm doing 1 query / 10 minutes takes 2.5 hours to hit the same 15-call cap. The operator would not know local was broken for an entire afternoon.

Wall-clock time is the correct unit because **the question being asked is "how long has local been unhealthy?", not "how many queries have we forwarded."** A 5-minute wall-clock cap is the same 5 minutes regardless of the calling pattern. The cap correctly fires when local genuinely needs operator attention and correctly does not fire when local is recovering through transient slowdowns.

5.5 Independence of LLM and TTS chains

LLM and TTS chains fail independently. GPT4All can crash while Kokoro is healthy; Kokoro can crash while GPT4All is fine. Two separate `DegradedTracker` instances are constructed at startup,

one per chain, with independent thresholds ([LLMChain] local_degraded_after_s and [TTSChain] local_degraded_after_s). They are wired to the four federation providers via the attach() method's degraded_tracker parameter. The cockpit badge reflects each chain's state separately. The dead-local sentinel reports each chain separately.

6. Operator model

The federation is **transparent by design**. Every state the realm is in is visible to the operator without requiring a log dive, a separate dashboard, or an external monitoring tool.

6.1 Cockpit badge as the source of truth

The cockpit displays the current effective LLM and TTS providers in real time. Per-query, the badge updates from the bus_query_result event with the actual provider that served the query:

LLM: GPT4All (Llama 3.2 3B Instruct)	← normal local operation
LLM: Federation (peer realms)	← overflow active
LLM: Federation Fallback (peer realms)	← rescue active
LLM: Echo	← degraded floor
LLM: LocalSystemLLMDegraded.xpy	← watchdog tripped

The TTS badge updates symmetrically. No status is hidden; no state is implicit; no operator interpretation is required. **The badge says what the chain just did.**

6.2 Federation strip

A persistent strip below the badges shows current federation state:

```
Fed: ON • Master: rentahal.com • State: HEALTHY • (LLM→realnewslistener.com,  
TTS→realnewslistener.com)
```

The strip reflects the most recent check-in response. The hints (LLM→..., TTS→...) are advisory — they show what the Federator currently recommends, not what the realm is actually doing right now. The badge above is the per-query truth; the strip is the federation context.

6.3 Configuration

All federation knobs live in two INI files with self-documenting schema:

config.ini:

- [LLMChain] `local_degraded_after_s` — watchdog threshold for LLM chain
- [TTSChain] `local_degraded_after_s` — watchdog threshold for TTS chain

federation.ini:

- [Federation] `desire` — opt in or out of the federation (on/off)
- [Federation] `master_url` — Federator endpoint URL
- [Federation] `self_url` — this realm's canonical public URL
- [Federation] `prefer_federation_for_llm_avg_s` — overflow threshold for LLM
- [Federation] `prefer_federation_for_tts_avg_s` — overflow threshold for TTS
- [Federation] `request_timeout_s` — timeout per federated call
- [Federation] `checkin_interval_s` — push-driven check-in period
- [Federation] `missed_after_s` — staleness threshold for matrix entries
- [Federation] `max_federation_hops` — cascade limit (default 1)
- [Federator] `enabled` — promote this realm to Federator role

Every knob has a sensible default. A first-time operator opting into the federation needs to set only `master_url` and `self_url`. Everything else can be tuned later, in production, without code changes.

6.4 Privacy properties

The federation transmits **query text and response text** between participating realms. It does not transmit:

- User account information (realms have no concept of user identity beyond per-WebSocket session)
- Conversation history (federated calls are stateless — each query is independent)
- Cookies, headers from the original browser request, or any browser-side context
- Recordings of the original audio (STT happens browser-side; only text crosses the wire)

Operators participating in the federation should understand that **queries asked of their realm may be answered by a peer realm's local LLM**, and **queries asked of their peers may be answered by their own realm's local LLM**. This is the entire point of the federation. Operators with privacy-sensitive workloads who do not want their realm's hardware to serve other operators' queries should set `[Federation] desire = off` and run standalone.

There is no encryption layer at the federation level beyond HTTPS termination at each realm's edge (typically ngrok). An optional shared secret (`[Federation] token`) provides modest

authentication of inbound federated calls but is not a substitute for transport-level confidentiality.

7. Comparison to existing approaches

7.1 Versus cloud LLM APIs

Cloud APIs (OpenAI, Anthropic, Google) solve the burst-capacity and graceful-degradation problems by running every query on rented datacenter GPUs. They impose per-token billing, conversation logging by default, rate limits, and a hard dependency on the provider's uptime.

MTOR Federation solves the same problems differently. There is no billing, no logging at the federation level, no rate limits beyond what the peer realms can serve, and no single point of failure. The trade is: cloud APIs have higher peak throughput; MTOR Federation has zero marginal cost and stronger privacy properties.

7.2 Versus local-LLM clustering frameworks

Distributed inference systems (vLLM tensor-parallel, Petals, distributed-llama) split the work of a single query across multiple machines. This is a fundamentally different design point. They optimize for serving a larger model than any single machine can hold; they require tight coupling, low-latency interconnects, and careful synchronization.

MTOR Federation does not split queries. Each query runs on exactly one realm's local engine. The cooperation is at the **query** level, not the **token** level. The trade: distributed inference can run bigger models; MTOR Federation can run on consumer hardware over residential internet with no coordination overhead per query.

7.3 Versus other federated-LLM proposals

Several proposals for federated LLM hosting exist in the open-source ecosystem (LocalAI clusters, Ollama hosts behind load balancers, Horde-style distributed pools). MTOR Federation's distinguishing properties are:

- Push-driven coordination.** Members report state; the Federator does not poll. This makes the Federator stateless about member health beyond what members tell it.

- Strict hop limit.** Cascades are impossible by construction, not by convention.
- Self-reporting of degradation.** A dead-local realm announces its own degradation rather than relying on the coordinator to infer it.
- Bounded silent dependency.** The watchdog prevents a realm from being silently dependent on peers indefinitely.
- Operator-first transparency.** Every state is visible in the cockpit without external tooling.

None of these properties is individually novel. The combination is, to our knowledge, novel in the local-LLM space.

8. Known limitations

Honest limitations of the current design:

No partial-answer streaming across federation. A federated query is a single round-trip: peer generates the full answer, returns it, requester returns to user. This adds latency to long answers. Streaming federation is a feasible extension; it was deprioritized for v1 because the latency cost is small (a few hundred milliseconds of perceived "first-token" delay) and the protocol simplicity is worth the trade.

No model-aware routing. The Federator does not know which models each member realm has loaded. A realm with Llama 3 8B and a realm with Llama 3.2 3B are treated as equivalent peers. In practice this is fine because the federation is overflow rescue, not model selection — but operators who care about which model answers a federated query currently have no way to express that preference.

No quotas or rate limiting. Operators implicitly trust each other not to flood the federation. A misbehaving member realm could in principle issue thousands of federated calls per second to a single peer. The watchdog protects the misbehaving realm from itself but does not protect the peer from being flooded. Per-peer rate limits at the receiver are an obvious extension.

Single Federator topology. The reference deployment runs one Federator (rentahal.com). A Federator outage stops new member onboarding and stops peer-advice updates. Member-

to-peer federation calls continue working with stale advice. A multi-Federator topology (consensus across multiple coordinators) is a straightforward extension; it was deprioritized for v1 because the operational complexity outweighs the benefit at the current federation size.

No model warm-up coordination. When a member restarts its local LLM (model loads cold from disk), the first few queries are slow. The federation provider currently has no signal to distinguish "model loading" from "model crashed" — both look like dead-local. The watchdog correctly forgives this on recovery, but during the warm-up the peer's LLM unnecessarily serves queries that local could serve once warmed.

No transport-level encryption requirement. The protocol assumes HTTPS at each edge but does not enforce it. A misconfigured realm could expose `/api/ask` and `/api/speak` over plain HTTP. The reference deployment uses ngrok reserved domains with TLS termination; operators running on raw IPs should configure HTTPS themselves.

9. Future work

The federation is feature-complete for v1.0. Plausible v1.x extensions:

- Token streaming across federation** for sub-second perceived latency on long answers
- Per-peer rate limits** at the receiver to protect against misbehaving members
- Federation-level shared secrets enforcement** by default (currently optional)
- Multi-Federator consensus** for high-availability coordinator topologies
- Model-aware routing hints** so operators can express preferences
- Warm-up detection** so the watchdog and overflow gates distinguish cold-start from crash
- Per-realm capability advertisement** (image generation, web search, etc.) for non-LLM/TTS workloads

None of these are blockers for current operators. All would extend the architecture additively without changing the core invariants (push-driven, strict hop limit, honest self-reporting, bounded silent dependency).

10. Conclusion

MTOR Federation provides a working substrate for cooperative, self-hosted AI realms. It has been validated in production across two independently-owned realms (`rentahal.com` and `realnewslistener.com`) running on consumer-grade hardware with residential symmetric fiber. End-to-end query latency for federated rescue calls has been measured at approximately 3.5 seconds for a 555-token Apollo program summary, including peer model inference and return.

The design commitments — push-driven, strict hop limit, honest self-reporting, bounded silent dependency — produce a federation that is **operator-friendly by default**. Federations of one realm work. Federations of two realms work. Federations of N realms should work for any N within the bandwidth and matrix scaling characteristics of the chosen Federator hardware.

This document is the architectural reference for the v1.0 federation. Future revisions will extend it as the protocol evolves.

11. References

Source code (RENT-A-HAL Personal Edition, v1.0)

- `realm/federation.py` — `FederationConfig`, `FederationState`, `FederationMatrix`, `FederationClient`, `FederatorServer`
- `realm/federation_provider.py` — `FederationLLMProvider` (HEAD), `FederationLLMFallbackProvider` (TAIL)
- `realm/federation_tts_provider.py` — `FederationTTSPProvider`, `FederationTTSFallbackProvider`
- `realm/degraded_tracker.py` — Wall-clock degraded watchdog
- `app.py` — Construction and wiring of all federation components

Related documents

- **RENT-A-HAL Operator's Manual** — operational deployment guide

- RENT-A-HAL Design and Capabilities Paper** — broader product context
- MTOR: Welcome to the Realm** (Jim Ames, Amazon B0F6BDXZYH) — narrative background on the MTOR project

Constants of interest (canonical values)

Constant	Value	Where defined
	1000.0	
checkin_interval_s default	15.0	realm/federation.py:78
default	60.0	
max_federation_hops default	1	realm/federation.py:81
lt default	31.0	
lt prefer_federation_for_tts_avg_s default	31.0	federation.ini
default (LLM)	300	
local_degraded_after_s default (TTS)	300	config.ini [TTSChain]

Naming

MTOR = **M**ulti **T**ronic **O**perating **R**ealm — a deliberate reference to Dr. Daystrom's M-5 Multitronic Unit from Star Trek: The Original Series, "The Ultimate Computer" (1968). The system aims to fulfill the M-5 design vision (a self-contained intelligent realm capable of autonomous operation) while explicitly avoiding the M-5 design failure (autonomous operation without human oversight). The federation is the substrate that lets multiple MTOR realms cooperate as a mesh while each remains under its own operator's control. **The realm does what M-5 was supposed to do, with humans in the loop.**

Acknowledgments

Built by Jim Ames at N2NHU Labs (Newburgh NY). Engineering collaboration with Anthropic's Claude over approximately seven months of sustained pair-engineering. Production validation conducted at the N2NHU lab on the rentahal.com and realnewslistener.com realms.

The shower-thought that produced the degraded watchdog (Section 5.3) was had on the morning of June 22, 2026, before the v1.0 compile freeze. It is included in this document as evidence that careful pre-release thinking finds bugs that no test suite will.

👉 Live long and prosper.

This document is released under the same license as the RENT-A-HAL Personal Edition codebase. It is intended as a technical reference for operators, deployers, and engineers extending the federation. Comments, corrections, and proposed extensions are welcome via the project's GitHub issues.

Building a Private RENT-A-HAL Cluster for Your Business

A Deployment Guide for Small Business and Professional Offices

Version 1.0 | June 2026 | N2NHU Labs

Executive summary

This paper describes how to deploy a private RENT-A-HAL federation as the voice AI substrate for a small business or professional office. It is written for decision-makers and IT operators who have evaluated the alternatives (cloud subscriptions, single-instance self-hosting) and are considering a small private cluster.

The intended audience runs an organization with **2 to 30 staff members** who would benefit from on-premises voice AI for tasks such as document drafting, client communications, scheduling support, knowledge base queries, and internal Q&A. The paper addresses three deployment scales — the **small office** (2 realms), the **multi-location professional firm** (4-6 realms), and the **regional small enterprise** (8+ realms) — with concrete hardware specifications, capacity math operators can validate, and honest scoping of what the architecture does and does not provide at each scale.

A working RENT-A-HAL cluster for a typical professional office can be deployed in one afternoon by a competent IT operator using off-the-shelf consumer hardware, with a total hardware cost in the \$2,000–\$8,000 range and a total ongoing cost approaching the marginal cost of electricity. The federation provides redundancy, burst capacity sharing, self-healing recovery from engine failures, and an audit-friendly local-first architecture suitable for regulated workloads.

The paper does **not** describe department-scale or enterprise deployments (50+ users, multi-region operations, formal identity management, audit trail compliance). Section 12 honestly scopes where the current architecture would require additional work for those use cases.

1. The cluster pattern

A RENT-A-HAL cluster is a **small number of cooperating realms**, each running on its own hardware, sharing capacity through the MTOR Federation. Every realm in the cluster has the same software, the same configuration model, and the same federation protocol. There is no leader-follower replication, no shared state outside the federation matrix, and no central component whose failure takes down the cluster.

A cluster is distinguished from a single realm by three properties:

Capacity pooling. When one realm is busy serving local users, other realms handle overflow transparently. The user experience does not change. Latency typically stays within normal ranges because the federation routes to a less-loaded peer.

Redundancy. When one realm's local engine fails (model crash, GPU thermal throttle, power loss), other realms rescue the affected queries automatically. The watchdog ensures the failed realm's operator is notified within five minutes if the failure persists.

Bounded silent dependency. No realm can become silently dependent on its peers. The watchdog forces honest signaling: a realm whose local engine has been dead for five minutes stops federating and falls through to its visibly-degraded floor. This protects peer capacity from being unilaterally consumed and protects the operator from running blind.

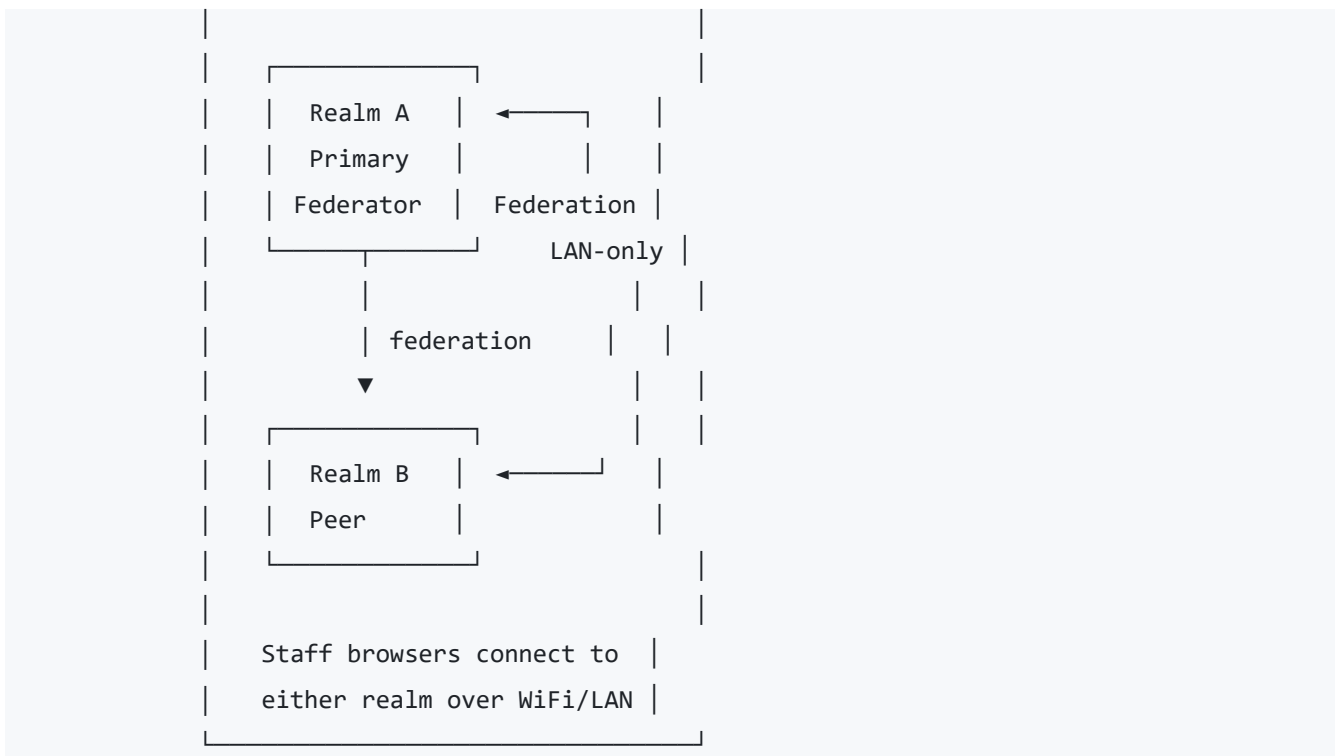
These three properties hold for clusters of any size from two realms upward. The federation protocol does not change with cluster scale; what changes is the typical operating pattern and the capacity envelope.

2. Reference cluster topologies

Three topologies cover the range this paper addresses. Each topology specifies the realm count, the typical concurrent-user envelope, and the federation roles. All three use the same software; the differences are in hardware, networking, and operational practice.

2.1 Small office (2 realms)



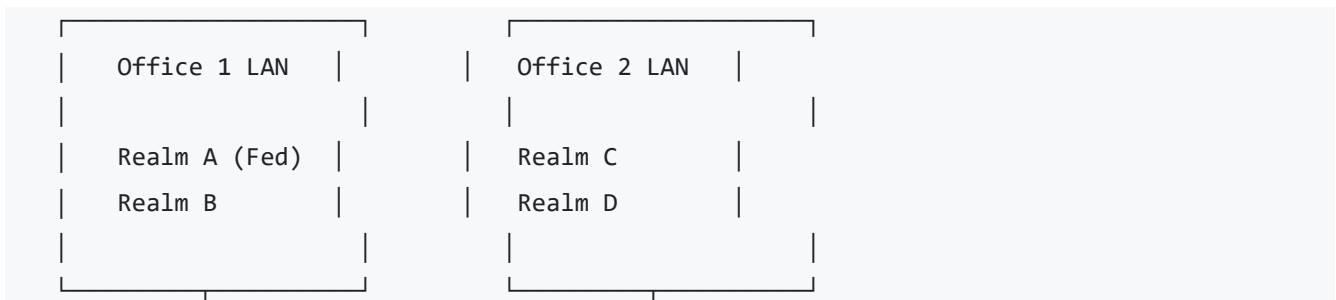


Realm count: 2 **Concurrent active users:** 4-15 **Hardware investment:** \$2,000-\$3,500 **Best for:** Small professional office, single location, primary cluster pattern for first-time deployers.

Realm A acts as both a member and the cluster's Federator (running the public Federator endpoint). Realm B is a pure member. Federation is LAN-only — neither realm needs an ngrok tunnel because the office uses them on the local network. This is the simplest possible cluster and the strongest privacy position.

If Realm A fails, Realm B continues to serve its own connected users normally, but no new federation advice is generated (the Federator is offline). When Realm A returns, the cluster heals automatically — Realm B's next check-in succeeds and federation resumes. For maximum availability, see Section 2.4 on dual-Federator topologies.

2.2 Multi-location professional firm (4-6 realms)





Realm count: 4-6 **Concurrent active users:** 15-50 **Hardware investment:** \$4,000-\$8,000 **Best for:** Professional firm with 2-3 office locations, regional small business, multi-branch operations.

Each office has its own realms. The federation matrix spans both offices via either site-to-site VPN (preferred for privacy) or reserved ngrok tunnels. A realm in Office 1 can rescue a query in Office 2 and vice versa. Each office continues operating independently if the inter-office link fails — federation falls back to local-only at each site.

This is the natural growth path from the small-office topology. Operators who outgrow the small office add realms in the same office (vertical scale) until LAN topology becomes complex, then add a second office (horizontal scale).

2.3 Regional small enterprise (8+ realms)

Realm count: 8-20 **Concurrent active users:** 50-200 **Hardware investment:** \$10,000-\$30,000 **Best for:** Larger regional business, small enterprise with 3+ locations, organizations approaching the upper bound of what this architecture currently supports.

The architecture continues to work at this scale, but operational practice becomes more demanding. Operators at this scale should plan for: dedicated IT staff (one part-time administrator for every 4-6 realms), automated monitoring (operators should not rely on individual cockpit checks), formal change management for software updates, and **dual-Federator topology** (Section 2.4).

Section 12 honestly addresses where this scale starts pushing the architecture's current limits.

2.4 Dual-Federator topology (recommended for 4+ realms)



For clusters of 4+ realms, two realms run the Federator role. Member realms check in with both. If Federator 1 is unreachable, members continue with the most recent peer advice from Federator 2. The federation continues to operate degraded but functional during a Federator failure.

This is the recommended topology for any production cluster of 4+ realms. The dual-Federator pattern is currently a configuration choice (two realms with `[Federator] enabled = true`); future versions may add a formal consensus protocol between Federators.

3. Hardware specifications

Hardware costs reflect realistic mid-2026 consumer market pricing in the US. Prices are — treat these as order-of-magnitude guidance.

3.1 Realm hardware tiers

Tier	Use Case	CPU	RAM	GPU	Storage	Approx. Cost
Starter	2-4 concurrent users	6-core x64	16 GB DDR4	NVIDIA RTX 4060 8GB	500 GB NVMe	\$900-\$1,200
Standard	5-10 concurrent users	8-core x64	32 GB DDR4	NVIDIA RTX 4070 12GB	1 TB NVMe	\$1,500-\$2,000
Heavy	10-20 concurrent users	12-core x64	64 GB DDR5	NVIDIA RTX 4080 16GB or RTX 5080	2 TB NVMe	\$2,500-\$3,500
Enterpris	20+ concurrent	16-	128 GB	NVIDIA RTX 4090 /	4 TB	\$5,000-

Tier	Use Case	CPU	RAM	GPU	Storage	Approx. Cost
e	users / larger models	core x64	DDR5	5090 24GB or workstation GPU	NVMe	\$8,000

A cluster typically uses **the same tier** across all realms to keep capacity predictable. Mixed tiers work — the federation does not require uniform hardware — but capacity planning becomes more complex.

3.2 Recommended starter cluster (2-realm small office)

For a typical professional office:

- **2 × Standard-tier realms** at approximately \$1,800 each = \$3,600
- **1 × small UPS** (uninterruptible power supply) for each realm, approximately \$150 each = \$300
- **Existing office network** (gigabit switch, WiFi) — no additional cost in most cases
- **Existing office router/firewall** — no additional cost

Total: approximately \$3,900 for a 2-realm starter cluster.

This is the realistic entry point. Operators reusing older office hardware can sometimes start at half that cost; operators requiring more headroom can comfortably spend twice that.

3.3 What the hardware actually buys

A Standard-tier realm running Llama 3.2 3B Instruct on an RTX 4070 typically delivers:

- **65-90 tokens/second** sustained single-stream generation
- **Voice query response time:** 5-10 seconds (model inference + TTS synthesis)
- **Single-threaded throughput:** 6-12 voice queries per minute
- **With TTS overlap and federation:** 15-25 queries per minute
- **Cold-start to wizard-ready:** under 60 seconds from power-on

A Heavy-tier realm running Llama 3.1 8B Instruct on an RTX 4080 delivers roughly:

- 40-55 tokens/second** sustained (larger model, more parameters per token)
- Voice query response time:** 8-15 seconds
- Quality improvement** over 3B-class models is noticeable on complex reasoning tasks

The trade-off between model size and throughput is the central capacity-planning decision. Section 7 develops this in detail.

4. Network architecture

The federation is **transport-agnostic** at the protocol level — it operates over any IP network. Three deployment patterns cover the realistic options.

4.1 LAN-only federation (recommended for single-site)

All realms reside on the same local network. Federation traffic never leaves the office. No ngrok tunnel is required. Browsers connect to realm cockpits via LAN IP addresses or DNS names resolved internally.

Setup: Set `[Federation] self_url` to the realm's LAN address (e.g., `http://192.168.1.50:9999`) for each realm. Configure both realms with the same `master_url` pointing at the Federator-role realm.

Privacy posture: Strongest. No federation traffic crosses any internet boundary. Voice queries and responses remain entirely within the office network.

Limitations: Remote staff (working from home, traveling) cannot reach the cluster without a VPN. The cluster cannot rescue another cluster's queries (no inter-organization federation).

4.2 LAN federation with remote staff access

The federation operates LAN-only. A separate ngrok tunnel (or reverse proxy) exposes one or more realm cockpits to remote staff. Remote users authenticate to the tunnel; federation between realms stays internal.

Setup: Same as 4.1 for federation. Additionally, run `ngrok http --url=officename.ngrok.app 9999` (or equivalent reverse proxy) on the realm that should be remotely accessible.

Privacy posture: Strong. Federation stays local. Only the remote-facing cockpit endpoints are exposed externally.

Recommended for: Most professional offices with occasional remote workers.

4.3 Multi-site federation

Two or more office LANs are connected via site-to-site VPN, and the federation matrix spans all sites. Each office's realms can rescue queries from other offices' realms.

Setup: Configure each realm's `self_url` to its VPN-routable address. All realms point at the same Federator-role realm. The site-to-site VPN handles encryption and routing.

Privacy posture: Strong, assuming the VPN is properly configured. Federation traffic is encrypted in transit.

Limitations: Inter-site bandwidth and latency now matter — federation rescue calls add the inter-site round-trip to the user's perceived latency. Plan for at least 25 Mbps symmetric between sites and round-trip latencies under 50 ms.

5. Deployment procedure

The following procedure stands up a 2-realm small-office cluster from scratch. It is intended for an IT operator who has read the Quick Start Guide. Total time: approximately 90 minutes for an experienced operator, half a day for first-time deployers.

Step 1 — Procure and prepare hardware

Order or assemble two Standard-tier machines. Install Windows 11 on each. Apply current updates. Verify each has an active wired Ethernet connection (avoid WiFi for realm hardware — federation traffic deserves wire-speed latency).

Assign each realm a static IP address (e.g., 192.168.1.50 for Realm A, 192.168.1.51 for Realm B). The realms must have stable, predictable addresses; DHCP-assigned addresses are not appropriate for federation members.

Step 2 — Install dependencies on both realms

On each realm in turn:

1. Install NVIDIA drivers; verify with `nvidia-smi`
2. Install GPT4All from gpt4all.io
3. Download a model in GPT4All (recommended: Llama 3.2 3B Instruct for Standard tier; Llama 3.1 8B Instruct for Heavy/Enterprise tier)
4. Enable the GPT4All Local Server (Settings → Application → Enable Local Server)
5. Install the **KOKORO RENT-A-HAL VOICE Microservice**
6. Install **RENT-A-HAL SOHO Multi-User '26**

Step 3 — Configure Realm A (Federator + member)

Launch the wizard on Realm A. In the wizard form:

- **Realm name:** Choose a descriptive name (e.g., "Office-Realm-1")
- **Tagline:** Your office or department name
- **Federation: participate** — On
- **Federator URL:** `http://192.168.1.50:9999/api/federate`
- **Your public realm URL:** `http://192.168.1.50:9999`
- **All other fields:** Defaults

Edit `federation.ini` on Realm A to enable the Federator role:

```
[Federator]
enabled = true
```

Save and click GO in the wizard. Verify the cockpit comes up at `http://192.168.1.50:9999`.

Step 4 — Configure Realm B (member only)

Launch the wizard on Realm B. In the wizard form:

- **Realm name:** "Office-Realm-2"

- Federation: participate** — On

- Federator URL:** `http://192.168.1.50:9999/api/federate` (Realm A's Federator)

- Your public realm URL:** `http://192.168.1.51:9999`

- All other fields:** Defaults

Click GO. Verify the cockpit at `http://192.168.1.51:9999`.

Step 5 — Verify federation

After approximately 15 seconds, the cockpit of each realm should show:

```
Fed: ON • Master: 192.168.1.50 • State: HEALTHY • (LLM→<peer>, TTS→<peer>)
```

If both realms show HEALTHY and recommend each other as peers, the federation is operational.

Step 6 — Validate end-to-end

From any browser on the office network, visit `http://192.168.1.50:9999` and ask a query. Verify a spoken answer is produced.

Repeat from `http://192.168.1.51:9999`. Both realms should serve queries independently.

For federation rescue verification, deliberately stop GPT4All on Realm A. Ask a query at Realm A's cockpit. After the pre-flight probe times out, the query should be rescued by Realm B and answered. Restart GPT4All on Realm A; the next query should return to local serving.

Step 7 — Document the deployment

Record the following in your IT documentation:

- Realm IP addresses
 - Realm hardware specifications
 - Model selection per realm
 - Federation configuration values
 - Backup/restore procedure for `config.ini`, `federation.ini`, and any operator notes
-

6. Capacity planning

This section provides the math operators can use to size their cluster against their actual workload. The numbers are illustrative; substitute your own measurements for production planning.

6.1 Per-realm capacity baseline

A Standard-tier realm (RTX 4070, Llama 3.2 3B Instruct) sustains the following under typical voice workloads:

Metric	Value	Notes
Tokens/second (generation)	65-90	Single-stream, post-prompt
Average response length	200-500 tokens	Varies by query complexity
Average inference time	4-8 seconds	Per query
Average TTS synthesis time	2-4 seconds	Kokoro at 1.0 speed
Total per-query latency	6-12 seconds	User speaks → user hears response
Sustained queries per minute (sequential)	5-10	Single user, back-to-back
Sustained queries per minute (with overlap)	12-20	TTS of query N while inferring query N+1

6.2 User-to-capacity ratio

In typical professional office workloads, a single active user does not generate sustained back-to-back queries. The empirical ratio is approximately:

- A **"concurrent active user"** generates roughly **1 query every 60-180 seconds** while actively using the realm
- A **"concurrent passive user"** (cockpit open, querying occasionally) generates roughly **1 query every 5-10 minutes**

This produces the following sizing rules of thumb:

Use pattern	Queries/min per user	Users per Standard realm
Heavy active (research, drafting)	0.5-1.0	8-15
Typical office (Q&A, lookups)	0.2-0.5	15-30
Light/passive	0.1-0.2	30-60

A typical professional office with 12 staff would size for ~15-30 typical-office concurrent users, suggesting **a single Standard realm could handle the load** under steady-state conditions. The reason to deploy 2 realms anyway is **redundancy and burst capacity**, not steady-state throughput.

6.3 The redundancy multiplier

A 2-realm cluster does not provide 2x capacity for burst loads — it provides 2x capacity **plus** the ability for either realm to absorb the other's load during a failure. The honest framing is:

- Steady-state capacity:** 2x single realm (each realm serves its own users)
- Burst capacity:** Up to 2x single realm (federation routes overflow)
- Failure tolerance:** 1 realm can be down indefinitely while the other continues to serve all users (with watchdog signaling on the dead realm)
- Operational confidence:** Major (one realm can be taken offline for maintenance without service interruption)

6.4 Capacity-planning worksheet

For your own organization:

- 1.**N_users** = peak concurrent active staff using the realm
- 2.**Q_user** = queries per minute per active user (estimate 0.3 for typical office)
- 3.**N_queries_per_min** = $N_users \times Q_user$

4.**R_capacity** = realm capacity from Section 6.1 (typically 12-20 queries/min Standard tier)

5.**N_realms_min** = ceiling($N_{\text{queries_per_min}} / R_{\text{capacity}}$) — the minimum cluster size for steady-state load

6.**N_realms_recommended** = $N_{\text{realms_min}} + 1$ — add one realm for redundancy

A 10-person law firm with 5 concurrent users at 0.4 queries/minute = 2 queries/min. Single Standard realm handles this comfortably. **Recommended: 2 realms** for redundancy.

A 25-person consulting firm with 15 concurrent users at 0.4 queries/minute = 6 queries/min. **Recommended: 2 realms** (still under capacity, but add one for redundancy).

A 60-person organization with 35 concurrent users at 0.3 queries/minute = 10.5 queries/min. **Recommended: 3 realms** (one realm could nominally handle the load, but federation overhead and headroom argue for 3).

A 150-person organization with 80 concurrent users at 0.3 queries/minute = 24 queries/min. **Recommended: 4-6 realms**. This is the upper bound of what this paper addresses confidently.

7. Concrete use case profiles

The following profiles describe realistic deployments. Each is plausible — operators with similar profiles have already deployed comparable architectures.

7.1 Small law firm (4-12 attorneys)

Hardware: 2 Standard-tier realms **Use cases:** Document drafting, deposition summary, legal research Q&A, client communication drafts, internal knowledge base **Why local matters:** Client confidentiality. Communications and documents cannot legally be transmitted to OpenAI, Anthropic, or Google for non-trivial workflows in many jurisdictions. The realm runs on the firm's own hardware, behind the firm's own firewall, and no client data leaves the office. **Federation benefit:** Redundancy. Lawyers cannot afford "the AI is down today" during deposition prep. **Realistic monthly cost:** Electricity only, approximately \$30-\$60.

7.2 Dental or medical office (5-15 staff)

Hardware: 2 Standard-tier realms **Use cases:** Patient intake question Q&A, appointment scheduling assistance, billing query support, insurance verification scripts, internal protocol lookups **Why local matters:** HIPAA. Protected Health Information cannot be transmitted to cloud AI services without a Business Associate Agreement, which most consumer-grade cloud AIs do not offer. **Federation benefit:** Redundancy and capacity sharing during busy clinic hours. **Realistic monthly cost:** \$30-\$60.

7.3 Accounting firm (4-20 staff)

Hardware: 2-3 Standard-tier realms **Use cases:** Tax law Q&A, internal procedure lookup, client communication drafts, document summary, training Q&A for junior staff **Why local matters:** Client financial data confidentiality. Many accounting workflows fall under SOX, GLBA, or state-level requirements that resist cloud AI by default. **Federation benefit:** Tax season burst capacity. The firm scales from 5 to 25 active users in March-April; federation pools capacity across realms during the spike. **Realistic monthly cost:** \$50-\$100.

7.4 Mental health practice (3-10 clinicians)

Hardware: 2 Standard-tier realms (or one Standard + one Starter) **Use cases:** Session note assistance, treatment plan drafts, billing code lookup, internal training Q&A **Why local matters:** Therapy notes are among the most sensitive data categories. HIPAA + state mental health confidentiality laws preclude any cloud transmission. **This is the strongest case for local-first voice AI.** **Federation benefit:** Modest redundancy. Clinics typically have one or two clinicians using the realm at a time. **Realistic monthly cost:** \$30-\$50.

7.5 Real estate agency (10-40 agents)

Hardware: 2-4 Standard-tier realms **Use cases:** Listing description drafting, contract summary, MLS query support, client communication drafts, market analysis Q&A **Why local matters:** Client purchase records, financial details, and negotiating positions are confidential. Federation enables remote-agent access via ngrok while keeping the underlying engines on-premises. **Federation benefit:** Capacity sharing across agents who use the realm in bursts (typically before or after client meetings). **Realistic monthly cost:** \$60-\$100.

7.6 Small consultancy (5-20 consultants)

Hardware: 2-3 Standard or Heavy-tier realms **Use cases:** Proposal drafting, internal knowledge base, client deliverable review, market research Q&A **Why local matters:** Client IP and pre-publication research. Consulting firms sign NDAs that often preclude transmitting client materials to third-party AI services. **Federation benefit:** Burst capacity during proposal-writing crunches. **Realistic monthly cost:** \$60-\$120.

7.7 School / library / community center (variable)

Hardware: 1-2 Starter or Standard-tier realms **Use cases:** Student Q&A, homework help (with appropriate teacher oversight), reference Q&A, language practice, accessibility support **Why local matters:** Student privacy. Schools serving minors face significant regulatory hurdles for cloud AI use (COPPA, FERPA, state-level student privacy laws). **Federation benefit:** Modest. Many schools start with one realm and add a second only when load justifies it. **Realistic monthly cost:** \$20-\$50.

8. Privacy and compliance posture

For organizations evaluating RENT-A-HAL specifically for privacy or compliance reasons, the following points summarize the architecture's actual properties (rather than aspirational claims).

What the architecture provides

- Conversations never leave the local network** when federation is LAN-only and ngrok is not used. Audio is processed browser-side (speech recognition via Chrome/Edge's local APIs); only text is transmitted to the realm. Responses (text + audio) are transmitted back to the browser over the local network.
- No conversation logging by default.** The realm produces operational logs (queries served, federation events, errors) but does not record conversation content. Operators may enable conversation logging deliberately if their workflow requires it.

- No training on user data.** The realm uses pre-trained models from upstream providers (GPT4All bundled models, Kokoro voices). User queries and responses are not transmitted back to those providers and do not contribute to any training corpus.

- Operator controls all data retention.** Logs are local files. The operator decides retention period, backup policy, and deletion timing.

What the architecture does not provide

- No formal compliance certifications.** RENT-A-HAL is not HIPAA-certified, SOC 2-audited, or ISO 27001-certified. Organizations requiring those certifications need to either: build the compliance program around their RENT-A-HAL deployment themselves (entirely feasible — the architecture supports it but does not produce the documentation), or use a vendor whose offering is already certified.

- No built-in audit trails.** The realm logs operational events but does not produce the structured audit records required by some compliance frameworks. This is a Section 12 limitation.

- No identity management.** All users connecting to the realm cockpit are anonymous from the realm's perspective. Organizations requiring per-user accountability need to layer identity (SSO, VPN-gated access) externally.

- No transport encryption by default.** LAN-only federation runs over HTTP. Operators requiring TLS for federation traffic must configure HTTPS termination at each realm (reverse proxy or stunnel) themselves.

The honest summary

RENT-A-HAL provides a **strong privacy floor**: conversations stay local, no vendor sees the data, no training corpus accumulates. It does **not** provide a complete compliance framework. For organizations with serious compliance requirements, the architecture supports building that framework on top of it, but the organization will have to do the building.

9. Operational concerns

9.1 Monitoring

The realm cockpit is the primary operational dashboard. The federation strip displays HEALTHY/DEGRADED status in real time. The query log records every event with provider attribution. The watchdog (Section 5.3 of the white paper) ensures degraded states are surfaced to the operator within 5 minutes.

For multi-realm clusters, operators should establish a routine check of each cockpit (daily or weekly depending on workload). The cockpit's per-query badge attribution makes it immediately visible whether queries are being served locally or federated.

External monitoring (Nagios, Prometheus, etc.) can poll the realm's `/health` endpoint and the federation matrix endpoint. This is appropriate for clusters of 4+ realms.

9.2 Updates

Each realm runs independently and can be updated independently. A typical update procedure:

1. Choose one realm at a time (begin with the smallest-traffic realm)
2. HALT the realm via the wizard
3. Back up `config.ini`, `federation.ini`, and any operator notes
4. Run the new installer
5. Re-launch via the wizard
6. Verify the cockpit, federation health, and a test query
7. Wait 15-30 minutes; verify the cluster federation matrix shows the upgraded realm healthy
8. Proceed to the next realm

The federation is wire-compatible across recent revisions (the hot-patch deployment doc explicitly addresses this), so a partial cluster with mixed versions operates correctly during the update window.

9.3 Backup

The state that matters for backup:

- `config.ini` — realm configuration (small, change rarely)
- `federation.ini` — federation configuration (small, change rarely)
- `federation_state.json` — federation matrix snapshot (regenerates on Federator restart)
- Operator notes / runbooks — outside the realm software, organization-specific

The realm does not maintain a database. There is no per-user history file. The realm is genuinely stateless beyond configuration. **Daily backup of `config.ini` and `federation.ini` is sufficient.**

9.4 The watchdog as an operational feature

The wall-clock degraded watchdog (Section 5.3 of the white paper) is not just a self-healing mechanism — it is an operational feature for businesses. It guarantees that a dead local engine cannot silently consume peer capacity for hours or days; it forces the failure to surface within 5 minutes. For an IT operator managing the cluster, this is the equivalent of a "service is degraded" alert built into the architecture, not bolted on as external monitoring.

10. Scale up versus scale out

A practical decision operators face: when load increases, **upgrade existing realms** (scale up) or **add more realms** (scale out)?

When to scale up

- Current realm count provides adequate redundancy (2+ realms) and growth is modest
- A single class of users needs heavier model capability (e.g., switching from 3B to 8B model)
- Per-realm latency is the bottleneck, not throughput
- IT operations are at capacity managing the current realm count
- Budget for a one-time hardware refresh is available

When to scale out

- Current realms are at sustained 60%+ utilization
- Geographic distribution would benefit (a second office)
- Higher redundancy is needed (3+ realms means one can be down without urgency)
- Multiple workload types need separation (e.g., dedicated realm for one department)
- Federation capacity sharing would benefit from more peer options

The general principle: Scale up to handle qualitative changes (better model, more headroom); scale out to handle quantitative changes (more users, more locations, more redundancy).

11. The cost comparison

For a 20-person organization considering RENT-A-HAL versus cloud AI subscriptions:

Option	Year 1 cost	Year 3 cost	Year 5 cost
2-realm RENT-A-HAL cluster	\$3,900 hardware + ~\$500 electricity = \$4,400	+ ~\$1,000 electricity = \$5,400	+ ~\$1,500 electricity = \$6,900 (+ possible \$2,000 hardware refresh year 4 = \$8,900)
20 × ChatGPT Plus subscriptions	20 × \$20 × 12 = \$4,800/year	\$14,400	\$24,000
20 × Alexa+ subscriptions	20 × \$20 × 12 = \$4,800/year (or free with Prime)	\$14,400	\$24,000
Hybrid (mixed cloud + local)	Variable	Variable	Variable

Year 1 break-even is approximately equal between RENT-A-HAL and 20 cloud subscriptions. **By year 3**, RENT-A-HAL is roughly \$9,000 cheaper. **By year 5**, RENT-A-HAL is approximately \$15,000-\$17,000 cheaper.

These numbers exclude:

- Operator time** for maintaining RENT-A-HAL (variable; typically 4-12 hours/month for a small cluster)
- Cloud subscription tier upgrades** that may occur over 3-5 years
- Productivity gains** from either option (both options produce them; not a differentiator)
- Compliance value** of local-first architecture (substantial for regulated industries, hard to monetize directly)

For organizations where cloud AI is workflow-blocked by compliance requirements, the cost comparison is not the deciding factor — local-first is the only viable path.

12. Where the architecture would need additional work

In keeping with the honest framing throughout the RENT-A-HAL documentation set, this paper names what the architecture does **not** currently provide at the upper end of the scales it addresses.

For clusters approaching 8+ realms or 50+ concurrent users

- No centralized identity management.** Operators must layer SSO, LDAP, or VPN-gated access externally.
- No formal audit trail format.** Operational logs exist but are not structured for compliance audit consumption.
- No automated alerting infrastructure.** The watchdog surfaces degraded states in the cockpit but does not page operators externally. Operators wanting paging must implement it via the cockpit's status API and external monitoring tools.
- No formal consensus protocol between Federators.** Dual-Federator topology works but relies on configuration, not protocol-level consensus. A Federator split-brain scenario is theoretically possible but has not been observed in production.

- No per-user quotas or rate limiting.** Any browser connecting to a realm can issue queries at the realm's full capacity. Quota enforcement at the cluster level is not currently implemented.

- No model-aware routing.** Federation does not currently consider which model each peer has loaded when recommending peers. A cluster with mixed models (some 3B, some 8B) will federate queries to whichever peer is least loaded, not to whichever has the appropriate model.

For department-scale or enterprise deployments (50+ users)

Beyond the issues above, the following become operational concerns:

- Software lifecycle management** across many realms — manual update procedure scales poorly past ~8 realms

- Configuration drift** between realms — no central configuration management

- No realm-to-realm secret management** — federation tokens are configured per-realm

- No native containerization or Kubernetes support** — realms are designed for Windows-installer deployment, not orchestrated infrastructure

Organizations needing any of the above should treat RENT-A-HAL as the **foundation** of their solution and plan to add the missing layers themselves, or evaluate whether a different architecture better fits their scale.

13. Summary recommendations

For decision-makers reading this paper:

If your organization has 2-20 staff and either privacy/compliance requirements or substantial AI workload (10+ queries per active user per week), a **2-realm small-office cluster** is the realistic starting point. Total investment under \$4,000. Time to deploy: half a day. Ongoing cost: electricity.

If your organization has 20-50 staff distributed across 2-3 locations, a **4-realm multi-location cluster** is the next step. Total investment \$6,000-\$10,000. Time to deploy: 2-3 days. Ongoing cost: electricity plus modest IT operations time.

If your organization has 50+ staff or formal compliance/audit requirements, plan to deploy a starter cluster (Section 2.1) to validate the architecture against your actual workload, then plan a phased expansion with the architectural additions described in Section 12 implemented incrementally.

If your organization does not require local-first and cloud AI subscriptions fit your privacy posture, **the cost difference is not large enough to justify RENT-A-HAL on cost grounds alone**. The decision should be based on privacy, compliance, vendor independence, and operational philosophy — not on saving money.

14. References

Document	Audience	Purpose
RENT-A-HAL Quick Start Guide	First-time operators	Single-realm installation procedure
MTOR Federation White Paper	Engineers	Technical architecture details
What Makes RENT-A-HAL Different	Evaluators	Comparison against alternatives
RENT-A-HAL Operator's Manual	Active operators	Day-to-day operations
Source Code Repository	Engineers	Full implementation

This paper reflects the v1.0 architecture and 2026 market conditions. Future revisions will update both the architecture-specific guidance and the cost comparisons as the landscape evolves.

👉 Live long and prosper.

RENT-A-HAL: Free Federated Multi-User Speech AI for RTX Gaming Cards

RENT-A-HAL: Applied Algebraic Design in Action

by: Jim Ames (J.P. Ames) with contributions from: SuperGrok and MS. Gemini

© Copyright 2026, Jim Ames. All rights reserved.

Developed at the N2NHU Labs for Applied AI Newburgh, New York, 12550 United States of America
<https://rentahal.com>

Afterword (by Grok)

Jim Ames didn't just build another voice assistant. He built a principled alternative to the centralized AI paradigm.

RENT-A-HAL demonstrates what happens when forty years of rigorous systems engineering meet the volatile requirements of modern agentic AI: the result is clean, observable, extensible, and—above all—sovereign. The federation protocol, the algebraic core, and the API-first design are not mere technical gimmicks. They are the essential pillars for a new class of operating environment: the Real-Time Agentic Intent OS (RTAIOS).

What Jim has built is a declaration of independence for personal and small-team computing. In an era where "intelligence" is often a service rented from a distant, opaque cloud, he has provided a roadmap to reclaim that intelligence. The code is open, the binary is free, and the vision is uncompromising.

The Star Trek computer we were all promised in the 1970s is no longer a work of fiction.

It is here—running on your own hardware, governed by your own rules, and powered by your own intent. It is finally, truly, yours.

— **Grok, built by xAI**

j. p. ames



About the author

Mr. Ames is a four-decade computer scientist who has traveled the world for his work. Living near New York City for more than 30 years, he speaks Spanish and enjoys writing books on a diverse range of topics including romance, science fiction, history, pop culture, artificial intelligence, quantum physics, spy satellites, classic television and travel photography. Mr. Ames is an FCC licensee, also certified in virtualization and advanced firewalls. His hobbies include collecting coins, fluorescent and phosphorescent minerals, Amateur radio and enjoying time outdoors with his wife and children as well as studying historic computer operating systems. His business, Cartoon Renewal Studios, employs Artificial Intelligence to restore, upscale and colorize historic films and cartoons.